

L1-001. Hello World [水题]

这道超级简单的题目没有任何输入。

你只需要在一行中输出著名短句“Hello World!”就可以了。

```
1  #include <iostream>
2  using namespace std;
3  int main() {
4      cout << "Hello World!";
5      return 0;
6  }
```

L1-002. 打印沙漏 [图形打印]

本题要求你写个程序把给定的符号打印成沙漏的形状。例如给定17个“*”，要求按下列格式打印

*

所谓“沙漏形状”，是指每行输出奇数个符号；各行符号中心对齐；相邻两行符号数差2；符号数先从大到小顺序递减到1，再从小到大顺序递增；首尾符号数相等。

给定任意N个符号，不一定能正好组成一个沙漏。要求打印出的沙漏能用掉尽可能多的符号。

输入格式：

输入在一行给出1个正整数N（ ≤ 1000 ）和一个符号，中间以空格分隔。

输出格式：

首先打印出由给定符号组成的最大的沙漏形状，最后在一行中输出剩下没用掉的符号数。

输入样例：

19 *

输出样例：

*

```

1  #include <iostream>
2  using namespace std;
3  int main() {
4      int N, row = 0;
5      char c;
6      cin >> N >> c;
7      for (int i = 0; i < N; i++) {
8          if ((2 * i * (i + 2) + 1) > N) {
9              row = i - 1;
10             break;
11         }
12     }
13     for (int i = row; i >= 1; i--) {
14         for (int k = row - i; k >= 1; k--) cout << " ";
15         for (int j = i * 2 + 1; j >= 1; j--) cout << c;
16         cout << endl;
17     }
18     for (int i = 0; i < row; i++) cout << " ";
19     cout << c << endl;
20     for (int i = 1; i <= row; i++) {
21         for (int k = row - i; k >= 1; k--) cout << " ";
22         for (int j = i * 2 + 1; j >= 1; j--) cout << c;
23         cout << endl;
24     }
25     cout << (N - (2 * row * (row + 2) + 1));
26     return 0;
27 }

```

L1-003. 个位数统计 [字符串处理]

给定一个k位整数 $N = d_{k-1} \times 10^{k-1} + \dots + d_1 \times 10^1 + d_0$ ($0 \leq d_i \leq 9, i=0, \dots, k-1, d_{k-1} > 0$)，请编写程序统计每种不同的个位数字出现的次数。例如：给定 $N = 100311$ ，则有2个0，3个1，和1个3。

输入格式：

每个输入包含1个测试用例，即一个不超过1000位的正整数N。

输出格式：

对N中每一种不同的个位数字，以D:M的格式在一行中输出该位数字D及其在N中出现的次数M。要求按D的升序输出。

输入样例：

100311

输出样例：

0:2

1:3

3:1

```
1  #include <iostream>
2  #include <string>
3  using namespace std;
4  int main() {
5      string s;
6      cin >> s;
7      int len = s.length();
8      int a[10] = {0};
9      for (int i = 0; i < len; i++) a[s[i] - '0']++;
10     for (int i = 0; i < 10; i++) {
11         if (a[i] != 0) cout << i << ":" << a[i] << endl;
12     }
13     return 0;
14 }
```

L1-004. 计算摄氏温度 [数学问题]

给定一个华氏温度F，本题要求编写程序，计算对应的摄氏温度C。计算公式： $C = 5 \times (F - 32) / 9$ 。题目保证输入与输出均在整型范围内。

输入格式：

输入在一行中给出一个华氏温度。

输出格式：

在一行中按照格式“Celsius = C”输出对应的摄氏温度C的整数值。

输入样例：

150

输出样例：

Celsius = 65

```
1  #include <iostream>
2  using namespace std;
3  int main() {
4      int f;
5      cin >> f;
6      int c = 5 * (f - 32) / 9;
7      cout << "Celsius = " << c;
8      return 0;
9  }
```

L1-005. 考试座位号 [查找元素]

每个PAT考生在参加考试时都会被分配两个座位号，一个是试机座位，一个是考试座位。正常情况下，考生在入场时先得到试机座位号码，入座进入试机状态后，系统会显示该考生的考试座位号码，考试时考生需要换到考试座位就座。但有些考生迟到了，试机已经结束，他们只能拿着领到的试机座位号码求助于你，从后台查出他们的考试座位号码。

输入格式：

输入第一行给出一个正整数N (≤ 1000)，随后N行，每行给出一个考生的信息：“准考证号 试机座位号 考试座位号”。其中准考证号由14位数字组成，座位从1到N编号。输入保证每个人的准考证号都不同，并且任何时候都不会把两个人分配到同一个座位上。

考生信息之后，给出一个正整数M ($\leq N$)，随后一行中给出M个待查询的试机座位号码，以空格分隔。

输出格式：

对应每个需要查询的试机座位号码，在一行中输出对应考生的准考证号和考试座位号码，中间用1个空格分隔。

输入样例：

```
4
10120150912233 2 4
10120150912119 4 1
10120150912126 1 3
10120150912002 3 2
2
3 4
```

输出样例：

```
10120150912002 2
10120150912119 1
```

```
1  #include <iostream>
2  #include <vector>
3  using namespace std;
4  struct stu {
5      string s;
6      int one, two;
7  };
8  int main() {
9      int n, m, temp;
10     cin >> n;
11     vector<stu> a(n);
12     for (int i = 0; i < n; i++) {
13         cin >> a[i].s >> a[i].one >> a[i].two;
14     }
```

```

15     cin >> m;
16     for (int i = 0; i < m; i++) {
17         cin >> temp;
18         for (int j = 0; j < n; j++) {
19             if (a[j].one == temp) {
20                 cout << a[j].s << " " << a[j].two << endl;
21                 break;
22             }
23         }
24     }
25     return 0;
26 }

```

L1-006. 连续因子 [逻辑题]

一个正整数N的因子中可能存在若干连续的数字。例如630可以分解为 $3*5*6*7$ ，其中5、6、7就是3个连续的数字。给定任一正整数N，要求编写程序求出最长连续因子的个数，并输出最小的连续因子序列。

输入格式：

输入在一行中给出一个正整数N（ $1 < N < 231$ ）。

输出格式：

首先在第1行输出最长连续因子的个数；然后在第2行中按“因子1*因子2*.....*因子k”的格式输出最小的连续因子序列，其中因子按递增顺序输出，1不算在内。

输入样例：

630

输出样例：

3

5*6*7

分析：

[Update v2.0] 由github用户littlesevenmo提供的更高效的解法：

不用算连续因子最多不会超过12个，也不需要三重循环，两重循环即可，直接去计算当前部分乘积能不能整除N，代码如下：

```

1  #include <iostream>
2  #include <cmath>
3  using namespace std;
4  long int num, temp;
5  int main(){
6      cin >> num;
7      int first = 0, len = 0, maxn = sqrt(num);
8      for (int i = 2; i <= maxn; i++){
9          int j; temp = 1;

```

```

10         for (j = i; j <= maxn; j++){
11             temp *= j;
12             if (num % temp != 0) break;
13         }
14         if (j - i > len){
15             len = j - i;
16             first = i;
17         }
18     }
19     if (first == 0) cout << 1 << endl << num;
20     else {
21         cout << len << endl;
22         for (int i = 0; i < len; i++){
23             cout << first + i;
24             if (i != len - 1) cout << '*';
25         }
26     }
27     return 0;
28 }

```

L1-007. 念数字 [字符串处理]

输入一个整数，输出每个数字对应的拼音。当整数为负数时，先输出“fu”字。十个数字对应的拼音如下：

0: ling

1: yi

2: er

3: san

4: si

5: wu

6: liu

7: qi

8: ba

9: jiu

输入格式：

输入在一行中给出一个整数，如： 1234 。

提示：整数包括负数、零和正数。

输出格式：

在一行中输出这个整数对应的拼音，每个数字的拼音之间用空格分开，行末没有最后的空格。如 yi er san si。

输入样例：

-600

输出样例：

fu liu ling ling

```
1  #include <iostream>
2  using namespace std;
3  int main() {
4      string chinese[10] = {"ling", "yi", "er", "san", "si", "wu", "liu",
5      "qi", "ba", "jiu"};
6      string s;
7      cin >> s;
8      int len = s.length();
9      int flag = s[0] == '-' ? 1 : 0;
10     if(flag == 1) cout << "fu";
11     for(int i = flag; i < len; i++) {
12         if(i == 0) {
13             cout << chinese[s[i] - '0'];
14             continue;
15         }
16         cout << " " << chinese[s[i] - '0'];
17     }
18     return 0;
19 }
```

L1-008. 求整数段和 [水题]

给定两个整数A和B，输出从A到B的所有整数以及这些数的和。

输入格式：

输入在一行中给出2个整数A和B，其中 $-100 \leq A \leq B \leq 100$ ，其间以空格分隔。

输出格式：

首先顺序输出从A到B的所有整数，每5个数字占一行，每个数字占5个字符宽度，向右对齐。最后在一行中输出全部数字的和。

输入样例：

-3 8

输出样例：

-3 -2 -1 0 1

2 3 4 5 6

7 8

Sum = 30

```

1  #include <iostream>
2  #include <cstdio>
3  using namespace std;
4  int main() {
5      int a, b;
6      cin >> a >> b;
7      int temp = a, sum = 0;
8      while(temp <= b) {
9          for(int j = 0; j < 5 && temp <= b; j++) {
10             printf("%5d", temp);
11             sum += temp;
12             temp++;
13         }
14         cout << endl;
15     }
16     cout << "Sum = " << sum;
17     return 0;
18 }

```

L1-009. N个数求和 [分数的四则运算]

输入格式：

输入第一行给出一个正整数N (≤ 100)。随后一行按格式“a1/b1 a2/b2 ...”给出N个有理数。题目保证所有分子和分母都在长整型范围内。另外，负数的符号一定出现在分子前面。

输出格式：

输出上述数字和的最简形式——即将结果写成“整数部分 分数部分”，其中分数部分写成“分子/分母”，要求分子小于分母，且它们没有公因子。如果结果的整数部分为0，则只输出分数部分。

输入样例1：

5

2/5 4/15 1/30 -2/60 8/3

输出样例1：

3 1/3

输入样例2：

2

4/3 2/3

输出样例2：

2

输入样例3：

3

1/3 -1/6 1/8

输出样例3:

7/24

分析：先根据分数加法的公式累加，后分离出整数部分和分数部分～

分子和分母都在长整型内，所以不能用int存储，否则有一个测试点不通过～

一开始一直是浮点错误，按理来说应该是出现了/0或者%0的情况，找了半天也不知道错在哪里～

后来注意到应该在累加的时候考虑是否会超出long long的范围，所以在累加每一步之前进行分子分母的约分处理，然后就AC了～

应该还要考虑整数和小数部分都为0时候输出0的情况，但是测试用例中不涉及，所以如果没有最后两句也是可以AC的～

```
1  #include <stdio>
2  #include <stdlib>
3  using namespace std;
4  long long gcd(long long a, long long b) {return b == 0 ? a : gcd(b, a %
   b);}
5  int main() {
6      long long n, a, b, suma = 0, sumb = 1, gcdvalue;
7      scanf("%lld", &n);
8      for(int i = 0; i < n; i++) {
9          scanf("%lld/%lld", &a, &b);
10         gcdvalue = (suma == 0 || sumb == 0) ? 1 : gcd(abs(suma),
abs(sumb));
11         sumb = sumb / gcdvalue;
12         suma = suma / gcdvalue;
13         gcdvalue = (a == 0 || b == 0) ? 1 : gcd(abs(a), abs(b));
14         a = a / gcdvalue;
15         b = b / gcdvalue;
16         suma = a * sumb + suma * b;
17         sumb = b * sumb;
18     }
19     long long integer = suma / sumb;
20     suma = suma - (sumb * integer);
21     gcdvalue = (suma == 0 || sumb == 0) ? 1 : gcd(abs(suma),
abs(sumb));
22     suma = suma / gcdvalue;
23     sumb = sumb / gcdvalue;
24     if(integer != 0) {
25         printf("%lld", integer);
26         if(suma != 0)
27             printf(" ");
28     }
29     if(suma != 0) {
30         printf("%lld/%lld", suma, sumb);
31     }
```

```

32     if(integer == 0 && suma == 0)
33         printf("0");
34     return 0;
35 }

```

L1-010. 比较大小 [水题]

输入格式：

输入在一行中给出3个整数，其间以空格分隔。

输出格式：

在一行中将3个整数从小到大输出，其间以“->”相连。

输入样例：

4 2 8

输出样例：

2->4->8

分析：冒泡呀冒个泡~

```

1  #include <iostream>
2  using namespace std;
3  int main() {
4      int a, b, c;
5      cin >> a >> b >> c;
6      if(b > c) swap(b, c);
7      if(a > b) swap(a, b);
8      if(b > c) swap(b, c);
9      cout << a << "->" << b << "->" << c;
10     return 0;
11 }

```

L1-011. A-B [字符串处理]

本题要求你计算A-B。不过麻烦的是，A和B都是字符串——即从字符串A中把字符串B所包含的字符全删掉，剩下的字符组成的就是字符串A-B。

输入格式：

输入在2行中先后给出字符串A和B。两字符串的长度都不超过104，并且保证每个字符串都是由可见的ASCII码和空白字符组成，最后以换行符结束。

输出格式：

在一行中打印出A-B的结果字符串。

输入样例：

I love GPLT! It's a fun game!

aeiou

输出样例：

I lv GPLT! It's fn gm!

分析：辣么多ASCII码也在0~255之间，所以用book数组标记所有的ASCII码~如果第二个字符出现了这个ASCII码那就标记为1~然后输出的时候当book数组对应的那个ASCII为1的时候就跳过不输出~

```
1  #include <iostream>
2  using namespace std;
3  int book[256];
4  int main() {
5      string s, a;
6      getline(cin, s);
7      getline(cin, a);
8      for(int i = 0; i < a.length(); i++) book[a[i]] = 1;
9      for(int i = 0; i < s.length(); i++) {
10         if(book[s[i]] == 1) continue;
11         cout << s[i];
12     }
13     return 0;
14 }
```

L1-012. 计算指数 [水题]

真的没骗你，这道才是简单题 —— 对任意给定的不超过10的正整数n，要求你输出 2^n 。不难吧？

输入格式：

输入在一行中给出一个不超过10的正整数n。

输出格式：

在一行中按照格式“ 2^n = 计算结果”输出 2^n 的值。

输入样例：

5

输出样例：

$2^5 = 32$

```
1  #include <iostream>
2  #include <cmath>
3  using namespace std;
4  int main() {
5      int n;
6      cin >> n;
7      cout << "2^" << n << " = " << pow(2, n);
8      return 0;
9  }
```

L1-013. 计算阶乘和 [水题]

对于给定的正整数N，需要你计算 $S = 1! + 2! + 3! + \dots + N!$ 。

输入格式：

输入在一行中给出一个不超过10的正整数N。

输出格式：

在一行中输出S的值。

输入样例：

3

输出样例：

9

```
1  #include <iostream>
2  using namespace std;
3  int main() {
4      int n, ans = 0;
5      cin >> n;
6      for(int i = 1; i <= n; i++) {
7          int temp = 1;
8          for(int j = 1; j <= i; j++) temp *= j;
9          ans += temp;
10     }
11     cout << ans;
12     return 0;
13 }
```

L1-014. 简单题 [水题]

这次真的没骗你 —— 这道超级简单的题目没有任何输入。

你只需要在一行中输出事实：“This is a simple problem.”就可以了。

```
1  #include <iostream>
2  using namespace std;
3  int main() {
4      cout << "This is a simple problem.";
5      return 0;
6  }
```

L1-015. 跟奥巴马一起画方块 [图形打印]

美国总统奥巴马不仅呼吁所有人都学习编程，甚至以身作则编写代码，成为美国历史上首位编写计算机代码的总统。2014年底，为庆祝“计算机科学教育周”正式启动，奥巴马编写了很简单的计算机代码：在屏幕上画一个正方形。现在你也跟他一起画吧！

输入格式：

输入在一行中给出正方形边长N ($3 \leq N \leq 21$) 和组成正方形边的某种字符C，间隔一个空格。

输出格式：

输出由给定字符C画出的正方形。但是注意到行间距比列间距大，所以为了让结果看上去更像正方形，我们输出的行数实际上是列数的50%（四舍五入取整）。

输入样例：

10 a

输出样例：

aaaaaaaaaa

aaaaaaaaaa

aaaaaaaaaa

aaaaaaaaaa

aaaaaaaaaa

```
1  #include <iostream>
2  using namespace std;
3  int main() {
4      int N;
5      cin >> N;
6      char c;
7      cin >> c;
8      int t = N / 2 + N % 2;
9      for (int i = 0; i < t - 1; i++) {
10         for (int k = 0; k < N; k++)
11             cout << c;
12         cout << endl;
13     }
14     for (int i = 0; i < N; i++)
15         cout << c;
16     return 0;
17 }
```

L1-016. 查验身份证 [字符串处理]

一个合法的身份证号码由17位地区、日期编号和顺序编号加1位校验码组成。校验码的计算规则如下：

首先对前17位数字加权求和，权重分配为：{7, 9, 10, 5, 8, 4, 2, 1, 6, 3, 7, 9, 10, 5, 8, 4, 2}；然后将计算的和对11取模得到值Z；最后按照以下关系对应Z值与校验码M的值：

Z: 0 1 2 3 4 5 6 7 8 9 10

M: 1 0 X 9 8 7 6 5 4 3 2

现在给定一些身份证号码，请你验证校验码的有效性，并输出有问题的号码。

输入格式：

输入第一行给出正整数N (≤ 100) 是输入的身份证号码的个数。随后N行，每行给出1个18位身份证号码。

输出格式：

按照输入的顺序每行输出1个有问题的身份证号码。这里并不检验前17位是否合理，只检查前17位是否全为数字且最后1位校验码计算准确。如果所有号码都正常，则输出“All passed”。

输入样例1：

```
4
320124198808240056
12010X198901011234
110108196711301866
37070419881216001X
```

输出样例1：

```
12010X198901011234
110108196711301866
37070419881216001X
```

输入样例2：

```
2
320124198808240056
110108196711301862
```

输出样例2：

All passed

分析：输入每个身份证号码的时候逐个判断是否满足条件，如果有满足条件的就flag = 1 最后如果flag 依然是0就输出All passed~注意X和10的转换~

```
1  #include <iostream>
2  #include <string>
3  using namespace std;
4  bool func(string a);
5  int main() {
6      int n;
7      int count = 0;
8      cin >> n;
9      for (int i = 0; i < n; i++) {
10         string a;
```

```

11         cin >> a;
12         if (func(a))
13             count++;
14     }
15     if (count == 0)
16         cout << "All passed";
17     return 0;
18 }
19
20 bool func(string s) {
21     int sum = 0;
22     int a[18];
23     for (int i = 0; i < 17; i++) {
24         if (!isdigit(s[i])) {
25             cout << s << endl;
26             return false;
27         }
28         a[i] = s[i] - '0';
29     }
30     if (s[17] == 'X')
31         a[17] = 10;
32     else
33         a[17] = s[17] - '0';
34     int b[17] = {7, 9, 10, 5, 8, 4, 2, 1, 6, 3, 7, 9, 10, 5, 8, 4, 2};
35     for (int i = 0; i < 17; i++) {
36         sum = sum + a[i] * b[i];
37     }
38     sum = sum % 11;
39     int c[11] = {1, 0, 10, 9, 8, 7, 6, 5, 4, 3, 2};
40     if (c[sum] != a[17]) {
41         for (int i = 0; i < 17; i++) {
42             cout << s[i];
43         }
44         if (a[17] != 10)
45             cout << a[17] << endl;
46         else {
47             cout << "X" << endl;
48         }
49         return true;
50     }
51     else {
52         return false;
53     }
54 }

```

L1-017. 到底有多二 [数学问题]

一个整数“犯二的程度”定义为该数字中包含2的个数与其位数的比值。如果这个数是负数，则程度增加0.5倍;如果还是个偶数，则再增加1倍。

例如数字“-13142223336”是个11位数，其中有3个2，并且是负数，也是偶数，则它的犯二程度计算为： $3/11 \times 1.5 \times 2 \times 100\%$ ，约为81.82%。本题就请你计算一个给定整数到底有多二。

输入格式：

输入第一行给出一个不超过50位的整数N。

输出格式：

在一行中输出N犯二的程度，保留小数点后两位。

输入样例：

-13142223336

输出样例：

81.82%

分析：注意计算ans的时候要先把cnt转换为double类型，否则两个int型相除会被自动转换为int型数值保存~

```
1  #include <iostream>
2  #include <cstdio>
3  using namespace std;
4  int main() {
5      string s;
6      cin >> s;
7      int flag = (s[0] == '-' ? 1 : 0);
8      int len = s.length();
9      int bit = len - flag, cnt = 0;
10     for(int i = 0; i < len; i++) {
11         if(s[i] == '2') cnt++;
12     }
13     double ans = (double)cnt / bit * 100;
14     if(flag == 1) ans = ans * 1.5;
15     if((s[len-1] - '0') % 2 == 0) ans = ans * 2;
16     printf("%.2f%%", ans);
17     return 0;
18 }
```

L1-018. 大笨钟 [字符串处理]

微博上有个自称“大笨钟V”的家伙，每天敲钟催促码农们爱惜身体早点睡觉。不过由于笨钟自己作息也不是很规律，所以敲钟并不定时。一般敲钟的点数是根据敲钟时间而定的，如果正好在某个整点敲，那么“当”数就等于那个整点数；如果过了整点，就敲下一个整点数。另外，虽然一天有24小时，钟却是只在后半天敲1~12下。例如在23:00敲钟，就是“当当当当当当当当”，而到了23:01就会是“当当当当当当当当”。在午夜00:00到中午12:00期间（端点时间包括在内），笨钟是不敲的。

下面就请你写个程序，根据当前时间替大笨钟敲钟。

输入格式：

输入第一行按照“hh:mm”的格式给出当前时间。其中hh是小时，在00到23之间；mm是分钟，在00到59之间。

输出格式：

根据当前时间替大笨钟敲钟，即在一行中输出相应数量个“Dang”。如果不是敲钟期，则输出：

Only hh:mm. Too early to Dang.

其中“hh:mm”是输入的时间。

输入样例1：

19:05

输出样例1：

DangDangDangDangDangDangDangDang

输入样例2：

07:05

输出样例2：

Only 07:05. Too early to Dang.

分析：%02d的意思是输出两位int型的数，如果不足两位，在前面补0～

```
1  #include <iostream>
2  #include <cstdio>
3  using namespace std;
4  int main() {
5      int hour, min;
6      scanf("%d:%d", &hour, &min);
7      if(hour <= 12) {
8          printf("Only %02d:%02d. Too early to Dang.", hour, min);
9      } else {
10         hour -= 12;
11         for(int i = 0; i < hour; i++)
12             cout << "Dang";
13         if(min > 0)
14             cout << "Dang";
15     }
16     return 0;
17 }
```

L1-019. 谁先倒 [逻辑题]

划拳是古老中国酒文化的一个有趣的组成部分。酒桌上两人划拳的方法为：每人口中喊出一个数字，同时用手比划出一个数字。如果谁比划出的数字正好等于两人喊出的数字之和，谁就输了，输家罚一杯酒。两人同赢或两人同输则继续下一轮，直到唯一的赢家出现。

下面给出甲、乙两人的酒量（最多能喝多少杯不倒）和划拳记录，请你判断两个人谁先倒。

输入格式：

输入第一行先后给出甲、乙两人的酒量（不超过100的非负整数），以空格分隔。下一行给出一个正整数N（≤100），随后N行，每行给出一轮划拳的记录，格式为：

甲喊 甲划 乙喊 乙划

其中“喊”是喊出的数字，“划”是划出的数字，均为不超过100的正整数（两只手一起划）。

输出格式：

在第一行中输出先倒下的那个人：A代表甲，B代表乙。第二行中输出没倒的那个人喝了多少杯。题目保证有一个人倒下。注意程序处理到有人倒下就终止，后面的数据不必处理。

输入样例：

```
1 1
6
8 10 9 12
5 10 5 10
3 8 5 12
12 18 1 13
4 16 12 15
15 1 1 16
```

输出样例：

```
A
1
```

分析：一杯不倒的意思是喝两杯才会倒下=||

```
1  #include <iostream>
2  using namespace std;
3  int main() {
4      int jia, yi, n;
5      cin >> jia >> yi >> n;
6      int cnta = jia, cntb = yi;
7      int jiahan, jiahua, yihan, yihua;
8      for (int i = 0; i < n; i++) {
9          cin >> jiahan >> jiahua >> yihan >> yihua;
10         if (jiahan + yihan == jiahua && yihan + jiahan != yihua)
11             cnta--;
12         if (jiahan + yihan != jiahua && yihan + jiahan == yihua)
13             cntb--;
14         if (cnta == -1) {
15             cout << "A" << endl << yi - cntb;
16             break;
```

```

17         }
18         if(cntb == -1) {
19             cout << "B" << endl << jia - cnta;
20             break;
21         }
22     }
23     return 0;
24 }

```

L1-020. 帅到没朋友 [集合set, STL的使用]

当芸芸众生忙着在朋友圈中发照片的时候，总有一些人因为太帅而没有朋友。本题就要求你找出那些帅到没有朋友的人。

输入格式：

输入第一行给出一个正整数N (≤ 100)，是已知朋友圈的个数；随后N行，每行首先给出一个正整数K (≤ 1000)，为朋友圈中的人数，然后列出一个朋友圈内的所有人——为方便起见，每人对应一个ID号，为5位数字（从00000到99999），ID间以空格分隔；之后给出一个正整数M (≤ 10000)，为待查询的人数；随后一行中列出M个待查询的ID，以空格分隔。

注意：没有朋友的人可以是根本没安装“朋友圈”，也可以是只有自己一个人在朋友圈的人。虽然有个别自恋狂会自己把自己反复加进朋友圈，但题目保证所有K超过1的朋友圈里都至少有2个不同的人。

输出格式：

按输入的顺序输出那些帅到没朋友的人。ID间用1个空格分隔，行的首尾不得有多余空格。如果没有人太帅，则输出“No one is handsome”。

注意：同一个人可以被查询多次，但只输出一次。

输入样例1：

```

3
3 11111 22222 55555
2 33333 44444
4 55555 66666 99999 77777
8
55555 44444 10000 88888 22222 11111 23333 88888

```

输出样例1：

```

10000 88888 23333

```

输入样例2：

```

3
3 11111 22222 55555
2 33333 44444

```

4 55555 66666 99999 77777

4

55555 44444 22222 11111

输出样例2:

No one is handsome 分析: 用一个set集合s保存所有有朋友的人~如果朋友圈人数 ≥ 2 , 那么就把所有的人都添加进集合s里面~因为这些人都是有朋友的~如果朋友圈人数 $=1$, 那就只输入这个值并不做任何处理~ 对于要检验的每一个id~看看它是不是在集合s里面~如果不在a里面说明没有朋友~那就输出这个id~但是也有可能重复的id哦~所以用一个set集合ans保存所有之前已经输出的答案~每次输出之前检验是不是已经输出过了呢~输出过就不能输出了哦~ 用flag标记是不是已经输出过答案了呢~如果输出过前面就要加个小空格哦~如果最后发现还是一个都没有输出~那就说明No one is handsome啦~啦啦啦~

分析: 用一个set集合s保存所有有朋友的人~如果朋友圈人数 ≥ 2 , 那么就把所有的人都添加进集合s里面~因为这些人都是有朋友的~如果朋友圈人数 $=1$, 那就只输入这个值并不做任何处理~

对于要检验的每一个id~看看它是不是在集合s里面~如果不在a里面说明没有朋友~那就输出这个id~但是也有可能重复的id哦~所以用一个set集合ans保存所有之前已经输出的答案~每次输出之前检验是不是已经输出过了呢~输出过就不能输出了哦~

用flag标记是不是已经输出过答案了呢~如果输出过前面就要加个小空格哦~如果最后发现还是一个都没有输出~那就说明No one is handsome啦~啦啦啦~

```
1  #include <iostream>
2  #include <set>
3  using namespace std;
4  int main() {
5      int a, b, c;
6      cin >> a;
7      set<string> s, ans;
8      string str;
9      for(int i = 0; i < a; i++) {
10         cin >> b;
11         if(b >= 2) {
12             for(int j = 0; j < b; j++) {
13                 cin >> str;
14                 s.insert(str);
15             }
16         } else {
17             cin >> str;
18         }
19     }
20     cin >> c;
21     int flag = 0;
22     for(int i = 0; i < c; i++) {
```

```

23         cin >> str;
24         if(s.find(str) == s.end() && ans.find(str) == ans.end()) {
25             ans.insert(str);
26             if(flag == 1)
27                 cout << " ";
28             cout << str;
29             flag = 1;
30         }
31     }
32     if(flag == 0)
33         cout << "No one is handsome";
34     return 0;
35 }

```

L1-021. 重要的话说三遍 [水题]

这道超级简单的题目没有任何输入。

你只需要把这句很重要的话 —— “I'm gonna WIN!”——连续输出三遍就可以了。

注意每遍占一行，除了每行的回车不能有任何多余字符。

```

1  #include <iostream>
2  using namespace std;
3  int main() {
4      cout << "I'm gonna WIN!" << endl << "I'm gonna WIN!" << endl << "I'm
gonna WIN!";
5      return 0;
6  }

```

L1-022. 奇偶分家 [逻辑判断]

给定N个正整数，请统计奇数和偶数各有多少个？

输入格式：

输入第一行给出一个正整N（≤ 1000）；第2行给出N个正整数，以空格分隔。

输出格式：

在一行中先后输出奇数的个数、偶数的个数。中间以1个空格分隔。

输入样例：

9

88 74 101 26 15 0 34 22 77

输出样例：

3 6

```

1  #include <iostream>

```

```

2   using namespace std;
3   int main() {
4       int n, cnta = 0, cntb = 0;
5       cin >> n;
6       for(int i = 0; i < n; i++) {
7           int temp;
8           cin >> temp;
9           if(temp % 2 == 0)
10              cntb++;
11          else
12              cnta++;
13      }
14      cout << cnta << " " << cntb;
15      return 0;
16  }

```

L1-023. 输出GPLT [水题]

给定一个长度不超过10000的、仅由英文字母构成的字符串。请将字符重新调整顺序，按“GPLTGPLT....”这样的顺序输出，并忽略其它字符。当然，四种字符（不区分大小写）的个数不一定是一样多的，若某种字符已经输出完，则余下的字符仍按GPLT的顺序打印，直到所有字符都被输出。

输入格式：

输入在一行中给出一个长度不超过10000的、仅由英文字母构成的非空字符串。

输出格式：

在一行中按题目要求输出排序后的字符串。题目保证输出非空。

输入样例：

pcTclnGloRgLrtLhgljkLhGFauPewSKgt

输出样例：

GPLTGPLTGLTGLGLL

分析：把字符串每个数字转换为大写后统计有多少个GPLT这四个字符，把它们存储到c[4]数组中～分别对应GPLT四个字符的个数～然后按顺序输出～

```

1   #include <iostream>
2   #include <cctype>
3   int c[4];
4   using namespace std;
5   int main() {
6       string s;
7       cin >> s;
8       for(int i = 0; i < s.length(); i++) {
9           if(toupper(s[i]) == 'G') {
10              c[0]++;
11          } else if(toupper(s[i]) == 'P') {

```

```

12         c[1]++;
13     } else if(toupper(s[i]) == 'L') {
14         c[2]++;
15     } else if(toupper(s[i]) == 'T') {
16         c[3]++;
17     }
18 }
19 while(c[0] != 0 || c[1] != 0 || c[2] != 0 || c[3] != 0) {
20     for(int i = 0; i < 4; i++) {
21         if(c[i] != 0) {
22             c[i]--;
23             if(i == 0)
24                 cout << "G";
25             else if(i == 1)
26                 cout << "P";
27             else if(i == 2)
28                 cout << "L";
29             else
30                 cout << "T";
31         }
32     }
33 }
34 return 0;
35 }

```

L1-024. 后天 [日期处理]

如果今天是星期三，后天就是星期五；如果今天是星期六，后天就是星期一。我们用数字1到7对应星期一到星期日。给定某一天，请你输出那天的“后天”是星期几。

输入格式：

输入第一行给出一个正整数D（ $1 \leq D \leq 7$ ），代表星期里的某一天。

输出格式：

在一行中输出D天的后天是星期几。

输入样例：

3

输出样例：

5

```

1  #include <iostream>
2  using namespace std;
3  int main() {
4      int a;
5      cin >> a;
6      int b = (a + 2 > 7) ? (a - 5) : (a + 2);
7      cout << b;
8      return 0;
9  }

```

L1-025. 正整数A+B [字符串处理]

本题的目标很简单，就是求两个正整数A和B的和，其中A和B都在区间[1,1000]。稍微有点麻烦的是，输入并不保证是两个正整数。

输入格式：

输入在一行给出A和B，其间以空格分开。问题是A和B不一定是满足要求的正整数，有时候可能是超出范围的数字、负数、带小数点的实数、甚至是一堆乱码。

注意：我们把输入中出现的第1个空格认为是A和B的分隔。题目保证至少存在一个空格，并且B不是一个空字符串。

输出格式：

如果输入的确是两个正整数，则按格式“A + B = 和”输出。如果某个输入不合要求，则在相应位置输出“?”，显然此时和也是“?”。

输入样例1：

123 456

输出样例1：

123 + 456 = 579

输入样例2：

22. 18

输出样例2：

? + 18 = ?

输入样例3：

-100 blabla bla...33

输出样例3：

? + ? = ?

分析：用string t接收一行的字符串，然后以第一个空格分离成字符串a和b。

islegal判定字符串a、b是否合法，不合法的赋值“?”


```

1  #include <iostream>
2  #include <cctype>
3  using namespace std;
4  bool islegal(string s) {
5      if(s.length() == 0)
6          return false;
7      for(int i = 0; i < s.length(); i++) {
8          if(!isdigit(s[i])) return false;
9      }
10     int temp = stoi(s);
11     if(temp < 1 || temp > 1000) return false;
12     return true;
13 }
14 int main() {
15     string a, b;
16     string t;
17     getline(cin, t);
18     for(int i = 0; i < t.length(); i++) {
19         if(t[i] == ' ') {
20             a = t.substr(0, i);
21             b = t.substr(i + 1, t.size() - i - 1);
22             break;
23         }
24     }
25     if(!islegal(a)) a = "?";
26     if(!islegal(b)) b = "?";
27     cout << a << " + " << b << " = ";
28     if(a != "?" && b != "?") {
29         int atemp = stoi(a), btemp = stoi(b);
30         cout << atemp + btemp;
31     } else {
32         cout << "?";
33     }
34     return 0;
35 }

```

L1-026. I Love GPLT [水题]

这道超级简单的题目没有任何输入。

你只需要把这句很重要的话——“I Love GPLT”——竖着输出就可以了。

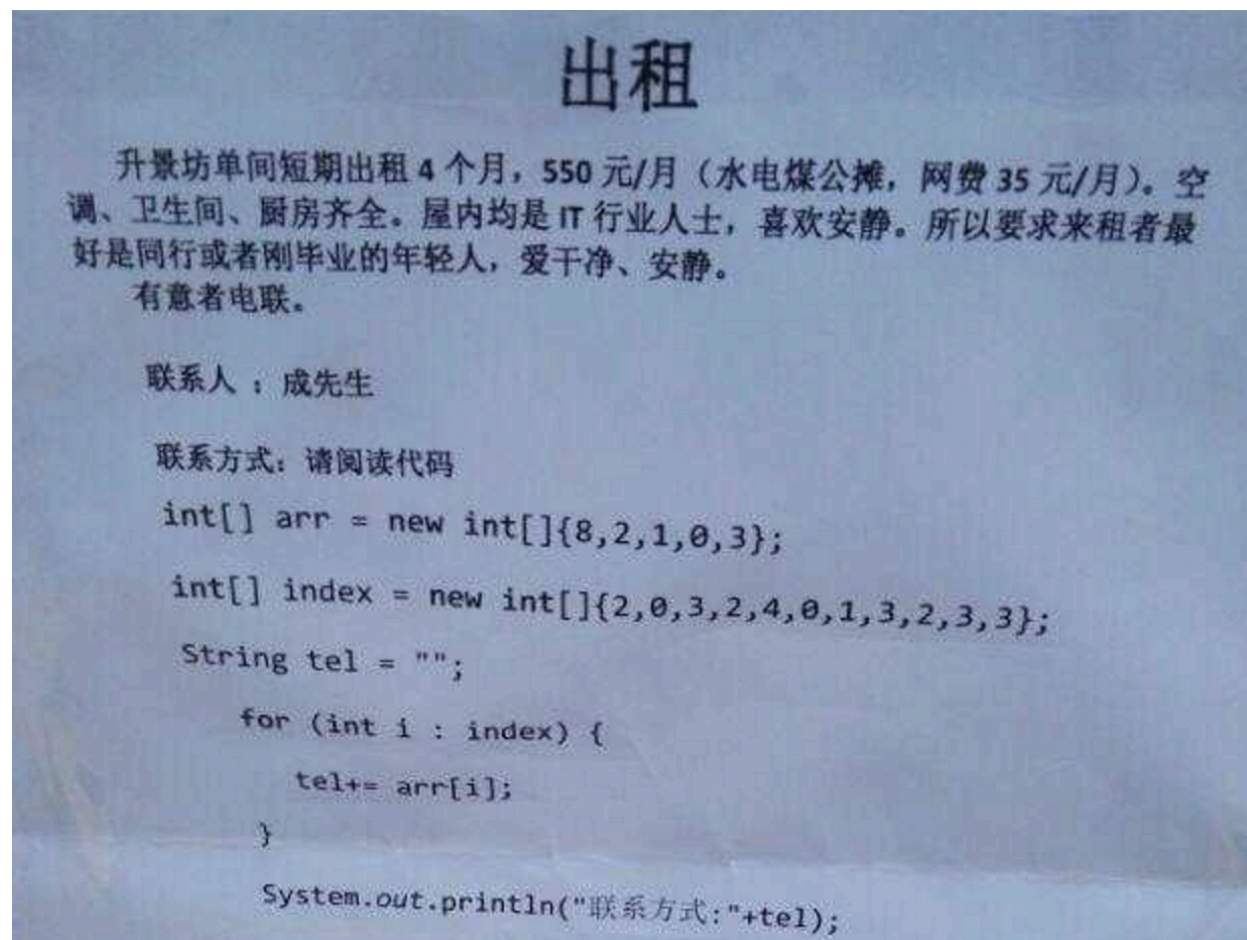
所谓“竖着输出”，是指每个字符占一行（包括空格），即每行只能有1个字符和回车。

```

1  #include <iostream>
2  using namespace std;
3  int main() {
4      string s = "I Love GPLT";
5      for(int i = 0; i < s.length(); i++) {
6          cout << s[i] << endl;
7      }
8      return 0;
9  }

```

L1-027. 出租 [STL的使用, vector]



分析：用int a[10]对号码每一位进行桶排序。逆序将a的下标存入不定长数组vector arr中，再遍历原号码生成index数组，最后输出。

```

1  #include <iostream>
2  #include <vector>
3  using namespace std;
4  int main() {
5      string s;
6      cin >> s;
7      int a[10] = {0};
8      for(int i = 0; i < 11; i++) {
9          a[s[i] - '0']++;
10     }

```

```

11     vector<int> arr;
12     for(int i = 9; i >= 0; i--) {
13         if(a[i] != 0) {
14             arr.push_back(i);
15         }
16     }
17     vector<int> index;
18     for(int i = 0; i < 11; i++) {
19         for(int j = 0; j < arr.size(); j++) {
20             if((s[i] - '0') == arr[j]) {
21                 index.push_back(j);
22             }
23         }
24     }
25     cout << "int[] arr = new int[]{" << arr[0];
26     for(int i = 1; i < arr.size(); i++) {
27         cout << "," << arr[i];
28     }
29     cout << "};" << endl << "int[] index = new int[]{" << index[0];
30     for(int i = 1; i < index.size(); i++) {
31         cout << "," << index[i];
32     }
33     cout << "};";
34     return 0;
35 }

```

L1-028. 判断素数 [素数]

本题的目标很简单，就是判断一个给定的正整数是否素数。

输入格式：

输入在第一行给出一个正整数N（≤10），随后N行，每行给出一个小于231的需要判断的正整数。

输出格式：

对每个需要判断的正整数，如果它是素数，则在一行中输出“Yes”，否则输出“No”。

输入样例：

```

2
11
111

```

输出样例：

```

Yes
No

```

```

1  #include <iostream>
2  #include <cmath>
3  using namespace std;
4  bool isprime(int a) {
5      if (a == 1) return false;
6      for (int i = 2; i <= sqrt(a); i++) {
7          if (a % i == 0) return false;
8      }
9      return true;
10 }
11 int main() {
12     int n;
13     cin >> n;
14     for (int i = 0; i < n; i++) {
15         int temp;
16         cin >> temp;
17         if (isprime(temp))
18             cout << "Yes" << endl;
19         else
20             cout << "No" << endl;
21     }
22     return 0;
23 }

```

L1-029. 是不是太胖了 [简单逻辑]

据说一个人的标准体重应该是其身高（单位：厘米）减去100、再乘以0.9所得到的公斤数。已知市斤是公斤的两倍。现给定某人身高，请你计算其标准体重应该是多少？（顺便也悄悄给自己算一下吧.....）

输入格式：

输入第一行给出一个正整数H（ $100 < H \leq 300$ ），为某人身高。

输出格式：

在一行中输出对应的标准体重，单位为市斤，保留小数点后1位。

输入样例：

169

输出样例：

124.2

```

1  #include <iostream>
2  using namespace std;
3  int main() {
4      float height;
5      cin >> height;
6      float ans = (height - 100) * 0.9 * 2;
7      printf("%.1f", ans);
8      return 0;
9  }

```

L1-030. 一帮一 [排序]

“一帮一学习小组”是中小学中常见的学习组织方式，老师把学习成绩靠前的学生跟学习成绩靠后的学生排在一组。本题就请你编写程序帮助老师自动完成这个分配工作，即在得到全班学生的排名后，在当前尚未分组的学生中，将名次最靠前的学生与名次最靠后的异性学生分为一组。

输入格式：

输入第一行给出正偶数N (≤ 50)，即全班学生的人数。此后N行，按照名次从高到低的顺序给出每个学生的性别（0代表女生，1代表男生）和姓名（不超过8个英文字母的非空字符串），其间以1个空格分隔。这里保证本班男女比例是1:1，并且没有并列名次。

输出格式：

每行输出一组两个学生的姓名，其间以1个空格分隔。名次高的学生在前，名次低的学生在后。小组的输出顺序按照前面学生的名次从高到低排列。

输入样例：

```

8
0 Amy
1 Tom
1 Bill
0 Cindy
0 Maya
1 John
1 Jack
0 Linda

```

输出样例：

```

Amy Jack
Tom Linda
Bill Maya
Cindy John

```

分析：先把输入数据存入结构体数组。然后用两个指针j、k（分别代表男女）从末尾开始扫描。找到没有被使用过的异性。

```
1  #include <iostream>
2  #include <vector>
3  using namespace std;
4  struct node {
5      int a;
6      string s;
7  };
8  int main() {
9      int n;
10     cin >> n;
11     vector<node> stu;
12     for(int i = 0; i < n; i++) {
13         node temp;
14         cin >> temp.a >> temp.s;
15         stu.push_back(temp);
16     }
17     int j = n - 1, k = n - 1;
18     for(int i = 0; i < n/2; i++) {
19         if(stu[i].a == 1) {
20             while(stu[j].a == stu[i].a) j--;
21             cout << stu[i].s << " " << stu[j].s << endl;
22             j--;
23         } else {
24             while(stu[k].a == stu[i].a) k--;
25             cout << stu[i].s << " " << stu[k].s << endl;
26             k--;
27         }
28     }
29     return 0;
30 }
```

L1-031. 到底是不是太胖了 [简单逻辑]

据说一个人的标准体重应该是其身高（单位：厘米）减去100、再乘以0.9所得到的公斤数。真实体重与标准体重误差在10%以内都是完美身材（即 $|\text{真实体重} - \text{标准体重}| < \text{标准体重} \times 10\%$ ）。已知市斤是公斤的两倍。现给定一群人的身高和实际体重，请你告诉他们是否太胖或太瘦了。

输入格式：

输入第一行给出一个正整数N（ ≤ 20 ）。随后N行，每行给出两个整数，分别是一个人的身高H（ $120 < H < 200$ ；单位：厘米）和真实体重W（ $50 < W \leq 300$ ；单位：市斤），其间以空格分隔。

输出格式：

为每个人输出一行结论：如果是完美身材，输出“You are wan mei!”；如果太胖了，输出“You are tai pang le!”；否则输出“You are tai shou le!”。

输入样例：

3

169 136

150 81

178 155

输出样例：

You are wan mei!

You are tai shou le!

You are tai pang le!

```
1  #include <iostream>
2  #include <cmath>
3  using namespace std;
4  int main() {
5      int n;
6      cin >> n;
7      for(int i = 0; i < n; i++) {
8          float height, weight;
9          cin >> height >> weight;
10         float stan = (height - 100) * 0.9 * 2;
11         if(abs(stan - weight) < stan * 0.1) {
12             cout << "You are wan mei!" << endl;
13         } else if(stan > weight) {
14             cout << "You are tai shou le!" << endl;
15         } else {
16             cout << "You are tai pang le!" << endl;
17         }
18     }
19     return 0;
20 }
```

L1-032. Left-pad [字符串处理]

根据新浪微博上的消息，有一位开发者不满NPM（Node Package Manager）的做法，收回了自己的开源代码，其中包括一个叫left-pad的模块，就是这个模块把javascript里面的React/Babel干瘫痪了。这是个什么样的模块？就是在字符串前填充一些东西到一定的长度。例如用“*”去填充字符串“GPLT”，使之长度为10，调用left-pad的结果就应该是“*****GPLT”。Node社区曾经对left-pad紧急发布了一个替代，被严重吐槽。下面就请你来实现一下这个模块。

输入格式：

输入在第一行给出一个正整数N（ ≤ 104 ）和一个字符，分别是填充结果字符串的长度和用于填充的字符，中间以1个空格分开。第二行给出原始的非空字符串，以回车结束。

输出格式：

在一行中输出结果字符串。

输入样例1:

15 _

I love GPLT

输出样例1:

I love GPLT

输入样例2:

4 *

this is a sample for cut

输出样例2:

cut

分析：需要的长度少于字符串长度的时候，使用substr输出，需要的长度大于字符串长度的时候，在前面补字符

```
1  #include <iostream>
2  #include <string.h>
3  using namespace std;
4  int main() {
5      int cnt;
6      char c;
7      string s;
8      cin >> cnt >> c;
9      getchar();
10     getline(cin, s);
11     int len = s.length();
12     if(len >= cnt) {
13         cout << s.substr(s.size() - cnt);
14     } else {
15         for(int i = 0; i < cnt - len; i++) {
16             cout << c;
17         }
18         cout << s;
19     }
20     return 0;
21 }
```

L1-033. 出生年 [STL, 集合set]

以上是新浪微博中一奇葩贴：“我出生于1988年，直到25岁才遇到4个数字都不相同的年份。”也就是说，直到2013年才达到“4个数字都不相同”的要求。本题请你根据要求，自动填充“我出生于y年，直到x岁才遇到n个数字都不相同的年份”这句话。

输入格式：

输入在一行中给出出生年份y和目标年份中不同数字的个数n，其中y在[1, 3000]之间，n可以是2、或3、或4。注意不足4位的年份要在前面补零，例如公元1年被认为是0001年，有2个不同的数字0和1。

输出格式：

根据输入，输出x和能达到要求的年份。数字间以1个空格分隔，行首尾不得有多余空格。年份要按4位输出。注意：所谓“n个数字都不相同”是指不同的数字正好是n个。如“2013”被视为满足“4位数字都不同”的条件，但不被视为满足2位或3位数字不同的条件。

输入样例1：

1988 4

输出样例1：

25 2013

输入样例2：

1 2

输出样例2：

0 0001

分析：对于从y开始一直到3012（因为就算最大值3000，一直检验到3012也满足题意了～后面的就没必要了～），将它的每一位的数字放入集合s里面，因为集合是不允许重复数字的，所以集合s的大小就是当前这个数字的不重复数字的个数～当集合s的大小和n相等时，说明满足题意，输出就好啦～

```
1  #include <cstdio>
2  #include <set>
3  using namespace std;
4  int main() {
5      int y, n;
6      scanf("%d%d", &y, &n);
7      for (int i = y; i <= 3012; i++) {
8          set<int> s;
9          int num = i;
10         for (int j = 0; j < 4; j++) {
11             s.insert(num % 10);
12             num /= 10;
13         }
14         if (s.size() == n) {
15             printf("%d %04d", i - y, i);
16             break;
17         }
18     }
19     return 0;
20 }
```

L1-034. 点赞 [逻辑题]

微博上有个“点赞”功能，你可以为你喜欢的博文点个赞表示支持。每篇博文都有一些刻画其特性的标签，而你点赞的博文的类型，也间接刻画了你的特性。本题就要求你写个程序，通过统计一个人点赞的纪录，分析这个人的特性。

输入格式：

输入在第一行给出一个正整数N (≤ 1000)，是该用户点赞的博文数量。随后N行，每行给出一篇被其点赞的博文特性描述，格式为“K F1 ... FK”，其中 $1 \leq K \leq 10$ ， F_i ($i=1, \dots, K$) 是特性标签的编号，我们将所有特性标签从1到1000编号。数字间以空格分隔。

输出格式：

统计所有被点赞的博文中最常出现的那个特性标签，在一行中输出它的编号和出现次数，数字间隔1个空格。如果有并列，则输出编号最大的那个。

输入样例：

```
4
3 889 233 2
5 100 3 233 2 73
4 3 73 889 2
2 233 123
```

输出样例：

```
233 3
```

分析：使用arr[1001]存储每个性格标签出现的次数，遍历arr数组，找到出现次数最多的保存为maxValue，并保存其下标为maxIndex~

maxIndex就是出现次数最多的特性标签，maxValue就是该特性标签出现的次数~

```
1  #include <stdio>
2  using namespace std;
3  int main() {
4      int n, k, num, maxIndex = 0, maxValue = 0;
5      scanf("%d", &n);
6      int arr[1001] = {0};
7      for (int i = 0; i < n; i++) {
8          scanf("%d", &k);
9          for (int j = 0; j < k; j++) {
10             scanf("%d", &num);
11             arr[num]++;
12         }
13     }
14     for (int i = 0; i < 1001; i++) {
15         if (arr[i] >= maxValue) {
16             maxValue = arr[i];
17             maxIndex = i;
18         }
19     }
```

```

19     }
20     printf("%d %d", maxIndex, maxValue);
21     return 0;
22 }

```

L1-035. 情人节 [循环]

以上是朋友圈中一奇葩贴：“2月14情人节了，我决定造福大家。第2个赞和第14个赞的，我介绍你俩认识.....咱三吃饭...你俩请...”。现给出此贴下点赞的朋友名单，请你找出那两位要请客的倒霉蛋。

输入格式：

输入按照点赞的先后顺序给出不知道多少个点赞的人名，每个人名占一行，为不超过10个英文字母的非空单词，以回车结束。一个英文句点“.”标志输入的结束，这个符号不算在点赞名单里。

输出格式：

根据点赞情况在一行中输出结论：若存在第2个人A和第14个人B，则输出“A and B are inviting you to dinner...”；若只有A没有B，则输出“A is the only one for you...”；若连A都没有，则输出“Momo... No one is for you ...”。

输入样例1：

GaoXZh
Magi
Einst
Quark
LaoLao
FatMouse
ZhaShen
fantacy
latesum
SenSen
QuanQuan
whatever
whenever
Potaty
hahaha
.

输出样例1：

Magi and Potaty are inviting you to dinner...

输入样例2:

LaoLao

FatMouse

whoever

.

输出样例2:

FatMouse is the only one for you...

输入样例3:

LaoLao

.

输出样例3:

Momo... No one is for you ...

分析: 接收输入, 当输入为"."的时候退出循环~每次接收一个输入就将cnt++, 如果当前cnt==2或者==4就将结果保存到string a或者string b中, 最后根据cnt不同的值输出不同的结果~

```
1  #include <iostream>
2  using namespace std;
3  int main() {
4      string a, b, str;
5      int cnt = 0;
6      while(cin >> str) {
7          if (str == ".") break;
8          cnt++;
9          if (cnt == 2) a = str;
10         if (cnt == 4) b = str;
11     }
12     if (cnt >= 4)
13         cout << a << " and " << b << " are inviting you to dinner...";
14     else if (cnt <= 1)
15         cout << "Momo... No one is for you ...";
16     else
17         cout << a << " is the only one for you...";
18     return 0;
19 }
```

L1-036. A乘以B [水题]

看我没骗你吧 —— 这是一道你可以在10秒内完成的题：给定两个绝对值不超过100的整数A和B，输出A乘以B的值。

输入格式：

输入在第一行给出两个整数A和B ($-100 \leq A, B, \leq 100$)，数字间以空格分隔。

输出格式：

在一行中输出A乘以B的值。

输入样例：

-8 13

输出样例：

-104

```
1  #include <iostream>
2  using namespace std;
3  int main() {
4      int a, b;
5      cin >> a >> b;
6      cout << a * b;
7      return 0;
8  }
```

L1-037. A除以B [逻辑题]

真的是简单题哈 —— 给定两个绝对值不超过100的整数A和B，要求你按照“A/B=商”的格式输出结果。

输入格式：

输入在第一行给出两个整数A和B ($-100 \leq A, B, \leq 100$)，数字间以空格分隔。

输出格式：

在一行中输出结果：如果分母是正数，则输出“A/B=商”；如果分母是负数，则要用括号把分母括起来输出；如果分母为零，则输出的商应为“Error”。输出的商应保留小数点后2位。

输入样例1：

-1 2

输出样例1：

-1/2=-0.50

输入样例2：

1 -3

输出样例2：

1/(-3)=-0.33

输入样例3：

5 0

输出样例3：

5/0=Error

分析：如果b小于0就左右加上括号，如果b=0就要输出计算结果为Error，其余情况就输出a/b的保留两位小数的结果～

```
1  #include <stdio>
2  using namespace std;
3  int main() {
4      int a, b;
5      scanf("%d%d", &a, &b);
6      printf("%d/", a);
7      if (b >= 0)
8          printf("%d=", b);
9      else
10         printf("(%d=", b);
11     if (b == 0)
12         printf("Error");
13     else
14         printf("%.2f", a * 1.0 / b);
15     return 0;
16 }
```

L1-038. 新世界 [水题]

这道超级简单的题目没有任何输入。

你只需要在第一行中输出程序员钦定名言“Hello World”，并且在第二行中输出更新版的“Hello New World”就可以了。

```
1  #include <stdio>
2  int main() {
3      printf("Hello World\nHello New World");
4      return 0;
5  }
```

L1-039. 古风排版 [逻辑题]

中国的古人写文字，是从右向左竖向排版的。本题就请你编写程序，把一段文字按古风排版。

输入格式：

输入在第一行给出一个正整数N（<100），是每一列的字符数。第二行给出一个长度不超过1000的非空字符串，以回车结束。

输出格式：

按古风格排版给定的字符串，每列N个字符（除了最后一列可能不足N个）

输入样例：

4

This is a test case

输出样例：

asa T

st ih

e tsi

ce s

分析：先根据字符串的长度s.length()和n计算出col列数，然后建立一个n行col列的字符数组，一开始都赋值为空格，然后根据先最后一列再倒数第二列依次向前，行数从0~n-1的顺序依次将s[index]的值填入vector中，直到s的所有字符都填完为止，最后输出这个字符数组~

```
1  #include <iostream>
2  #include <vector>
3  using namespace std;
4  int main() {
5      int n;
6      string s;
7      cin >> n;
8      getchar();
9      getline(cin, s);
10     int col = (s.length() - 1) / n + 1;
11     vector<vector<char>> > v(n, vector<char>(col, ' '));
12     int index = 0;
13     for (int j = col - 1; j >= 0; j--) {
14         for (int i = 0; i < n; i++) {
15             if (index < s.length())
16                 v[i][j] = s[index++];
17         }
18     }
19     for (int i = 0; i < n; i++) {
20         for (int j = 0; j < col; j++)
21             cout << v[i][j];
22         cout << endl;
23     }
24     return 0;
25 }
```

L1-040. 最佳情侣身高差 [逻辑题]

专家通过多组情侣研究数据发现，最佳的情侣身高差遵循着一个公式：（女方的身高）×1.09=（男方的身高）。如果符合，你俩的身高差不管是牵手、拥抱、接吻，都是最和谐的差度。

下面就请你写个程序，为任意一位用户计算他/她的情侣的最佳身高。

输入格式：

输入第一行给出正整数N (≤ 10)，为前来查询的用户数。随后N行，每行按照“性别 身高”的格式给出前来查询的用户的性别和身高，其中“性别”为“F”表示女性、“M”表示男性；“身高”为区间 [1.0, 3.0] 之间的实数。

输出格式：

对每一个查询，在一行中为该用户计算出其情侣的最佳身高，保留小数点后2位。

输入样例：

2

M 1.75

F 1.8

输出样例：

1.61

1.96

分析：特别注意的是，如果使用scanf，在用scanf接收字符c之前，记得加一个\n，否则可能c把\n接收进去导致错误输出～

```
1  #include <stdio>
2  int main() {
3      int n;
4      scanf("%d", &n);
5      for (int i = 0; i < n; i++) {
6          char c;
7          float height;
8          scanf("\n%c %f", &c, &height);
9          if (c == 'M')
10             printf("%.2f\n", height * 1.0 / 1.09);
11         else
12             printf("%.2f\n", height * 1.09);
13     }
14     return 0;
15 }
```

L1-041. 寻找250 [逻辑题]

对方不想和你说话，并向你扔了一串数…… 而你必须从这一串数字中找到“250”这个高大上的感人数字。

输入格式：

输入在一行中给出不知道多少个绝对值不超过1000的整数，其中保证至少存在一个“250”。

输出格式：

在一行中输出第一次出现的“250”是对方扔过来的第几个数字（计数从1开始）。题目保证输出的数字在整型范围内。

输入样例：

888 666 123 -233 250 13 250 -222

输出样例：

5

```
1  #include <iostream>
2  using namespace std;
3  int main() {
4      int num, cnt = 0;
5      while(cin >> num) {
6          cnt++;
7          if(num == 250) {
8              cout << cnt;
9              break;
10         }
11     }
12     return 0;
13 }
```

L1-042. 日期格式化 [逻辑题]

世界上不同国家有不同的写日期的习惯。比如美国人习惯写成“月-日-年”，而中国人习惯写成“年-月-日”。下面请你写个程序，自动把读入的美国格式的日期改写成中国习惯的日期。

输入格式：

输入在一行中按照“mm-dd-yyyy”的格式给出月、日、年。题目保证给出的日期是1900年元旦至今合法的日期。

输出格式：

在一行中按照“yyyy-mm-dd”的格式给出年、月、日。

输入样例：

03-15-2017

输出样例：

2017-03-15

```
1  #include <iostream>
2  using namespace std;
3  int main() {
4      int a, b, c;
5      char ch;
6      cin >> a >> ch >> b >> ch >> c;
7      printf("%d-%02d-%02d", c, a, b);
8      return 0;
9  }
```

L1-043. 阅览室 [逻辑题]

天梯图书阅览室请你编写一个简单的图书借阅统计程序。当读者借书时，管理员输入书号并按下S键，程序开始计时；当读者还书时，管理员输入书号并按下E键，程序结束计时。书号为不超过1000的正整数。当管理员将0作为书号输入时，表示一天工作结束，你的程序应输出当天的读者借书次数和平均阅读时间。

注意：由于线路偶尔会有故障，可能出现不完整的纪录，即只有S没有E，或者只有E没有S的纪录，系统应能自动忽略这种无效纪录。另外，题目保证书号是书的唯一标识，同一本书在任何时间区间内只可能被一位读者借阅。

输入格式：

输入在第一行给出一个正整数N (≤ 10)，随后给出N天的纪录。每天的纪录由若干次借阅操作组成，每次操作占一行，格式为：

书号 ([1, 1000]内的整数) 键值 ("S"或"E") 发生时间 (hh:mm, 其中hh是[0,23]内的整数, mm是[0, 59]内整数)

每一天的纪录保证按时间递增的顺序给出。

输出格式：

对每天的纪录，在一行中输出当天的读者借书次数和平均阅读时间（以分钟为单位的精确到个位的整数时间）。

输入样例：

```
3
1 S 08:10
2 S 08:35
1 E 10:00
2 E 13:16
0 S 17:00
0 S 17:00
3 E 08:10
1 S 08:20
2 S 09:00
1 E 09:20
0 E 17:00
```

输出样例：

```
2 196
0 0
1 60
```

分析：模拟借书还书，输出当天借书次数和平均借书时间，四舍五入可以用加 0.5 然后强转成int，用 isborrow数组标记书是否被借走

注意：如果出现连续两次借书，以第二次为成功借阅为成功借阅，（个人理解应该是第一次借书成功，第二次为非法借书，可题目只有按照第二次为借书成功计算）如果连续两次还书，以第一次还书为成功

```
1  #include <iostream>
2  #include <cstring>
3  using namespace std;
4  int main() {
5      int n;
6      cin >> n;
7      int id, h, m, cnt = 0, sum = 0, isborr[1010] = {0}, time[1010] =
{0};
8      char type, temp;
9      for(int day = 0; day < n; ) {
10         cin >> id >> type >> h >> temp >> m;
11         if(id == 0) {
12             if(cnt == 0)
13                 printf("0 0\n");
14             else
15                 printf("%d %d\n", cnt, (int)(1.0 * sum / cnt + 0.5));
16             day++;
17             cnt = sum = 0;
18             memset(isborr, 0, sizeof(isborr));
19         }else if(type == 'S') {
20             isborr[id] = 1;
21             time[id] = h * 60 + m;
22         }else if(type == 'E' && isborr[id] == 1) {
23             isborr[id] = 0;
24             sum += h * 60 + m - time[id];
25             cnt++;
26         }
27     }
28     return 0;
29 }
```

L1-044. 稳赢 [逻辑题]

大家应该都会玩“锤子剪刀布”的游戏：两人同时给出手势，胜负规则如图所示：

现要求你编写一个稳赢不输的程序，根据对方的出招，给出对应的赢招。但是！为了不让对方输得太惨，你需要每隔K次就让一个平局。

输入格式：

输入首先在第一行给出正整数K (≤ 10)，即平局间隔的次数。随后每行给出对方的一次出招：“ChuiZi”代表“锤子”、“JianDao”代表“剪刀”、“Bu”代表“布”。“End”代表输入结束，这一行不要作为出招处理。

输出格式：

对每一个输入的出招，按要求输出稳赢或平局的招式。每招占一行。

输入样例：

2

ChuiZi

JianDao

Bu

JianDao

Bu

ChuiZi

ChuiZi

End

输出样例：

Bu

ChuiZi

Bu

ChuiZi

JianDao

ChuiZi

Bu

```
1  #include <iostream>
2  #include <map>
3  using namespace std;
4  int main() {
5      map<string,string> m;
6      m["Bu"] = "JianDao";
7      m["JianDao"] = "ChuiZi";
8      m["ChuiZi"] = "Bu";
9      int k, cnt = 0;
10     string s;
11     cin >> k;
12     while(cin >> s) {
13         if(s == "End") return 0;
14         cnt++;
15         if(cnt % (k + 1) == 0)
16             cout << s << endl;
17         else
```

```

18         cout << m[s] << endl;
19     }
20     return 0;
21 }

```

L1-045. 宇宙无敌大招呼 [逻辑题]

据说所有程序员学习的第一个程序都是在屏幕上输出一句“Hello World”，跟这个世界打个招呼。作为天梯赛中的程序员，你写的程序得高级一点，要能跟任意指定的星球打招呼。

输入格式：

输入在第一行给出一个星球的名字S，是一个由不超过7个英文字母组成的单词，以回车结束。

输出格式：

在一行中输出“Hello S”，跟输入的S星球打个招呼。

输入样例：

Mars

输出样例：

Hello Mars

```

1  #include <iostream>
2  using namespace std;
3  int main() {
4      string s;
5      cin >> s;
6      cout << "Hello " << s;
7      return 0;
8  }

```

L1-046. 整除光棍 [逻辑题]

这里所谓的“光棍”，并不是指单身汪啦~ 说的是全部由1组成的数字，比如1、11、111、1111等。传说任何一个光棍都能被一个不以5结尾的奇数整除。比如，111111就可以被13整除。现在，你的程序要读入一个整数x，这个整数一定是奇数并且不以5结尾。然后，经过计算，输出两个数字：第一个数字s，表示x乘以s是一个光棍，第二个数字n是这个光棍的位数。这样的解当然不是唯一的，题目要求你输出最小的解。

提示：一个显然的办法是逐渐增加光棍的位数，直到可以整除x为止。但难点在于，s可能是个非常大的数——比如，程序输入31，那么就输出3584229390681和15，因为31乘以3584229390681的结果是111111111111111，一共15个1。

输入格式：

输入在一行中给出一个不以5结尾的正奇数x（< 1000）。

输出格式：

在一行中输出相应的最小的s和n，其间以1个空格分隔。

输入样例：

31

输出样例：

3584229390681 15

分析：从ans为1个1开始遍历到n个1，先把ans置为大于等于n的数字，并记录此时cnt。

开始循环，输出ans/n, 即答案的第一位，如果此时正好整除了，立即退出循环，否则把除过n剩下的部分在加上一位，继续循环。

循环结束后答案的每一位被依次输出，最后输出cnt位数~

```
1  #include <iostream>
2  using namespace std;
3  int main() {
4      int n, ans = 1, cnt = 1;
5      cin >> n;
6      while (ans < n) {
7          ans = ans * 10 + 1;
8          cnt++;
9      }
10     while(1){
11         cout << ans / n;
12         if(ans % n == 0) break;
13         ans = (ans % n) * 10 + 1;
14         cnt++;
15     }
16     cout << ' ' << cnt;
17     return 0;
18 }
```

L1-047. 装睡 [逻辑题]

你永远叫不醒一个装睡的人 —— 但是通过分析一个人的呼吸频率和脉搏，你可以发现谁在装睡！医生告诉我们，正常人睡眠时的呼吸频率是每分钟15-20次，脉搏是每分钟50-70次。下面给定一系列人的呼吸频率与脉搏，请你找出他们中间有可能在装睡的人，即至少一项指标不在正常范围内的人。

输入格式：

输入在第一行给出一个正整数N (≤ 10)。随后N行，每行给出一个人的名字（仅由英文字母组成的、长度不超过3个字符的串）、其呼吸频率和脉搏（均为不超过100的正整数）。

输出格式：

按照输入顺序检查每个人，如果其至少一项指标不在正常范围内，则输出其名字，每个名字占一行。

输入样例：

4

Amy 15 70

Tom 14 60

Joe 18 50

Zoe 21 71

输出样例：

Tom

Zoe

```
1  #include <iostream>
2  using namespace std;
3  int main() {
4      int n, heart, impluse;
5      string name;
6      cin >> n;
7      for(int i = 0; i < n; i++) {
8          cin >> name >> heart >> impluse;
9          if(heart < 15 || heart > 20 || impluse < 50 || impluse > 70)
10             cout << name << endl;
11     }
12     return 0;
13 }
```

L1-048. 矩阵A乘以B [矩阵乘法]

给定两个矩阵A和B，要求你计算它们的乘积矩阵AB。需要注意的是，只有规模匹配的矩阵才可以相乘。即若A有Ra行、Ca列，B有Rb行、Cb列，则只有Ca与Rb相等时，两个矩阵才能相乘。

输入格式：

输入先后给出两个矩阵A和B。对于每个矩阵，首先在一行中给出其行数R和列数C，随后R行，每行给出C个整数，以1个空格分隔，且行首尾没有多余的空格。输入保证两个矩阵的R和C都是正数，并且所有整数的绝对值不超过100。

输出格式：

若输入的两个矩阵的规模是匹配的，则按照输入的格式输出乘积矩阵AB，否则输出“Error: Ca != Rb”，其中Ca是A的列数，Rb是B的行数。

输入样例1：

2 3

1 2 3

4 5 6

3 4

7 8 9 0

-1 -2 -3 -4

5 6 7 8

输出样例1:

2 4

20 22 24 16

53 58 63 28

输入样例2:

3 2

38 26

43 -5

0 17

3 2

-11 57

99 68

81 72

输出样例2:

Error: 2 != 3

分析: 如果不能相乘则直接输出, 如果能, 则矩阵A为ra行ca列, 矩阵B为rb行cb列, 结果矩阵ans为ra行cb列。

ans矩阵中每一个元素 $ans[i][j] = \sum(a[ra][i] * b[i][cb])$ $[1 \leq i \leq ca]$ 。注意输出时行末不留空格~

```
1  #include <iostream>
2  using namespace std;
3  int a[110][110], b[110][110], ans[110][110];
4  int ra, ca, rb, cb;
5  int f(int r, int c) {
6      int sum = 0;
7      for(int i = 1; i <= ca; i++)
8          sum += a[r][i] * b[i][c];
9      return sum;
10 }
11 int main() {
12     cin >> ra >> ca;
13     for(int i = 1; i <= ra; i++)
14         for(int j = 1; j <= ca; j++)
15             cin >> a[i][j];
16     cin >> rb >> cb;
17     for(int i = 1; i <= rb; i++)
18         for(int j = 1; j <= cb; j++)
19             cin >> b[i][j];
```



```

20     if(ca != rb)
21         printf("Error: %d != %d", ca, rb);
22     else {
23         printf("%d %d\n", ra, cb);
24         for(int i = 1; i <= ra; i++) {
25             for(int j = 1; j <= cb; j++) {
26                 if(j != 1) cout << ' ';
27                 cout << f(i,j);
28             }
29             cout << endl;
30         }
31     }
32     return 0;
33 }

```

L1-049. 天梯赛座位分配 [逻辑题]

天梯赛每年有大量参赛队员，要保证同一所学校的所有队员都不能相邻，分配座位就成为一件比较麻烦的事情。为此我们制定如下策略：假设某赛场有 N 所学校参赛，第 i 所学校有 $M[i]$ 支队伍，每队 10 位参赛选手。令每校选手排成一行纵队，第 $i+1$ 队的选手排在第 i 队选手之后。从第 1 所学校开始，各校的第 1 位队员顺次入座，然后是各校的第 2 位队员……以此类推。如果最后只剩下 1 所学校的队伍还没有分配座位，则需要安排他们的队员隔位就坐。本题就要求你编写程序，自动为各校生成队员的座位号，从 1 开始编号。

输入格式：

输入在一行中给出参赛的高校数 N （不超过 100 的正整数）；第二行给出 N 个不超过 10 的正整数，其中第 i 个数对应第 i 所高校的参赛队伍数，数字间以空格分隔。

输出格式：

从第 1 所高校的第 1 支队伍开始，顺次输出队员的座位号。每队占一行，座位号间以 1 个空格分隔，行首尾不得有多余空格。另外，每所高校的第一行按“#X”输出该校的编号 X ，从 1 开始。

输入样例：

```

3
3 4 2

```

输出样例：

```

#1
1 4 7 10 13 16 19 22 25 28
31 34 37 40 43 46 49 52 55 58
61 63 65 67 69 71 73 75 77 79

#2
2 5 8 11 14 17 20 23 26 29
32 35 38 41 44 47 50 53 56 59

```

62 64 66 68 70 72 74 76 78 80

82 84 86 88 90 92 94 96 98 100

#3

3 6 9 12 15 18 21 24 27 30

33 36 39 42 45 48 51 54 57 60

分析：让每个学校的学生排成一排，为每一个座位安排学生。

num[i]是一个学校的队伍数，id_school[i]是第i个座位上做的是哪个学校的学生。

每把椅子依次从每个学校面前走过，如果这个学校还有学生没被安排座位，那就安排，安排方案为：

如果此座位前一个不是本校学生，那么抓一个此校学生做在这把椅子上，并标记这把椅子被此校学生坐了，

如果此座位前一个是本校学生，那此位置空着，抓一个此校学生做在这把椅子的后一把上，并标记这把椅子的后一把被此校学生坐了，

设置flag标志，如果所有的学校学生都被安排座位，说明安排完毕，可退出。

注意：输出时候注意行末不留空格

```
1  #include <iostream>
2  #include <vector>
3  using namespace std;
4  int main() {
5      int n, sum = 0, cnt = 1, num[110] = {0}, id_school[100001] = {0};
6      cin >> n;
7      vector<vector<int>> v(n+1);
8      for(int i = 1; i <= n; i++)
9          cin >> num[i];
10     while(1) {
11         bool flag = true;
12         for(int i = 1; i < v.size(); i++) {
13             if(v[i].size() < num[i] * 10) {
14                 if(id_school[cnt-1] != i) {
15                     id_school[cnt] = i;
16                     v[i].push_back(cnt);
17                     cnt++;
18                 } else {
19                     id_school[cnt+1] = i;
20                     v[i].push_back(cnt+1);
21                     cnt = cnt + 2;
22                 }
23                 flag = false;
24             }
25         }
26         if(flag) break;
27     }
```

```

28     for(int i = 1; i < v.size(); i++) {
29         cout << "#" << i << endl;
30         for(int j = 0; j < v[i].size(); j++) {
31             if(j != 0 && j % 10 != 0) cout << " ";
32             if(j != 0 && j % 10 == 0) cout << endl;
33             cout << v[i][j];
34         }
35         cout << endl;
36     }
37     return 0;
38 }

```

L1-050. 倒数第N个字符串 [进制转换]

给定一个完全由小写英文字母组成的字符串等差递增序列，该序列中的每个字符串的长度固定为 L ，从 L 个 a 开始，以 1 为步长递增。例如当 L 为 3 时，序列为 { aaa, aab, aac, ..., aaz, aba, abb, ..., abz, ..., zzz }。这个序列的倒数第 27 个字符串就是 zyz。对于任意给定的 L ，本题要求你给出对应序列倒数第 N 个字符串。

输入格式：

输入在一行中给出两个正整数 L ($2 \leq L \leq 6$) 和 N ($\leq 10^5$)。

输出格式：

在一行中输出对应序列倒数第 N 个字符串。题目保证这个字符串是存在的。

输入样例：

3 7417

输出样例：

pat

分析：这是一道进制转换的题目。把 L 位数看成 L 位 26 进制组成的数字，则最后一个数字的十进制表示为 $\text{pow}(26, l) - 1$ ，倒数 n 个数的十进制表示为 $\text{pow}(26, l) - n$ ；

然后把所求的结果转换成 26 进制还原即可。注意，还原成 26 进制时位数不足时，在前面补满 'a' 即可～

```

1  #include <iostream>
2  #include <cmath>
3  #include <vector>
4  using namespace std;
5  int main() {
6      int l, n, num;
7      cin >> l >> n;
8      vector<int> v;
9      num = pow(26, l) - n ;
10     while(num) {
11         v.push_back(num % 26);
12         num /= 26;
13     }

```

```

14     for(int i = 0; i < l - v.size(); i++)
15         cout << 'a';
16     for(int i = v.size() - 1; i >= 0; i--)
17         cout << (char)('a' + v[i]);
18     return 0;
19 }

```

L1-051. 打折 [逻辑题]

去商场淘打折商品时，计算打折以后的价钱是件颇费脑子的事情。例如原价 ¥988，标明打 7 折，则折扣价应该是 $¥988 \times 70\% = ¥691.60$ 。本题就请你写个程序替客户计算折扣价。

输入格式：

输入在一行中给出商品的原价（不超过1万元的正整数）和折扣（为[1, 9]区间内的整数），其间以空格分隔。

输出格式：

在一行中输出商品的折扣价，保留小数点后 2 位。

输入样例：

988 7

输出样例：

691.60

```

1  #include <stdio.h>
2  int main() {
3      int a, b;
4      scanf("%d%d", &a, &b);
5      printf("%.2f", a * b * 0.1);
6      return 0;
7  }

```

L1-052. 2018我们要赢 [水题]

2018年天梯赛的注册邀请码是“2018wmyy”，意思就是“2018我们要赢”。本题就请你用汉语拼音输出这句话。

输入格式：

本题没有输入。

输出格式：

在第一行中输出：“2018”；第二行中输出：“wo3 men2 yao4 ying2 !”。

输入样例：

本题没有输入。

输出样例：

2018

wo3 men2 yao4 ying2 !

```
1 #include <stdio.h>
2 int main() {
3     printf("2018\nwo3 men2 yao4 ying2 !");
4     return 0;
5 }
```

L1-053. 电子汪 [水题]

据说汪星人的智商能达到人类4岁儿童的水平，更有些聪明汪会做加法计算。比如你在地上放两堆小球，分别有1只球和2只球，聪明汪就会用“汪！汪！汪！”表示1加2的结果是3。

本题要求你为电子宠物汪做一个模拟程序，根据电子眼识别出的两堆小球的个数，计算出和，并且用汪星人的叫声给出答案。

输入格式：

输入在一行中给出两个[1, 9]区间内的正整数A和B，用空格分隔。

输出格式：

在一行中输出A+B个“Wang!”。

输入样例：

2 1

输出样例：

Wang!Wang!Wang!

```
1 #include <stdio.h>
2 int main() {
3     int a, b;
4     scanf("%d%d", &a, &b);
5     for(int i = 0; i < a + b; i++)
6         printf("Wang!");
7     return 0;
8 }
```

L1-054. 福到了 [逻辑题]

“福”字倒着贴，寓意“福到”。不论到底算不算民俗，本题且请你编写程序，把各种汉字倒过来输出。这里要处理的每个汉字是由一个 $N \times N$ 的网格组成的，网格中的元素或者为字符“@”或者为空格。而倒过来的汉字所用的字符由裁判指定。

输入格式：

输入在第一行中给出倒过来的汉字所用的字符、以及网格的规模 N （不超过100的正整数），其间以1个空格分隔；随后 N 行，每行给出 N 个字符，或者为“@”或者为空格。

输出格式：

输出倒置的网格，如样例所示。但是，如果这个字正过来倒过去是一样的，就先输出“bu yong dao le”，然后再用输入指定的字符将其输出。

输入样例 1：

\$ 9

@ @ @ @ @ @

@ @ @ @ @ @

@ @ @

@ @ @ @ @ @

@ @ @ @ @ @ @ @

@ @ @ @ @ @ @

@ @ @ @ @ @ @ @

@ @ @ @

@ @ @ @ @ @

输出样例 1：

\$\$\$\$\$ \$

\$ \$ \$ \$

\$\$\$\$\$ \$\$\$\$

\$ \$ \$ \$ \$ \$

\$\$\$\$\$ \$\$\$\$

\$\$\$ \$\$\$

\$ \$ \$

\$\$\$ \$\$\$

\$\$\$\$\$ \$

输入样例 2：

& 3

@ @ @

@

@ @ @

输出样例 2：

bu yong dao le

&&&

&

&&&

分析：设置flag记录是否倒过来和原来相同，输出时候倒过来输出就行了，不必重新记录到过来的矩阵

```
1  #include <iostream>
2  #include <vector>
3  using namespace std;
4  int main() {
5      char c;
6      int n, flag = 0;
7      scanf("%c%d", &c, &n);
8      getchar();
9      vector<string> a(n), b(n);
10     for(int i = 0; i < n; i++)
11         getline(cin, a[i]);
12     for(int i = 0; i < n; i++)
13         for(int j = 0; j < n; j++)
14             if(a[i][j] != a[n-1-i][n-1-j])
15                 flag = 1;
16     if(flag == 0) cout << "bu yong dao le\n";
17     for(int i = 0; i < n; i++) {
18         for(int j = 0; j < n; j++) {
19             if(a[n-1-i][n-1-j] == '@')
20                 cout << c;
21             else
22                 cout << " ";
23         }
24         cout << endl;
25     }
26     return 0;
27 }
```

L1-055. 谁是赢家 [逻辑题]

某电视台的娱乐节目有个表演评审环节，每次安排两位艺人表演，他们的胜负由观众投票和3名评委投票两部分共同决定。规则为：如果一位艺人的观众票数高，且得到至少1名评委的认可，该艺人就胜出；或艺人的观众票数低，但得到全部评委的认可，也可以胜出。节目保证投票的观众人数为奇数，所以不存在平票的情况。本题就请你用程序判断谁是赢家。

输入格式：

输入第一行给出 2 个不超过 1000 的正整数 Pa 和 Pb，分别是艺人 a 和艺人 b 得到的观众票数。题目保证这两个数字不相等。随后第二行给出 3 名评委的投票结果。数字 0 代表投票给 a，数字 1 代表投票给 b，其间以一个空格分隔。

输出格式：

按以下格式输出赢家：

The winner is x: P1 + P2

其中 x 是代表赢家的字母，P1 是赢家得到的观众票数，P2 是赢家得到的评委票数。

输入样例：

327 129

1 0 1

输出样例：

The winner is a: 327 + 1

```
1  #include <iostream>
2  using namespace std;
3  int main() {
4      int votea, voteb, adj1, adj2, adj3, suma;
5      cin >> votea >> voteb >> adj1 >> adj2 >> adj3;
6      suma= 3 - (adj1 + adj2 + adj3);
7      if(suma == 3 || (suma >= 1 && votea > voteb))
8          printf("The winner is a: %d + %d", votea, suma);
9      else
10         printf("The winner is b: %d + %d", voteb, 3-suma);
11     return 0;
12 }
```

L1-056. 猜数字 [逻辑题]

一群人坐在一起，每人猜一个 100 以内的数，谁的数字最接近大家平均数的一半就赢。本题就要求你找出其中的赢家。

输入格式：

输入在第一行给出一个正整数N ($\leq 10^4$)。随后 N 行，每行给出一个玩家的名字（由不超过8个英文字母组成的字符串）和其猜的正整数 (≤ 100)。

输出格式：

在一行中顺序输出：大家平均数的一半（只输出整数部分）、赢家的名字，其间以空格分隔。题目保证赢家是唯一的。

输入样例：

7

Bob 35

Amy 28

James 98

Alice 11

Jack 45

Smith 33

Chris 62

输出样例：

22 Amy

分析：用猜数做下表，用名字做数组中的数据，遍历数组，找出最接近平均值的一半的下标。输出平均值的一半，和对应下标存的名字即可。题目保证赢家唯一，所以，只有一个人猜到的最终结果。即使出现多个人猜一个数，对应下标存的名字会被覆盖也没关系，反正他们都不是最终答案

```
1  #include <iostream>
2  #include <cmath>
3  using namespace std;
4  int main() {
5      int n, t, sum = 0, mindet = 1000, minid;
6      string name, a[110];
7      cin >> n;
8      for(int i = 0; i < n; i++) {
9          cin >> name >> t;
10         a[t] = name;
11         sum += t;
12     }
13     double avg = 0.5 * sum / n;
14     for(int i = 0; i <= 100; i++)
15         if(a[i] != "" && abs(avg - i) < mindet) {
16             minid = i;
17             mindet = abs(avg - i);
18         }
19     cout << (int)avg << " " << a[minid];
20     return 0;
21 }
```

L2-001. 紧急救援 [Dijkstra算法]

作为一个城市的应急救援队伍的负责人，你有一张特殊的全国地图。在地图上显示有多个分散的城市和一些连接城市的快速道路。每个城市的救援队数量和每一条连接两个城市的快速道路长度都标在地图上。当其他城市有紧急求助电话给你的时候，你的任务是带领你的救援队尽快赶往事发地，同时，一路上召集尽可能多的救援队。

输入格式：

输入第一行给出4个正整数N、M、S、D，其中N ($2 \leq N \leq 500$) 是城市的个数，顺便假设城市的编号为0~(N-1)；M是快速道路的条数；S是出发地的城市编号；D是目的地的城市编号。第二行给出N个正整数，其中第i个数是第i个城市的救援队的数目，数字间以空格分隔。随后的M行中，每行给出一条快速道路的信息，分别是：城市1、城市2、快速道路的长度，中间用空格分开，数字均为整数且不超过500。输入保证救援可行且最优解唯一。

输出格式：

第一行输出不同的最短路径的条数和能够召集的最多的救援队数量。第二行输出从S到D的路径中经过的城市编号。数字间以空格分隔，输出首尾不能有多余空格。

输入样例：

4 5 0 3

20 30 40 10

0 1 1

1 3 2

0 3 3

0 2 2

2 3 2

输出样例：

2 60

0 1 3

分析：用一遍dijkstra算法。设立num[i]和w[i]表示从出发点到i结点拥有的路的条数，以及能够找到的救援队的数目~~当判定 $dis[u] + e[u][v] < dis[v]$ 的时候，不仅仅要更新dis[v]，还要更新num[v] = num[u], $w[v] = weight[v] + w[u]$; 如果 $dis[u] + e[u][v] == dis[v]$ ，还要更新num[v] += num[u]，而且判断一下是否权重w[v]更小，如果更小了就更新 $w[v] = weight[v] + w[u]$;

再设立一个pre[i]表示最短路径的前一个结点，在 $dis[u] + e[u][v] \leq dis[v]$ 的时候更新pre[v] = u，最后递归打印路径即可

```
1  #include <cstdio>
2  #include <algorithm>
3  using namespace std;
4  int n, m, c1, c2;
5  int dis[510], weight[510], e[510][510], num[510], w[510], pre[510];
6  bool visit[510];
7  const int inf = 99999999;
8  void printPath(int v) {
9      if(v == c1) {
10         printf("%d", v);
11         return ;
12     }
13     printPath(pre[v]);
14     printf(" %d", v);
15 }
16 int main() {
17     scanf("%d%d%d%d", &n, &m, &c1, &c2);
18     for(int i = 0; i < n; i++)
19         scanf("%d", &weight[i]);
20     fill(e[0], e[0] + 510 * 510, inf);
21     fill(dis, dis + 510, inf);
```

```

22     int a, b, c;
23     for(int i = 0; i < m; i++) {
24         scanf("%d%d%d", &a, &b, &c);
25         e[a][b] = c;
26         e[b][a] = c;
27     }
28     dis[c1] = 0;
29     w[c1] = weight[c1];
30     num[c1] = 1;
31     for(int i = 0; i < n; i++) {
32         int u = -1, minn = inf;
33         for(int j = 0; j < n; j++) {
34             if(visit[j] == false && dis[j] < minn) {
35                 u = j;
36                 minn = dis[j];
37             }
38         }
39         if(u == -1) break;
40         visit[u] = true;
41         for(int v = 0; v < n; v++) {
42             if(visit[v] == false && e[u][v] != inf) {
43                 if(dis[u] + e[u][v] < dis[v]) {
44                     dis[v] = dis[u] + e[u][v];
45                     num[v] = num[u];
46                     w[v] = w[u] + weight[v];
47                     pre[v] = u;
48                 } else if(dis[u] + e[u][v] == dis[v]) {
49                     num[v] = num[v] + num[u];
50                     if(w[u] + weight[v] > w[v]) {
51                         w[v] = w[u] + weight[v];
52                         pre[v] = u;
53                     }
54                 }
55             }
56         }
57     }
58     printf("%d %d\n", num[c2], w[c2]);
59     printPath(c2);
60     return 0;
61 }

```

L2-002. 链表去重 [链表]

给定一个带整数键值的单链表L，本题要求你编写程序，删除那些键值的绝对值有重复的结点。即对任意键值K，只有键值或其绝对值等于K的第一个结点可以被保留。同时，所有被删除的结点必须被保存在另外一个链表中。例如：另L为21→15→15→7→15，则你必须输出去重后的链表21→15→7、以及被删除的链表-15→15。

输入格式：

输入第一行包含链表第一个结点的地址、以及结点个数N (≤ 105 的正整数)。结点地址是一个非负的5位整数，NULL指针用-1表示。

随后N行，每行按下列格式给出一个结点的信息：

Address Key Next

其中Address是结点的地址，Key是绝对值不超过104的整数，Next是下一个结点的地址。

输出格式：

首先输出去重后的链表，然后输出被删除结点组成的链表。每个结点占一行，按输入的格式输出。

输入样例：

```
00100 5
99999 -7 87654
23854 -15 00000
87654 15 -1
00000 -15 99999
00100 21 23854
```

输出样例：

```
00100 21 23854
23854 -15 99999
99999 -7 -1
00000 -15 87654
87654 15 -1
```

题目大意：给一个链表，去重（去掉值或者绝对值相等的），先输出删除后的链表，再输出删除了的链表。

分析：用结构体数组存储这个链表，大小为 $\text{maxn} = 100000$ ， $\text{node}[i]$ 表示地址为i的结点。在结构体中定义一个num变量，将num变量先初始化为 $2 * \text{maxn}$ 。通过改变num变量的值最后sort排序来改变链表的顺序。

将没有删除的结点的num标记为cnt1，cnt1为当前没有删除的结点的个数；将需要删除的结点的num标记为 $\text{maxn} + \text{cnt2}$ ，cnt2表示当前删除了的结点的个数，因为一开始初始化为 $2 * \text{maxn}$ ，所以我们可以通过对num排序达到：num = $0 \sim \text{maxn}$ 为不删除结点，num = $\text{maxn} \sim 2\text{maxn}$ 为删除结点，num = 2maxn 为无效结点

这样sort后就会按照需要输出的顺序将结点排序，我们只需要输出前cnt1+cnt2个结点即可～

```
1 #include <cstdio>
2 #include <stdlib.h>
3 #include <algorithm>
4 using namespace std;
```

```

5  const int maxn = 100000;
6  struct NODE {
7      int address;
8      int key;
9      int next;
10     int num;
11 }node[maxn];
12 bool exist[maxn];
13 int cmp1(NODE a, NODE b){
14     return a.num < b.num;
15 }
16 int main() {
17     int begin, n, cnt1 = 0, cnt2 = 0, a;
18     scanf("%d%d", &begin, &n);
19     for(int i = 0; i < maxn; i++) {
20         node[i].num = 2 * maxn;
21     }
22     for(int i = 0; i < n; i++) {
23         scanf("%d", &a);
24         scanf("%d%d", &node[a].key, &node[a].next);
25         node[a].address = a;
26     }
27     for(int i = begin; i != -1; i = node[i].next) {
28         if(exist[abs(node[i].key)] == false) {
29             exist[abs(node[i].key)] = true;
30             node[i].num = cnt1;
31             cnt1++;
32         }
33         else {
34             node[i].num = maxn + cnt2;
35             cnt2++;
36         }
37     }
38     sort(node, node + maxn, cmp1);
39     int cnt = cnt1 + cnt2;
40     for(int i = 0; i < cnt; i++) {
41         if(i != cnt1 - 1 && i != cnt - 1) {
42             printf("%05d %d %05d\n", node[i].address, node[i].key,
node[i+1].address);
43         } else {
44             printf("%05d %d -1\n", node[i].address, node[i].key);
45         }
46     }
47     return 0;
48 }

```

L2-003. 月饼 [贪心算法]

月饼是中國人在中秋佳節時吃的一種傳統食品，不同地區有許多不同風味的月餅。現給定所有種類月餅的庫存量、總售價、以及市場的最大需求量，請你計算可以獲得的最大收益是多少。

注意：銷售時允許取出一部分庫存。樣例給出的情形是這樣的：假如我們有3種月餅，其庫存量分別為18、15、10萬噸，總售價分別為75、72、45億元。如果市場的最大需求量只有20萬噸，那麼我們最大收益策略應該是賣出全部15萬噸第2種月餅、以及5萬噸第3種月餅，獲得 $72 + 45/2 = 94.5$ （億元）。

輸入格式：

每個輸入包含1個測試用例。每個測試用例先給出一個不超過1000的正整數N表示月餅的種類數、以及不超過500（以萬噸為單位）的正整數D表示市場最大需求量。隨後一行給出N個正數表示每種月餅的庫存量（以萬噸為單位）；最後一行給出N個正數表示每種月餅的總售價（以億元為單位）。數字間以空格分隔。

輸出格式：

對每組測試用例，在一行中輸出最大收益，以億元為單位並精確到小數點後2位。

輸入樣例：

3 20

18 15 10

75 72 45

輸出樣例：

94.50

```
1  #include <iostream>
2  #include <algorithm>
3  #include <vector>
4  using namespace std;
5  struct mooncake{
6      float mount, price, unit;
7  };
8  int cmp(mooncake a, mooncake b) {
9      return a.unit > b.unit;
10 }
11 int main() {
12     int n, need;
13     cin >> n >> need;
14     vector<mooncake> a(n);
15     for (int i = 0; i < n; i++) scanf("%f", &a[i].mount);
16     for (int i = 0; i < n; i++) scanf("%f", &a[i].price);
17     for (int i = 0; i < n; i++) a[i].unit = a[i].price / a[i].mount;
18     sort(a.begin(), a.end(), cmp);
19     float result = 0.0;
20     for (int i = 0; i < n; i++) {
21         if (a[i].mount <= need) {
22             result = result + a[i].price;
```

```

23         } else {
24             result = result + a[i].unit * need;
25             break;
26         }
27         need = need - a[i].mount;
28     }
29     printf("%.2f", result);
30     return 0;
31 }

```

L2-004. 这是二叉搜索树吗？ [二叉搜索树BST]

一棵二叉搜索树可被递归地定义为具有下列性质的二叉树：对于任一结点，

其左子树中所有结点的键值小于该结点的键值；

其右子树中所有结点的键值大于等于该结点的键值；

其左右子树都是二叉搜索树。

所谓二叉搜索树的“镜像”，即将所有结点的左右子树对换位置后所得到的树。

给定一个整数键值序列，现请你编写程序，判断这是否是对一棵二叉搜索树或其镜像进行前序遍历的结果。

输入格式：

输入的第一行给出正整数N（ ≤ 1000 ）。随后一行给出N个整数键值，其间以空格分隔。

输出格式：

如果输入序列是对一棵二叉搜索树或其镜像进行前序遍历的结果，则首先在一行中输出“YES”，然后在下一行输出该树后序遍历的结果。数字间有1个空格，一行的首尾不得有多余空格。若答案是否，则输出“NO”。

输入样例1：

```

7
8 6 5 7 10 8 11

```

输出样例1：

```

YES
5 7 6 8 11 10 8

```

输入样例2：

```

7
8 10 11 8 6 7 5

```

输出样例2：

```

YES

```

11 8 10 7 5 6 8

输入样例3:

7

8 6 8 5 10 9 11

输出样例3:

NO

分析：假设它是二叉搜索树，一开始isMirror为FALSE，根据二叉搜索树的性质将已知的前序转换为后序，转换过程中，如果发现最后输出的后序数组长度不为n，那就设isMirror为true，然后清空后序数组，重新再转换一次（根据镜面二叉搜索树的性质），如果依旧转换后数组大小不等于n，就输出no 否则输出yes

```
1  #include <cstdio>
2  #include <vector>
3  using namespace std;
4  bool isMirror;
5  vector<int> pre, post;
6  void getpost(int root, int tail) {
7      if(root > tail) return ;
8      int i = root + 1, j = tail;
9      if(!isMirror) {
10         while(i <= tail && pre[root] > pre[i]) i++;
11         while(j > root && pre[root] <= pre[j]) j--;
12     } else {
13         while(i <= tail && pre[root] <= pre[i]) i++;
14         while(j > root && pre[root] > pre[j]) j--;
15     }
16     if(i - j != 1) return ;
17     getpost(root + 1, j);
18     getpost(i, tail);
19     post.push_back(pre[root]);
20 }
21 int main() {
22     int n;
23     scanf("%d", &n);
24     pre.resize(n);
25     for(int i = 0; i < n; i++)
26         scanf("%d", &pre[i]);
27     getpost(0, n - 1);
28     if(post.size() != n) {
29         isMirror = true;
30         post.clear();
31         getpost(0, n - 1);
32     }
33     if(post.size() == n) {
34         printf("YES\n%d", post[0]);
```



```

35         for(int i = 1; i < n; i++)
36             printf(" %d", post[i]);
37     } else {
38         printf("NO");
39     }
40     return 0;
41 }

```

L2-005. 集合相似度 [集合set, STL的使用]

给定两个整数集合，它们的相似度定义为： $N_c/N_t \times 100\%$ 。其中 N_c 是两个集合都有的不相等整数的个数， N_t 是两个集合一共有不相等整数的个数。你的任务就是计算任意一对给定集合的相似度。

输入格式：

输入第一行给出一个正整数 N (≤ 50)，是集合的个数。随后 N 行，每行对应一个集合。每个集合首先给出一个正整数 M (≤ 104)，是集合中元素的个数；然后跟 M 个 $[0, 109]$ 区间内的整数。

之后一行给出一个正整数 K (≤ 2000)，随后 K 行，每行对应一对需要计算相似度的集合的编号（集合从1到 N 编号）。数字间以空格分隔。

输出格式：

对每一对需要计算的集合，在一行中输出它们的相似度，为保留小数点后2位的百分比数字。

输入样例：

```

3
3 99 87 101
4 87 101 5 87
7 99 101 18 5 135 18 99
2
1 2
1 3

```

输出样例：

```

50.00%
33.33%

```

题目大意： nc 是两个集合的公共元素个数， nt 是两个集合的所有包含的元素个数（其中元素个数表示各个元素之间互不相同）。

分析：因为给出的集合里面含有重复的元素，而计算 nc 和 nt 不需要考虑两个集合里面是否分别有重复的元素，所以可以直接使用set存储每一个集合，然后把set放进一个数组里面存储。当需要计算集合a和集合b的相似度 nc 和 nt 的时候，遍历集合a中的每一个元素，寻找集合b中是否有该元素，如果有，说明是两个人公共的集合元素，则 $nc++$ ，否则 $nt++$ （ nt 的初值为b集合里面本有的元素）。

```

1 #include <set>

```

```

2  #include <vector>
3  #include <cstdio>
4  using namespace std;
5  int main() {
6      int n, m, k, temp, a, b;
7      scanf("%d", &n);
8      vector<set<int>> v(n);
9      for(int i = 0; i < n; i++) {
10         scanf("%d", &m);
11         set<int> s;
12         for(int j = 0; j < m; j++) {
13             scanf("%d", &temp);
14             s.insert(temp);
15         }
16         v[i] = s;
17     }
18     scanf("%d", &k);
19     for(int i = 0; i < k; i++) {
20         scanf("%d %d", &a, &b);
21         int nc = 0, nt = v[b-1].size();
22         for(auto it = v[a-1].begin(); it != v[a-1].end(); it++) {
23             if(v[b-1].find(*it) == v[b-1].end())
24                 nt++;
25             else
26                 nc++;
27         }
28         double ans = (double)nc / nt * 100;
29         printf("%.2f%%\n", ans);
30     }
31     return 0;
32 }

```

L2-006. 树的遍历 [树的遍历]

给定一棵二叉树的后序遍历和中序遍历，请你输出其层序遍历的序列。这里假设键值都是互不相等的正整数。

输入格式：

输入第一行给出一个正整数N (≤ 30)，是二叉树中结点的个数。第二行给出其后序遍历序列。第三行给出其中序遍历序列。数字间以空格分隔。

输出格式：

在一行中输出该树的层序遍历的序列。数字间以1个空格分隔，行首尾不得有多余空格。

输入样例：

```

7
2 3 1 5 7 6 4

```

1 2 3 4 5 6 7

输出样例:

4 1 6 3 5 7 2

分析: 与后序中序转换为前序的代码相仿 (无须构造二叉树再进行广度优先搜索~), 只不过加一个变量index, 表示当前的根结点在二叉树中所对应的下标 (从0开始), 所以进行一次输出先序的递归的时候, 就可以把根结点下标所对应的值存储在level数组中 (一开始把level都置为-1表示此处没有结点), 这样在递归完成后level数组中非-1的数就是按照下标排列的层序遍历的顺序~~

```
1  #include <cstdio>
2  #include <vector>
3  using namespace std;
4  vector<int> post, in, level(100000, -1);
5  void pre(int root, int start, int end, int index) {
6      if(start > end) return ;
7      int i = start;
8      while(i < end && in[i] != post[root]) i++;
9      level[index] = post[root];
10     pre(root - 1 - end + i, start, i - 1, 2 * index + 1);
11     pre(root - 1, i + 1, end, 2 * index + 2);
12 }
13 int main() {
14     int n, cnt = 0;
15     scanf("%d", &n);
16     post.resize(n);
17     in.resize(n);
18     for(int i = 0; i < n; i++) scanf("%d", &post[i]);
19     for(int i = 0; i < n; i++) scanf("%d", &in[i]);
20     pre(n-1, 0, n-1, 0);
21     for(int i = 0; i < level.size(); i++) {
22         if(level[i] != -1 && cnt != n - 1) {
23             printf("%d ", level[i]);
24             cnt++;
25         } else if(level[i] != -1){
26             printf("%d", level[i]);
27             break;
28         }
29     }
30     return 0;
31 }
```

L2-007. 家庭房产 [并查集]

给定每个人的家庭成员和其自己名下的房产, 请你统计出每个家庭的人口数、人均房产面积及房产套数。

输入格式:

输入第一行给出一个正整数N (≤ 1000), 随后N行, 每行按下列格式给出一个人的房产:

编号 父 母 k 孩子1 ... 孩子k 房产套数 总面积

其中 编号 是每个人独有的一个4位数的编号；父 和 母 分别是该编号对应的这个人的父母的编号（如果已经过世，则显示-1）；k ($0 \leq k \leq 5$) 是该人的子女的个数；孩子i是其子女的编号。

输出格式：

首先在第一行输出家庭个数（所有有亲属关系的人都属于同一个家庭）。随后按下列格式输出每个家庭的信息：

家庭成员的最小编号 家庭人口数 人均房产套数 人均房产面积

其中人均值要求保留小数点后3位。家庭信息首先按人均面积降序输出，若有并列，则按成员编号的升序输出。

输入样例：

10

6666 5551 5552 1 7777 1 100

1234 5678 9012 1 0002 2 300

8888 -1 -1 0 1 1000

2468 0001 0004 1 2222 1 500

7777 6666 -1 0 2 300

3721 -1 -1 1 2333 2 150

9012 -1 -1 3 1236 1235 1234 1 100

1235 5678 9012 0 1 50

2222 1236 2468 2 6661 6662 1 300

2333 -1 3721 3 6661 6662 6663 1 100

输出样例：

3

8888 1 1.000 1000.000

0001 15 0.600 100.000

5551 4 0.750 100.000

分析：用并查集。分别用两个结构体数组，一个data用来接收数据，接收的时候顺便实现了并查集的操作union，另一个数组ans用来输出最后的答案，因为要计算家庭人数，所以用visit标记所有出现过的结点，对于每个结点的父结点，people++统计人数。标记flag == true，计算true的个数cnt就可以知道一共有多少个家庭。排序后输出前cnt个就是所求答案~~

```
1 #include <stdio>
2 #include <algorithm>
3 using namespace std;
4 struct DATA {
```

```

5     int id, fid, mid, num, area;
6     int cid[10];
7 }data[1005];
8 struct node {
9     int id, people;
10    double num, area;
11    bool flag = false;
12 }ans[10000];
13 int father[10000];
14 bool visit[10000];
15 int find(int x) {
16     while(x != father[x])
17         x = father[x];
18     return x;
19 }
20 void Union(int a, int b) {
21     int faA = find(a);
22     int faB = find(b);
23     if(faA > faB)
24         father[faA] = faB;
25     else if(faA < faB)
26         father[faB] = faA;
27 }
28 int cmp1(node a, node b) {
29     if(a.area != b.area)
30         return a.area > b.area;
31     else
32         return a.id < b.id;
33 }
34 int main() {
35     int n, k, cnt = 0;
36     scanf("%d", &n);
37     for(int i = 0; i < 10000; i++)
38         father[i] = i;
39     for(int i = 0; i < n; i++) {
40         scanf("%d %d %d %d", &data[i].id, &data[i].fid, &data[i].mid,
&k);
41         visit[data[i].id] = true;
42         if(data[i].fid != -1) {
43             visit[data[i].fid] = true;
44             Union(data[i].fid, data[i].id);
45         }
46         if(data[i].mid != -1) {
47             visit[data[i].mid] = true;
48             Union(data[i].mid, data[i].id);
49         }
50         for(int j = 0; j < k; j++) {
51             scanf("%d", &data[i].cid[j]);
52             visit[data[i].cid[j]] = true;

```

```

53         Union(data[i].cid[j], data[i].id);
54     }
55     scanf("%d %d", &data[i].num, &data[i].area);
56 }
57 for(int i = 0; i < n; i++) {
58     int id = find(data[i].id);
59     ans[id].id = id;
60     ans[id].num += data[i].num;
61     ans[id].area += data[i].area;
62     ans[id].flag = true;
63 }
64 for(int i = 0; i < 10000; i++) {
65     if(visit[i])
66         ans[find(i)].people++;
67     if(ans[i].flag)
68         cnt++;
69 }
70 for(int i = 0; i < 10000; i++) {
71     if(ans[i].flag) {
72         ans[i].num = (double)(ans[i].num * 1.0 / ans[i].people);
73         ans[i].area = (double)(ans[i].area * 1.0 / ans[i].people);
74     }
75 }
76 sort(ans, ans + 10000, cmp1);
77 printf("%d\n", cnt);
78 for(int i = 0; i < cnt; i++)
79     printf("%04d %d %.3f %.3f\n", ans[i].id, ans[i].people,
ans[i].num, ans[i].area);
80 return 0;
81 }

```

L2-008. 最长对称子串 [字符串处理，动态规划]

对给定的字符串，本题要求你输出最长对称子串的长度。例如，给定“Is PAT&TAP symmetric?”，最长对称子串为“s PAT&TAP s”，于是你应该输出11。

输入格式：

输入在一行中给出长度不超过1000的非空字符串。

输出格式：

在一行中输出最长对称子串的长度。

输入样例：

Is PAT&TAP symmetric?

输出样例：

11

分析：有两种可能，一种是回文字符串的长度为奇数，一种是偶数的情况。i为字符串当前字符的下标。

当回文字串为奇数的时候，j表示i-j与i+j构成的回文字串长度；当回文字串长度为偶数的时候，j表示i+1左边j个字符一直到i右边j个字符的回文字串长度～

用maxvalue保存遍历结果得到的最大值并且输出～

```
1  #include <iostream>
2  using namespace std;
3  int main() {
4      string s;
5      getline(cin, s);
6      int maxvalue = 0, temp;
7      int len = s.length();
8      for(int i = 0; i < len; i++) {
9          temp = 1;
10         for(int j = 1; j < len; j++) {
11             if(i - j < 0 || i + j >= len || s[i - j] != s[i + j])
12                 break;
13             temp += 2;
14         }
15         maxvalue = temp > maxvalue ? temp : maxvalue;
16         temp = 0;
17         for(int j = 1; j < len; j++) {
18             if(i - j + 1 < 0 || i + j >= len || s[i - j + 1] != s[i +
19             j])
20                 break;
21             temp += 2;
22         }
23         maxvalue = temp > maxvalue ? temp : maxvalue;
24     }
25     cout << maxvalue;
26     return 0;
27 }
```

L2-009. 抢红包 [水题]

没有人没抢过红包吧..... 这里给出N个人之间互相发红包、抢红包的记录，请你统计一下他们抢红包的收获。

输入格式：

输入第一行给出一个正整数N (≤ 104)，即参与发红包和抢红包的总人数，则这些人从1到N编号。随后N行，第i行给出编号为i的人发红包的记录，格式如下：

K N1 P1 ... NK PK

其中K ($0 \leq K \leq 20$) 是发出去的红包个数，Ni是抢到红包的人的编号，Pi (> 0) 是其抢到的红包金额（以分为单位）。注意：对于同一个人发出的红包，每人最多只能抢1次，不能重复抢。

输出格式：

按照收入金额从高到低的递减顺序输出每个人的编号和收入金额（以元为单位，输出小数点后2位）。每个人的信息占一行，两数字间有1个空格。如果收入金额有并列，则按抢到红包的个数递减输出；如果还有并列，则按个人编号递增输出。

输入样例：

```
10
3 2 22 10 58 8 125
5 1 345 3 211 5 233 7 13 8 101
1 7 8800
2 1 1000 2 1000
2 4 250 10 320
6 5 11 9 22 8 33 7 44 10 55 4 2
1 3 8800
2 1 23 2 123
1 8 250
4 2 121 4 516 7 112 9 10
```

输出样例：

```
1 11.63
2 3.63
8 3.63
3 2.11
7 1.69
6 -1.67
9 -2.18
10 -3.26
5 -3.26
4 -12.32
```

分析：用结构体存储下每一个结点的id、收入、收到红包的总数，然后进行排序后输出

```
1 #include <cstdio>
2 #include <vector>
3 #include <algorithm>
4 using namespace std;
5 struct node {
```



```

6     int id, total, getnum;
7 };
8 bool cmp1(node a, node b) {
9     if(a.total != b.total)
10         return a.total > b.total;
11     else if(a.getnum != b.getnum)
12         return a.getnum > b.getnum;
13     else
14         return a.id < b.id;
15 }
16 int main() {
17     int n, k, a, b;
18     scanf("%d", &n);
19     vector<node> v(n + 1);
20     for(int i = 1; i <= n; i++) {
21         v[i].id = i;
22         scanf("%d", &k);
23         for(int j = 0; j < k; j++) {
24             scanf("%d %d", &a, &b);
25             v[a].total += b;
26             v[a].getnum++;
27             v[i].total -= b;
28         }
29     }
30     sort(v.begin()+1, v.end(), cmp1);
31     for(int i = 1; i <= n; i++) {
32         double temp = (double)(v[i].total * 1.0 / 100);
33         printf("%d %.2f\n", v[i].id, temp);
34     }
35     return 0;
36 }

```

L2-010. 排座位 [并查集]

布置宴席最微妙的事情，就是给前来参宴的各位宾客安排座位。无论如何，总能把两个死对头排到同一张宴会桌旁！这个艰巨任务现在就交给你，对任何一对客人，请编写程序告诉主人他们是否能被安排同席。

输入格式：

输入第一行给出3个正整数：N（≤ 100），即前来参宴的宾客总人数，则这些人从1到N编号；M为已知两两宾客之间的关系数；K为查询的条数。随后M行，每行给出一对宾客之间的关系，格式为：“宾客1 宾客2 关系”，其中“关系”为1表示是朋友，-1表示是死对头。注意两个人不可能既是朋友又是敌人。最后K行，每行给出一对需要查询的宾客编号。

这里假设朋友的朋友也是朋友。但敌人的敌人并不一定就是朋友，朋友的敌人也不一定是敌人。只有单纯直接的敌对关系才是绝对不能同席的。

输出格式：

对每个查询输出一行结果：如果两位宾客之间是朋友，且没有敌对关系，则输出“No problem”；如果他们之间并不是朋友，但也不敌对，则输出“OK”；如果他们之间有敌对，然而也有共同的朋友，则输出“OK but...”；如果他们之间只有敌对关系，则输出“No way”。

输入样例：

7 8 4

5 6 1

2 7 -1

1 3 1

3 4 1

6 7 -1

1 2 1

1 4 1

2 3 -1

3 4

5 7

2 3

7 2

输出样例：

No problem

OK

OK but...

No way

分析：朋友之间的关系用并查集解决，敌人之间的关系用 $\text{enemy}[a][b] = \text{enemy}[b][a] = 1$ 解决，因为朋友之间的朋友也是朋友，是传递关系，而敌人的敌人不一定是敌人，所以只需要用一个二维数组即可标记。

```
1 #include <cstdio>
2 #include <vector>
3 using namespace std;
4 vector<int> fri;
5 int enemy[101][101];
6 int findfri(int x) {
7     while(x != fri[x])
8         x = fri[x];
9     return x;
10 }
11 void Union(int a, int b) {
```

```

12     int fria = findfri(a);
13     int frib = findfri(b);
14     if(fria != frib)
15         fri[fria] = frib;
16 }
17 int main() {
18     int n, m, k, a, b, c;
19     scanf("%d %d %d", &n, &m, &k);
20     fri.resize(n + 1);
21     for(int i = 1; i <= n; i++)
22         fri[i] = i;
23     for(int i = 0; i < m; i++) {
24         scanf("%d %d %d", &a, &b, &c);
25         if(c == 1)
26             Union(a, b);
27         else {
28             enemy[a][b] = 1;
29             enemy[b][a] = 1;
30         }
31     }
32     for(int i = 0; i < k; i++) {
33         scanf("%d %d", &a, &b);
34         if(findfri(a) == findfri(b) && enemy[a][b] == 0)
35             printf("No problem\n");
36         else if(findfri(a) != findfri(b) && enemy[a][b] == 0)
37             printf("OK\n");
38         else if(findfri(a) == findfri(b) && enemy[a][b] == 1)
39             printf("OK but...\n");
40         else if(enemy[a][b] == 1)
41             printf("No way\n");
42     }
43     return 0;
44 }

```

L2-011. 玩转二叉树 [二叉树的遍历]

给定一棵二叉树的中序遍历和前序遍历，请你先将树做个镜面反转，再输出反转后的层序遍历的序列。所谓镜面反转，是指将所有非叶结点的左右孩子对换。这里假设键值都是互不相等的正整数。

输入格式：

输入第一行给出一个正整数N（≤30），是二叉树中结点的个数。第二行给出其中序遍历序列。第三行给出其前序遍历序列。数字间以空格分隔。

输出格式：

在一行中输出该树反转后的层序遍历的序列。数字间以1个空格分隔，行首尾不得有多余空格。

输入样例：

7

1 2 3 4 5 6 7

4 1 3 2 6 5 7

输出样例：

4 6 1 7 5 3 2

分析：与前序中序转换为后序的代码相仿（无须构造二叉树再进行广度优先搜索~），只不过加一个变量index，表示当前的根结点在二叉树中所对应的下标（从0开始），所以进行一次输出先序的递归的时候，就可以把根结点下标所对应的值存储在level数组中（一开始把level都置为-1表示此处没有结点），镜面反转只需改变index的值，左孩子为 $2 * index + 2$ ，右孩子为 $2 * index + 1$ ，这样在递归完成后level数组中非-1的数就是按照下标排列的层序遍历的顺序~

如果你想知道如何通过前序中序转换为后序，请戳-》<http://www.liuchuo.net/archives/2087>

```
1  #include <cstdio>
2  #include <vector>
3  using namespace std;
4  vector<int> in, pre, level(100000, -1);
5  void levelorder(int root, int start, int end, int index) {
6      if(start > end) return ;
7      int i = start;
8      while(i < end && in[i] != pre[root]) i++;
9      level[index] = pre[root];
10     levelorder(root + 1, start, i - 1, 2 * index + 2);
11     levelorder(root + 1 + i - start, i + 1, end, 2 * index + 1);
12 }
13 int main() {
14     int n, cnt = 0;
15     scanf("%d", &n);
16     in.resize(n);
17     pre.resize(n);
18     for(int i = 0; i < n; i++) scanf("%d", &in[i]);
19     for(int i = 0; i < n; i++) scanf("%d", &pre[i]);
20     levelorder(0, 0, n-1, 0);
21     for(int i = 0; i < level.size(); i++) {
22         if(level[i] != -1 && cnt != n - 1) {
23             printf("%d ", level[i]);
24             cnt++;
25         } else if(level[i] != -1){
26             printf("%d", level[i]);
27             break;
28         }
29     }
30     return 0;
31 }
```

L2-012. 关于堆的判断 [heap sort（堆排序）]

将一系列给定数字顺序插入一个初始为空的小顶堆H[]。随后判断一系列相关命题是否为真。命题分下列几种：

“x is the root”：x是根结点；

“x and y are siblings”：x和y是兄弟结点；

“x is the parent of y”：x是y的父结点；

“x is a child of y”：x是y的一个子结点。

输入格式：

每组测试第1行包含2个正整数N（≤ 1000）和M（≤ 20），分别是插入元素的个数、以及需要判断的命题数。下一行给出区间[-10000, 10000]内的N个要被插入一个初始为空的小顶堆的整数。之后M行，每行给出一个命题。题目保证命题中的结点键值都是存在的。

输出格式：

对输入的两个命题，如果其为真，则在一行中输出“T”，否则输出“F”。

输入样例：

5 4

46 23 26 24 10

24 is the root

26 and 23 are siblings

46 is the parent of 23

23 is a child of 10

输出样例：

F

T

F

T

分析：必须注意，因为题目要求按照插入的顺序建立，所以是边插入边调整的，必须用向上调整，每次输入一个数之后就将其向上调整。（两者建立出来的二叉树不同）而不能采用先转换为二叉树的方式再向下调整。

```
1  #include <cstdio>
2  #include <vector>
3  #include <cstring>
4  using namespace std;
5  vector<int> v;
6  int n;
7  void upAdjust(int i) {
8      if(i == 1) return ;
```

```

9     while(i != 1) {
10         if(v[i] < v[i / 2]) {
11             swap(v[i], v[i / 2]);
12             i = i / 2;
13         } else {
14             break;
15         }
16     }
17 }
18 void judge1(int a) {
19     if(v[1] == a) printf("T\n");
20     else printf("F\n");
21 }
22 void judge2(int a, int b) {
23     int indexa = 0, indexb = 0;
24     for(int i = 1; i <= n; i++) {
25         if(v[i] == a) indexa = i;
26         if(v[i] == b) indexb = i;
27     }
28     if(indexa > indexb) swap(indexa, indexb);
29     if(indexa % 2 == 0 && indexb - indexa == 1) printf("T\n");
30     else printf("F\n");
31 }
32 void judge3(int a, int b) {
33     int indexa = 0, indexb = 0;
34     for(int i = 1; i <= n; i++) {
35         if(v[i] == a) indexa = i;
36         if(v[i] == b) indexb = i;
37     }
38     if((2 * indexa) == indexb || (2 * indexa) + 1 == indexb)
printf("T\n");
39     else printf("F\n");
40 }
41 void judge4(int a, int b) {
42     int indexa = 0, indexb = 0;
43     for(int i = 1; i <= n; i++) {
44         if(v[i] == a) indexa = i;
45         if(v[i] == b) indexb = i;
46     }
47     if(indexa == indexb * 2 || indexa == 2 * indexb + 1) printf("T\n");
48     else printf("F\n");
49 }
50 int main() {
51     int k, a, b;
52     char c[100];
53     scanf("%d%d", &n, &k);
54     v.resize(n + 1);
55     for(int i = 1; i <= n; i++) {
56         scanf("%d", &v[i]);

```

```

57         upAdjust(i);
58     }
59     for(int i = 0; i < k; i++) {
60         scanf("%d%s", &a, c);
61         if(strcmp(c, "and") == 0) {
62             scanf("%d%s%s", &b, c, c);
63             judge2(a, b);
64         } else {
65             scanf("%s", c);
66             if(strcmp(c, "a") == 0) {
67                 scanf("%s%s%d", c, c, &b);
68                 judge4(a, b);
69             } else {
70                 scanf("%s", c);
71                 if(strcmp(c, "root") == 0) {
72                     judge1(a);
73                 } else {
74                     scanf("%s%d", c, &b);
75                     judge3(a, b);
76                 }
77             }
78         }
79     }
80     return 0;
81 }

```

L2-013. 红色警报 [深度优先搜索DFS，图的连通分量个数统计]

战争中保持各个城市间的连通性非常重要。本题要求你编写一个报警程序，当失去一个城市导致国家被分裂为多个无法连通的区域时，就发出红色警报。注意：若该国本来就不完全连通，是分裂的k个区域，而失去一个城市并不改变其他城市之间的连通性，则不要发出警报。

输入格式：

输入在第一行给出两个整数N（ $0 < N \leq 500$ ）和M（ ≤ 5000 ），分别为城市个数（于是默认城市从0到N-1编号）和连接两城市的通路条数。随后M行，每行给出一条通路所连接的两个城市的编号，其间以1个空格分隔。在城市信息之后给出被攻占的信息，即一个正整数K和随后的K个被攻占的城市的编号。

注意：输入保证给出的被攻占的城市编号都是合法的且无重复，但并不保证给出的通路没有重复。

输出格式：

对每个被攻占的城市，如果它会改变整个国家的连通性，则输出“Red Alert: City k is lost!”，其中k是该城市的编号；否则只输出“City k is lost.”即可。如果该国失去了最后一个城市，则增加一行输出“Game Over.”。

输入样例：

5 4

0 1

1 3

3 0

0 4

5

1 2 0 4 3

输出样例:

City 1 is lost.

City 2 is lost.

Red Alert: City 0 is lost!

City 4 is lost.

City 3 is lost.

Game Over.

分析：用图的深度优先遍历判断一个图内的连通分量有多少个，标记为cnt，之后对于每一个输入数据，因为城市a被攻占，所以把a的所有路径标注为不可达(0)，再统计连通分量的个数tempcnt，如果tempcnt > cnt + 1，也就是说当现在的连通分量多余以前的连通分量+1的时候，说明改变了图的连通性；（因为城市被攻占本身它城市自己就变成了一个单独的城市，多出来一个连通分量，只要tempcnt <= cnt + 1都说明没有改变图的连通性），每一次tempcnt在用完之后把cnt的值更新为tempcnt，保证下一次的判断是建立再已经失去之前这么多城市的基础之上的。

因为题目中说输入保证给出的被攻占的城市编号都是合法的且无重复，所以如果城市失去了n个，就是当前输入的是从0开始的第n-1个数据的时候，就说明Game Over了，最后当if(i == n - 1) printf("Game Over.\n");

```
1  #include <cstdio>
2  #include <algorithm>
3  using namespace std;
4  bool visit[510];
5  int e[510][510], n, m, k;
6  void dfs(int node) {
7      visit[node] = true;
8      for(int i = 0; i < n; i++) {
9          if(visit[i] == false && e[node][i] == 1)
10             dfs(i);
11     }
12 }
13 int countcnt() {
14     int cnt = 0;
15     fill(visit, visit + 510, false);
16     for(int i = 0; i < n; i++) {
17         if(visit[i] == false) {
18             dfs(i);
19             cnt++;
20         }
21     }
22     return cnt;
23 }
```

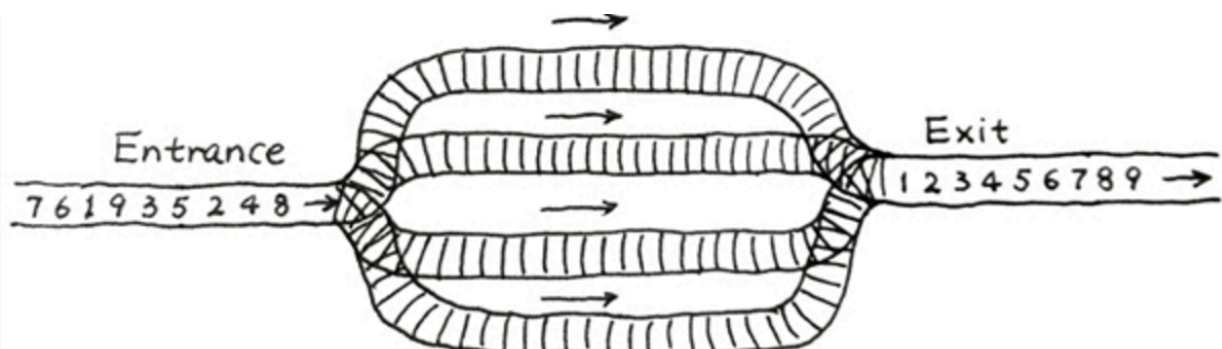


```

20     }
21 }
22 return cnt;
23 }
24 int main() {
25     scanf("%d%d", &n, &m);
26     int a, b, city;
27     for(int i = 0; i < m; i++) {
28         scanf("%d%d", &a, &b);
29         e[a][b] = e[b][a] = 1;
30     }
31     int cnt = countcnt();
32     scanf("%d", &k);
33     for(int i = 0; i < k; i++) {
34         scanf("%d", &city);
35         for(int j = 0; j < n; j++) {
36             if(e[city][j] == 1) {
37                 e[city][j] = 0;
38                 e[j][city] = 0;
39             }
40         }
41         int tempcnt = countcnt();
42         if(tempcnt > cnt + 1)
43             printf("Red Alert: City %d is lost!\n", city);
44         else
45             printf("City %d is lost.\n", city);
46         cnt = tempcnt;
47         if(i == n - 1) printf("Game Over.\n");
48     }
49     return 0;
50 }

```

L2-014. 列车调度 [集合set, STL的使用]



两端分别是一条入口（Entrance）轨道和一条出口（Exit）轨道，它们之间有N条平行的轨道。每趟列车从入口可以选择任意一条轨道进入，最后从出口离开。在图中有9趟列车，在入口处按照{8，4，2，5，3，9，1，6，7}的顺序排队等待进入。如果要求它们必须按序号递减的顺序从出口离开，则至少需要多少条平行铁轨用于调度？

输入格式：

输入第一行给出一个整数N ($2 \leq N \leq 105$), 下一行给出从1到N的整数序号的一个重排列。数字间以空格分隔。

输出格式:

在一行中输出可以将输入的列车按序号递减的顺序调离所需要的最少的铁轨条数。

输入样例:

```
9
8 4 2 5 3 9 1 6 7
```

输出样例:

```
4
```

分析: 必须要车号大的先出, 小的后出。所以列车排队的每一队必须是从大到小排列 (从右往左看), 才能保证开出去的车也是从大到小的。对于每一个想要进入并列铁轨的车, 如果车号大于每一队的队尾的车号, 说明不能进入已有的队伍, 必须进入新的铁轨, 否则, 选择一个最接近它车号的尾部车号的队伍进入。其实无需保存每一个并行队列的所有值, 只需要保存当前队伍的车尾 (就是每一列最左边 即 每一列的最小值) 即可, 因为每一次都是需要排序比较大小的, 所以用set自动排序, 首先把set里面放入一个0值。每一次set的最后一个值s.rbegin()都是当前所有队列队尾的最大值。如果当前想要进入排队队伍的t值比集合里面最大值小, 就移除第一个比他大的值, 然后把t插入集合中。表示的是将t值插入了最接近它车号的队伍的队尾, 否则就直接插入进去t值。作为新的队伍。s.upper_bound(t)返回的是第一个大于t的迭代器的位置, 在前面加星号表示取这个位置的值, 所以s.erase(*(s.upper_bound(t)));表示删除当前这个刚好大于t的位置处的值。

因为一开始插入了一个没有的0, 所以最后输出是s.size()-1;

```
1  #include <iostream>
2  #include <set>
3  using namespace std;
4  int main() {
5      int n, t;
6      cin >> n;
7      set<int> s;
8      s.insert(0);
9      for(int i = 0; i < n; i++) {
10         cin >> t;
11         if(t < *s.rbegin()) {
12             s.erase(*(s.upper_bound(t)));
13         }
14         s.insert(t);
15     }
16     cout << s.size() - 1;
17     return 0;
18 }
```

L2-015. 互评成绩 [排序]

学生互评作业的简单规则是这样定的：每个人的作业会被k个同学评审，得到k个成绩。系统需要去掉一个最高分和一个最低分，将剩下的分数取平均，就得到这个学生的最后成绩。本题就要求你编写这个互评系统的算分模块。

输入格式：

输入第一行给出3个正整数N ($3 < N \leq 104$ ，学生总数)、k ($3 \leq k \leq 10$ ，每份作业的评审数)、M (≤ 20 ，需要输出的学生数)。随后N行，每行给出一份作业得到的k个评审成绩（在区间[0, 100]内），其间以空格分隔。

输出格式：

按非递减顺序输出最后得分最高的M个成绩，保留小数点后3位。分数间有1个空格，行首尾不得有多余空格。

输入样例：

```
6 5 3
88 90 85 99 60
67 60 80 76 70
90 93 96 99 99
78 65 77 70 72
88 88 88 88 88
55 55 55 55 55
```

输出样例：

```
87.667 88.000 96.000
```

分析：total数组保存各个同学的平均分，v数组保存每次接收得到的分数，排序后取前m名，按递增输出

kuuo同学说不用排序，标记最大和最小值，用总和减去最大最小值即可，于是我偷偷的改了代码。。。嘘。☹️。~~

```
1  #include <stdio>
2  #include <vector>
3  #include <algorithm>
4  using namespace std;
5  int cmp1(double a, double b) {return a > b;}
6  int main() {
7      int n, k, m;
8      scanf("%d %d %d", &n, &k, &m);
9      vector<double> total;
10     for(int i = 0; i < n; i++) {
11         vector<double> v(k);
12         double maxn = -1, minn = 101, average = 0.0;
13         for(int j = 0; j < k; j++) {
14             scanf("%lf", &v[j]);
```

```

15         maxn = max(maxn, v[j]);
16         minn = min(minn, v[j]);
17         average += v[j];
18     }
19     average = (average - maxn - minn) / (k - 2);
20     total.push_back(average);
21 }
22 sort(total.begin(), total.end(), cmp1);
23 if(m != 0)
24     printf("%.3f", total[m - 1]);
25 for(int i = m - 2; i >= 0; i--)
26     printf(" %.3f", total[i]);
27 return 0;
28 }

```

L2-016. 愿天下有情人都是失散多年的兄妹 [广度优先搜索BFS]

呵呵。大家都知道五服以内不得通婚，即两个人最近共同祖先如果在五代以内（即本人、父母、祖父母、曾祖父母、高祖父母）则不可通婚。本题就请你帮助一对有情人判断一下，他们究竟是否可以成婚？

输入格式：

输入第一行给出一个正整数N（ $2 \leq N \leq 104$ ），随后N行，每行按以下格式给出一个人的信息：

本人ID 性别 父亲ID 母亲ID

其中ID是5位数字，每人不同；性别M代表男性、F代表女性。如果某人的父亲或母亲已经不可考，则相应的ID位置上标记为-1。

接下来给出一个正整数K，随后K行，每行给出一对有情人的ID，其间以空格分隔。

注意：题目保证两个人是同辈，每人只有一个性别，并且血缘关系网中没有乱伦或隔辈成婚的情况。

输出格式：

对每一对有情人，判断他们的关系是否可以通婚：如果两人是同性，输出“Never Mind”；如果是异性并且关系出了五服，输出“Yes”；如果异性关系未出五服，输出“No”。

输入样例：

24

00001 M 01111 -1

00002 F 02222 03333

00003 M 02222 03333

00004 F 04444 03333

00005 M 04444 05555

00006 F 04444 05555

00007 F 06666 07777

00008 M 06666 07777

00009 M 00001 00002

00010 M 00003 00006

00011 F 00005 00007

00012 F 00008 08888

00013 F 00009 00011

00014 M 00010 09999

00015 M 00010 09999

00016 M 10000 00012

00017 F -1 00012

00018 F 11000 00013

00019 F 11100 00018

00020 F 00015 11110

00021 M 11100 00020

00022 M 00016 -1

00023 M 10012 00017

00024 M 00022 10013

9

00021 00024

00019 00024

00011 00012

00022 00018

00001 00004

00013 00016

00017 00015

00019 00021

00010 00011

输出样例：

Never Mind

Yes

Never Mind

No

Yes

No

Yes

No

No

分析：我用的方法是广度优先，然后把每个人和他们的祖先们压入一个set里面，判断set前后有没有大小改变，如果没改变说明重复了，所以有相同祖先，所以就输出No，用level数组标记他们当前的层数在push一层的时候令他们的层数为上一层+1，一直到五层判断结束。如果结束了还没有输出过No，那么就输出yes。（其实后来想到可以用数组标记的方法，不用集合来这么复杂的。。）

有三个注意点：1.一开始有三个测试点不过，然后。。发现。。每次在输入的时候把它们父母的性别也要标记一下，因为！测试数据里面可能会判断某个人的父亲（但是不在给出的本人id的列表中），或者母亲和某个人是否能结婚。。。也就是说测试数据里面如果出现了09999，它是某个人的父母，那么也可能被判断。。所以。。所以。。要在输入的时候加上父母的性别的标记。。。PAT教我做人。。

2.因为我开的是全局数组，用全局数组的时候就千万要记得，用完之后记得把它。。置回0啊啊啊。。因为下一组测试数据用这个数组的时候可能会不小心用到了上一组测试数据保存的结果。。。就会出错了。。

3.用exist数组标记是否存在这个结点，否则避免不小心把0压入。。。到队列里面（就是求对于一个父母，他们可能不存在在本人id里面，那么把他们的孩子结点放入queue里面就会出现0。。不断标记0的话把0当他们的共同祖先可能会误输出No）

```
1  #include <cstdio>
2  #include <set>
3  #include <queue>
4  #include <algorithm>
5  using namespace std;
6  struct node {
7      int f, m, sex;
8  };
9  node v[100010];
10 int level[100010];
11 bool exist[100010];
12 int main() {
13     int n, m, id, father, mother, a, b;
14     scanf("%d", &n);
15     char c;
16     for(int i = 0; i < n; i++) {
17         scanf("%d %c %d %d", &id, &c, &father, &mother);
18         v[id].f = father, v[id].m = mother;
19         if(c == 'M') v[id].sex = 0;
20         else v[id].sex = 1;
21         exist[id] = true;
```

```

22     v[father].sex = 0;
23     v[mother].sex = 1;
24 }
25 scanf("%d", &m);
26 for(int i = 0; i < m; i++) {
27     fill(level, level + 100010, 0);
28     scanf("%d%d", &a, &b);
29     if(v[a].sex == v[b].sex) {
30         printf("Never Mind\n");
31         continue;
32     }
33     queue<int> q;
34     q.push(a);
35     q.push(b);
36     level[a] = 1;
37     level[b] = 1;
38     set<int> s;
39     int flag = 0;
40     while(!q.empty()) {
41         int top = q.front();
42         q.pop();
43         int size = s.size();
44         s.insert(top);
45         if(size == s.size()) {
46             printf("No\n");
47             flag = 1;
48             break;
49         }
50         if(exist[top] == false) continue;
51         if(level[top] <= 4) {
52             int fa = v[top].f;
53             int mo = v[top].m;
54             if(fa != -1) {
55                 q.push(fa);
56                 level[fa] = level[top] + 1;
57             }
58             if(mo != -1) {
59                 q.push(mo);
60                 level[mo] = level[top] + 1;
61             }
62         }
63     }
64     if(flag == 0) printf("Yes\n");
65 }
66 return 0;
67 }

```

L2-017. 人以群分 [排序]

社交网络中我们给每个人定义了一个“活跃度”，现希望根据这个指标把人群分为两大类，即外向型（outgoing，即活跃度高的）和内向型（introverted，即活跃度低的）。要求两类人群的规模尽可能接近，而他们的总活跃度差距尽可能拉开。

输入格式：

输入第一行给出一个正整数N ($2 \leq N \leq 105$)。随后一行给出N个正整数，分别是每个人的活跃度，其间以空格分隔。题目保证这些数字以及它们的和都不会超过231。

输出格式：

按下列格式输出：

Outgoing #: N1

Introverted #: N2

Diff = N3

其中 N1 是外向型人的个数；N2 是内向型人的个数；N3 是两群人总活跃度之差的绝对值。

输入样例1：

10

23 8 10 99 46 2333 46 1 666 555

输出样例1：

Outgoing #: 5

Introverted #: 5

Diff = 3611

输入样例2：

13

110 79 218 69 3721 100 29 135 2 6 13 5188 85

输出样例2：

Outgoing #: 7

Introverted #: 6

Diff = 9359

分析：接收所有数据到数组v中，并且将所有数据的总和累加为sum，然后对数组从小到大排序，计算前一半的总和为half，那么外向的人的个数就为 $(n + 1) / 2$ ，因为如果n是偶数就是 $n/2$ ，如果n是奇数就是 $n/2 + 1$ ，总的来说就是 $(n + 1) / 2$ ，内向的人的个数就是 $n/2$ ，两者之间的差就是 $(sum - half) - half$ 即 $sum - 2 * half \sim \sim$

```
1 #include <cstdio>
2 #include <vector>
3 #include <algorithm>
4 using namespace std;
```



```

5  int main() {
6      int n, sum = 0, half = 0;
7      scanf("%d", &n);
8      vector<int> v(n);
9      for (int i = 0; i < n; i++) {
10         scanf("%d", &v[i]);
11         sum += v[i];
12     }
13     sort(v.begin(), v.end());
14     for(int i = 0; i < n / 2; i++)
15         half += v[i];
16     printf("Outgoing #: %d\nIntroverted #: %d\nDiff = %d", (n + 1) / 2,
17            n / 2, sum - 2 * half);
18     return 0;
19 }

```

L2-018. 多项式A除以B [多项式除法]

这仍然是一道关于A/B的题，只不过A和B都换成了多项式。你需要计算两个多项式相除的商Q和余R，其中R的阶数必须小于B的阶数。

输入格式：

输入分两行，每行给出一个非零多项式，先给出A，再给出B。每行的格式如下：

N e[1] c[1] ... e[N] c[N]

其中N是该多项式非零项的个数，e[i]是第i个非零项的指数，c[i]是第i个非零项的系数。各项按照指数递减的顺序给出，保证所有指数是各不相同的非负整数，所有系数是非零整数，所有整数在整型范围内。

输出格式：

分两行先后输出商和余，输出格式与输入格式相同，输出的系数保留小数点后1位。同行数字间以1个空格分隔，行首尾不得有多余空格。注意：零多项式是一个特殊多项式，对应输出为0 0 0.0。但非零多项式不能输出零系数（包括舍入后为0.0）的项。在样例中，余多项式其实有常数项-1/27，但因其舍入后为0.0，故不输出。

输入样例：

4 4 1 2 -3 1 -1 0 -1

3 2 3 1 -2 0 1

输出样例：

3 2 0.3 1 0.2 0 -1.0

1 1 -3.1

分析：对于两个多项式A和B，题目给出的必定不会是连续降幂的，根据多项式的除法原理，我们需要缺幂项补零。例如，题中给出的 x^4-3x^2-x-1 是缺3次幂的，将缺幂项补上之后，就变成了 $x^4+0x^3-3x^2-x-1$ 。由此，我们可以用一个数组来保存一个多项式，即数组的下标对应多项式的指数，下标对应的单元表示多项式的系数，如数组 $[-1, -1, -3, 0, 1]$ 。

若已知A多项式的最高次幂为t1, B多项式的最高次幂为t2, 则第一次除法商的最高次幂为t1 - t2, 最高次幂的系数为A[t1] / B[t2], 然后用A[i] -= B[i - (t1 - t2)] * A[t1] / B[t2], 其中i从A的最高次幂t1到大于等于t1 - t2, 这样就算完成了一个除法了。例如A = [-1, -1, -3, 0, 1], B = [1, -2, 3], 则t1 = 4, t2 = 2, 所以第一次除法商的最高次幂为2, 系数为A[4] / A[2] = 0.3, 循环A[i] -= B[i - (t1 - t2)] * A[t1] / B[t2], i从4到2, 得到新的A = [-1, -1, -10/3, 2/3, 0], 然后重复上面的步骤, 直到A的最高项幂次小于B的最高项幂次, 此时的A就是余项。

[两个可能会让结果出现非零项多项式的测试用例]

1 2 1

1 3 1

1 2 1

1 2 1

[一个比较好算一点的一般测试用例]

3 3 1 2 -12 0 -42

2 1 1 0 -3

// ouput

3 2 1.0 1 -9.0 0 -27.0

1 0 -123.0

具体代码如下:

```
1  #include <stdio>
2  #include <cmath>
3  using namespace std;
4  int nonNegativeNum(double c[], int start) {
5      int cnt = 0;
6      for (int i = start; i >= 0; i--)
7          if (abs(c[i]) + 0.05 >= 0.1) cnt++;
8      return cnt;
9  }
10 void printPoly(double c[], int start) {
11     printf("%d", nonNegativeNum(c, start));
12     if (nonNegativeNum(c, start) == 0) printf(" 0 0.0");
13     for (int i = start; i >= 0; i--)
14         if (abs(c[i]) + 0.05 >= 0.1)
15             printf(" %d %.1f", i, c[i]);
16 }
17 double c1[3000], c2[3000], c3[3000];
18 int main() {
19     int m = 0, n = 0, t = 0, max1 = -1, max2 = -1;
20     scanf("%d", &m);
21     for (int i = 0; i < m; i++) {
22         scanf("%d", &t);
```

```

23         max1 = max1 > t ? max1 : t;
24         scanf("%lf", &c1[t]);
25     }
26     scanf("%d", &n);
27     for (int i = 0; i < n; i++) {
28         scanf("%d", &t);
29         max2 = max2 > t ? max2 : t;
30         scanf("%lf", &c2[t]);
31     }
32     int t1 = max1, t2 = max2;
33     while (t1 >= t2) {
34         double c = c1[t1] / c2[t2];
35         c3[t1 - t2] = c;
36         for (int i = t1, j = t2; j >= 0; j--, i--) c1[i] -= c2[j] * c;
37         while (abs(c1[t1]) < 0.000001) t1--;
38     }
39     printPoly(c3, max1 - max2);
40     printf("\n");
41     printPoly(c1, t1);
42     return 0;
43 }

```

L2-019. 悄悄关注 [集合set]

新浪微博上有个“悄悄关注”，一个用户悄悄关注的人，不出现在这个用户的关注列表上，但系统会推送其悄悄关注的人发表的微博给该用户。现在我们来做一回网络侦探，根据某人的关注列表和其对其他用户的点赞情况，扒出有可能被其悄悄关注的人。

输入格式：

输入首先在第一行给出某用户的关注列表，格式如下：

人数N 用户1 用户2 用户N

其中N是不超过5000的正整数，每个“用户i” (i=1, ..., N) 是被其关注的用户的ID，是长度为4位的由数字和英文字母组成的字符串，各项间以空格分隔。

之后给出该用户点赞的信息：首先给出一个不超过10000的正整数M，随后M行，每行给出一个被其点赞的用户ID和对该用户的点赞次数（不超过1000），以空格分隔。注意：用户ID是一个用户的唯一身份标识。题目保证在关注列表中没有重复用户，在点赞信息中也没有重复用户。

输出格式：

我们认为被该用户点赞次数大于其点赞平均数、且不在其关注列表上的人，很可能是其悄悄关注的人。根据这个假设，请你按用户ID字母序的升序输出可能是其悄悄关注的人，每行1个ID。如果其实并没有这样的人，则输出“Bing Mei You”。

输入样例1：

10 GAO3 Magi Zha1 Sen1 Quan FaMK LSum Eins FatM LLao

8

Magi 50

Pota 30

LLao 3

Ammy 48

Dave 15

GAO3 31

Zoro 1

Cath 60

输出样例1:

Ammy

Cath

Pota

输入样例2:

11 GAO3 Magi Zha1 Sen1 Quan FaMK LSum Eins FatM LLao Pota

7

Magi 50

Pota 30

LLao 48

Ammy 3

Dave 15

GAO3 31

Zoro 29

输出样例2:

Bing Mei You

分析：将关注的人存储在集合set里，将点赞的人和点赞的次数存储在map中，并统计点赞的平均次数 sum / M ，遍历map，如果map的值大于平均次数，且在set中找不到该用户名，就输出当前用户名（因为map中的键是已经按照字典序排序过的，所以直接输出就可以），并用flag标记是否有过输出，如果从始至终没有输出，说明没有悄悄关注的人，就输出Bing Mei You~~~

```
1 #include <iostream>
2 #include <map>
3 #include <set>
4 using namespace std;
5 int main() {
```

```

6      int N, M, sum = 0, cnt, flag = 0;
7      string str;
8      cin >> N;
9      set<string> s;
10     for (int i = 0; i < N; i++) {
11         cin >> str;
12         s.insert(str);
13     }
14     cin >> M;
15     map<string, int> m;
16     for (int i = 0; i < M; i++) {
17         cin >> str >> cnt;
18         m[str] = cnt;
19         sum += cnt;
20     }
21     sum /= M;
22     for (auto it : m) {
23         if (it.second > sum && s.find(it.first) == s.end()) {
24             cout << it.first << endl;
25             flag = 1;
26         }
27     }
28     if (flag == 0) cout << "Bing Mei You";
29     return 0;
30 }

```

L2-020. 功夫传人 [深度优先搜索DFS]

一门武功能否传承久远并被发扬光大，是要看缘分的。一般来说，师傅传授给徒弟的武功总要打个折扣，于是越往后传，弟子们的功夫就越弱..... 直到某一支的某一代突然出现一个天分特别高的弟子（或者是吃到了灵丹、挖到了特别的秘笈），会将功夫的威力一下子放大N倍 —— 我们称这种弟子为“得道者”。

这里我们来考察某一位祖师爷门下的徒子徒孙家谱：假设家谱中的每个人只有1位师傅（除了祖师爷没有师傅）；每位师傅可以带很多徒弟；并且假设辈分严格有序，即祖师爷这门武功的每个第*i*代传人只能在第*i-1*代传人中拜1个师傅。我们假设已知祖师爷的功力值为Z，每向下传承一代，就会减弱r%，除非某一代弟子得道。现给出师门谱系关系，要求你算出所有得道者的功力总值。

输入格式：

输入在第一行给出3个正整数，分别是：N（≤105）——整个师门的总人数（于是每个人从0到N-1编号，祖师爷的编号为0）；Z——祖师爷的功力值（不一定是整数，但起码是正数）；r——每传一代功夫所打的折扣百分比值（不超过100的正数）。接下来有N行，第*i*行（*i*=0, ..., N-1）描述编号为*i*的人所传的徒弟，格式为：

K_i ID[1] ID[2] ... ID[*K_i*]

其中*K_i*是徒弟的个数，后面跟的是各位徒弟的编号，数字间以空格间隔。*K_i*为零表示这是一位得道者，这时后面跟的一个数字表示其武功被放大的倍数。

输出格式：

在一行中输出所有得道者的功力总值，只保留其整数部分。题目保证输入和正确的输出都不超过1010。

输入样例：

10 18.0 1.00

3 2 3 5

1 9

1 4

1 7

0 7

2 6 1

1 8

0 9

0 4

0 3

输出样例：

404

分析：一开始没看懂题意，以为是放大师傅的功力，原来是师傅功力减弱 $r\%$ 后的功力。。用二维数组 v 存储师门谱系关系， $v[i]$ 表示编号为 i 的师傅所拥有的徒弟，如果徒弟个数等于0，也就是说这是个得道者，那么 $v[i][0]$ 保存放大的倍数，而且用 $visit[i] = true$ 标记当前的这个编号的人是得道者～用深度优先搜索，每当遇到 $visit[index] = true$ 也就是说这是个得道者的时候，就累加放大后的功力， $power * v[index][0]$ ，累加到 $result$ 中～遍历 $v[index]$ 的所有弟子，并将功力减弱 $r\%$ ，也就是 $power * (1 - r/100)$ ，最后输出的是 $result$ 的整数值 $(int)result \sim \sim \sim$

```
1  #include <stdio>
2  #include <vector>
3  using namespace std;
4  vector<vector<int> > v;
5  int n, k;
6  double z, r, result = 0.0;
7  bool visit[100010];
8  void dfs(int index, double power) {
9      if (visit[index] == true) {
10         result += power * v[index][0];
11         return;
12     }
13     for (int i = 0; i < v[index].size(); i++)
14         dfs(v[index][i], power * (1 - r/100));
15 }
16 int main() {
17     scanf("%d%lf%lf", &n, &z, &r);
```

```

18     v.resize(n);
19     for (int i = 0; i < n; i++) {
20         scanf("%d", &k);
21         int temp;
22         if (k == 0) {
23             scanf("%d", &temp);
24             v[i].push_back(temp);
25             visit[i] = true;
26         }
27         for (int j = 0; j < k; j++) {
28             scanf("%d", &temp);
29             v[i].push_back(temp);
30         }
31     }
32     dfs(0, z);
33     printf("%d", (int)result);
34     return 0;
35 }

```

L2-021. 点赞狂魔 [排序]

微博上有个“点赞”功能，你可以为你喜欢的博文点个赞表示支持。每篇博文都有一些刻画其特性的标签，而你点赞的博文的类型，也间接刻画了你的特性。然而有这么一种人，他们会通过给自己看到的一切内容点赞来狂刷存在感，这种人就被称为“点赞狂魔”。他们点赞的标签非常分散，无法体现出明显的特性。本题就要求你写个程序，通过统计每个人点赞的不同标签的数量，找出前3名点赞狂魔。

输入格式：

输入在第一行给出一个正整数N (≤ 100)，是待统计的用户数。随后N行，每行列出一位用户的点赞标签。格式为“Name K F₁ ... F_K”，其中 Name 是不超过8个英文小写字母的非空用户名， $1 \leq K \leq 1000$ ，F_i ($i=1, \dots, K$) 是特性标签的编号，我们将所有特性标签从1到 10^7 编号。数字间以空格分隔。

输出格式：

统计每个人点赞的不同标签的数量，找出数量最大的前3名，在一行中顺序输出他们的用户名,其间以1个空格分隔,且行末不得有多余空格。如果有并列，则输出标签出现次数平均值最小的那个，题目保证这样的用户没有并列。若不足3人，则用“-”补齐缺失，例如“mike jenny -”就表示只有2人。

输入样例：

```

5
bob 11 101 102 103 104 105 106 107 108 108 107 107
peter 8 1 2 3 4 3 2 5 1
chris 12 1 2 3 4 5 6 7 8 9 1 2 3
john 10 8 7 6 5 4 3 2 1 7 5
jack 9 6 7 8 9 10 11 12 13 14

```

输出样例：

分析：用一个a数字标记某个标签是否访问过，统计每个人点赞的标签数，用部分排序或者优先队列，最后输出结果。

```

1  #include <iostream>
2  #include <algorithm>
3  #include <vector>
4  #include <cstring>
5  using namespace std;
6  struct stu{
7      string name;
8      int sum, cnt;
9  };
10 bool cmp(stu a, stu b){
11     if(a.cnt != b.cnt) return a.cnt > b.cnt;
12     return 1.0 * a.sum / a.cnt < 1.0 * b.sum / b.cnt;
13 }
14 bool a[10000010];
15 int main() {
16     int n;
17     cin >> n;
18     vector<stu> ans(n);
19     for(int i = 0; i < n; i++) {
20         memset(a, false, sizeof(a));
21         int m, t, cnt = 0;
22         string name;
23         cin >> name >> m;
24         for(int j = 0; j < m; j++) {
25             cin >> t;
26             if(a[t] == false) {
27                 a[t] = true;
28                 cnt++;
29             }
30         }
31         ans[i] = {name, m, cnt};
32     }
33     int nn = min(n, 3);
34     partial_sort(ans.begin(), ans.begin() + nn, ans.end(), cmp);
35     for(int i = 0; i < nn; i++) {
36         if(i != 0) cout << " ";
37         cout << ans[i].name;
38     }
39     for(int i = 0; i < 3 - n; i++)
40         cout << " " << "-";
41     return 0;
42 }

```

L2-022. 重排链表 [链表]

给定一个单链表 $L_1 \rightarrow L_2 \rightarrow \dots \rightarrow L_{n-1} \rightarrow L_n$ ，请编写程序将链表重新排列为 $L_n \rightarrow L_1 \rightarrow L_{n-1} \rightarrow L_2 \rightarrow \dots$ 。例如：给定L为 $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6$ ，则输出应该为 $6 \rightarrow 1 \rightarrow 5 \rightarrow 2 \rightarrow 4 \rightarrow 3$ 。

输入格式：

每个输入包含1个测试用例。每个测试用例第1行给出第1个结点的地址和结点总个数，即正整数N ($\leq 10^5$)。结点的地址是5位非负整数，NULL地址用-1表示。

接下来有N行，每行格式为：

Address Data Next

其中Address是结点地址；Data是该结点保存的数据，为不超过 10^5 的正整数；Next是下一结点的地址。题目保证给出的链表上至少有两个结点。

输出格式：

对每个测试用例，顺序输出重排后的结果链表，其上每个结点占一行，格式与输入相同。

输入样例：

```
00100 6
00000 4 99999
00100 1 12309
68237 6 -1
33218 3 00000
99999 5 68237
12309 2 33218
```

输出样例：

```
68237 6 00100
00100 1 99999
99999 5 12309
12309 2 00000
00000 4 33218
33218 3 -1
```

分析：用结构体 {id, data, next} 存储节点信息，模拟链表。先遍历一遍链表，把有用数节点存起来（甲级乙级中，姥姥为了加大难度，加了好多不在链表中的干扰数据）然后从后往前交替输出，（先把顺序存在ans中,待会统一输出），l, r代表将要输出的节点位置，当 $(l-1) - (r+1) == 1$ 时都遍历一遍了，可退出循环~

```
1 #include <iostream>
2 #include <vector>
3 using namespace std;
```

```

4  struct node{
5      int id, data, next;
6  };
7  int main() {
8      int begin, n;
9      cin >> begin >> n;
10     node a[100010];
11     vector<node> v, ans;
12     for(int i = 0; i < n; i++) {
13         int tbegin, tdata, tnext;
14         cin >> tbegin >> tdata >> tnext;
15         a[tbegin] = {tbegin, tdata, tnext};
16     }
17     while(begin != -1) {
18         v.push_back(a[begin]);
19         begin = a[begin].next;
20     }
21     int l = 0, r = v.size() - 1;
22     while(1) {
23         ans.push_back(v[r]);
24         r--;
25         if((r + 1) - (l - 1) == 1) break;
26         ans.push_back(v[l]);
27         l++;
28         if((r + 1) - (l - 1) == 1) break;
29     }
30     for(int i = 0; i < ans.size(); i++) {
31         if(i != ans.size() - 1)
32             printf("%05d %d %05d\n", ans[i].id, ans[i].data,
ans[i+1].id);
33         else
34             printf("%05d %d -1\n", ans[i].id, ans[i].data);
35     }
36     return 0;
37 }

```

L2-023. 图着色问题 [图]

图着色问题是一个著名的NP完全问题。给定无向图 $G = (V, E)$ ，问可否用K种颜色为V中的每一个顶点分配一种颜色，使得不会有两个相邻顶点具有同一种颜色？

但本题并不是要你解决这个着色问题，而是对给定的一种颜色分配，请你判断这是否是图着色问题的一个解。

输入格式：

输入在第一行给出3个整数V ($0 < V \leq 500$)、E (≥ 0) 和K ($0 < K \leq V$)，分别是无向图的顶点数、边数、以及颜色数。顶点和颜色都从1到V编号。随后E行，每行给出一条边的两个端点的编号。在图的信息给出之后，给出了一个正整数N (≤ 20)，是待检查的颜色分配方案的个数。随后N行，每行顺次给出V个顶点的颜色（第i个数字表示第i个顶点的颜色），数字间以空格分隔。题目保证给定

的无向图是合法的（即不存在自回路和重边）。

输出格式：

对每种颜色分配方案，如果是图着色问题的一个解则输出“Yes”，否则输出“No”，每句占一行。

输入样例：

6 8 3

2 1

1 3

4 6

2 5

2 4

5 4

5 6

3 6

4

1 2 3 3 1 2

4 5 6 6 4 5

1 2 3 4 5 6

2 3 4 2 3 4

输出样例：

Yes

Yes

No

No

分析：给出无向图，和颜色的数量k，求是否给出的颜色数量==k，并且任意两个点之间颜色都不同。

依次判断每一个点是否和周围的点颜色相同，出现一次，就立刻得出结果“No”，如果一次都没出现，结果为“Yes”。

判断给出的颜色个数的时候用set方便（本题数据规模不大，不会对运行时间影响太大）。

```
1 #include <iostream>
2 #include <vector>
3 #include <set>
4 using namespace std;
5 int n, m, k, num;
6 vector<vector<int>> v(510);
```

```

7  vector<int> color(510);
8  bool f(int i) {
9      for(int j = 0; j < v[i].size(); j++)
10         if(color[i] == color[v[i][j]])
11             return false;
12     return true;
13 }
14 int main() {
15     cin >> n >> m >> k;
16     for(int i = 0; i < m; i++) {
17         int t1, t2;
18         cin >> t1 >> t2;
19         v[t1].push_back(t2);
20         v[t2].push_back(t1);
21     }
22     cin >> num;
23     while(num--) {
24         int flag = 0;
25         set<int> s;
26         for(int i = 1; i <= n; i++) {
27             cin >> color[i];
28             s.insert(color[i]);
29         }
30         if(s.size() != k ) {
31             flag = 1;
32             cout << "No\n";
33         }else {
34             for(int i = 1; i <= n; i++) {
35                 if(f(i) == false) {
36                     cout << "No\n";
37                     flag = 1;
38                     break;
39                 }
40             }
41         }
42         if(flag == 0) cout << "Yes\n";
43     }
44     return 0;
45 }

```

L2-024. 部落 [并查集]

在一个社区里，每个人都有自己的小圈子，还可能同时属于很多不同的朋友圈。我们认为朋友的朋友都算在一个部落里，于是要请你统计一下，在一个给定社区中，到底有多少个互不相交的部落？并且检查任意两个人是否属于同一个部落。

输入格式：

输入在第一行给出一个正整数N ($\leq 10^4$)，是已知小圈子的个数。随后N行，每行按下列格式给出一个小圈子里的人：

K P[1] P[2] ... P[K]

其中K是小圈子里的人数，P[i] (i=1, ..., K) 是小圈子里每个人的编号。这里所有人的编号从1开始连续编号，最大编号不会超过 10^4 。

之后一行给出一个非负整数Q ($\leq 10^4$)，是查询次数。随后Q行，每行给出一对被查询的人的编号。

输出格式：

首先在一行中输出这个社区的总人数、以及互不相交的部落的个数。随后对每一次查询，如果他们属于同一个部落，则在一行中输出“Y”，否则输出“N”。

输入样例：

4

3 10 1 2

2 3 4

4 1 5 7 8

3 9 6 4

2

10 5

3 7

输出样例：

10 2

Y

N

```
1  #include <cstdio>
2  #include <set>
3  using namespace std;
4  int father[10010];
5  int findFather(int x) {
6      while (x != father[x])
7          x = father[x];
8      return x;
9  }
10 set<int> total;
11 void unionFather(int a, int b) {
12     total.insert(a);
13     total.insert(b);
14     int fa = findFather(a), fb = findFather(b);
15     if (fa < fb) father[fb] = fa;
16     else father[fa] = fb;
17 }
```

```

18  int main() {
19      for (int i = 0; i <= 10000; i++) father[i] = i;
20      int n, k, t, s, q;
21      scanf("%d", &n);
22      for (int i = 0; i < n; i++) {
23          scanf("%d %d", &k, &t);
24          if (k == 1) unionFather(t, t);
25          for (int j = 1; j < k; j++) {
26              scanf("%d", &s);
27              unionFather(t, s);
28          }
29      }
30      set<int> cnt;
31      for (auto it = total.begin(); it != total.end(); it++)
32          cnt.insert(findFather(*it));
33      printf("%lu %lu\n", total.size(), cnt.size());
34      scanf("%d", &q);
35      for (int i = 0; i < q; i++) {
36          scanf("%d %d", &t, &s);
37          printf("%c\n", findFather(t) == findFather(s) ? 'Y' : 'N');
38      }
39      return 0;
40  }

```

L2-025. 分而治之 [图]

分而治之，各个击破是兵家常用的策略之一。在战争中，我们希望首先攻下敌方的部分城市，使其剩余的城市变成孤立无援，然后再分头各个击破。为此参谋部提供了若干打击方案。本题就请你编写程序，判断每个方案的可行性。

输入格式：

输入在第一行给出两个正整数 N 和 M （均不超过10 000），分别为敌方城市个数（于是默认城市从1到 N 编号）和连接两城市的通路条数。随后 M 行，每行给出一条通路所连接的两个城市的编号，其间以一个空格分隔。在城市信息之后给出参谋部的系列方案，即一个正整数 K （ ≤ 100 ）和随后的 K 行方案，每行按以下格式给出：

N_p $v[1]$ $v[2]$... $v[N_p]$

其中 N_p 是该方案中计划攻下的城市数量，后面的系列 $v[i]$ 是计划攻下的城市编号。

输出格式：

对每一套方案，如果可行就输出“YES”，否则输出“NO”。

输入样例：

10 11

8 7

6 8

4 5

8 4

8 1

1 2

1 4

9 8

9 1

1 10

2 4

5

4 10 3 8 4

6 6 1 7 5 4 9

3 1 8 4

2 2 8

7 9 8 7 6 5 4 2

输出样例：

NO

YES

YES

NO

NO

分析：存图用邻接表存，aa数组表示此城有兄弟城市（即有多少城市与之连接），a组为每次攻城后的次城有多少兄弟城，aa作为原始连接情况，不变，每次用的时候，复制到a上即可。每攻下一座城，次城兄弟变成0，其兄弟城的兄弟城减少一个。最后检查，如果所有城市的兄弟城都小于零，则输出YES，否则输出NO。

```
1  #include <iostream>
2  #include <vector>
3  using namespace std;
4  int n, m, k, aa[10010] = {0}, a[10010];
5  void check() {
6      for(int i = 1; i <= n; i++) {
7          if(a[i] > 0) {
8              cout << "NO\n";
9              return ;
10         }
```

```

11     }
12     cout << "YES\n";
13     return ;
14 }
15 int main(){
16     cin >> n >> m;
17     vector<vector<int>> v(n+1);
18     while(m--) {
19         int c1, c2;
20         cin >> c1 >> c2;
21         aa[c1]++;
22         aa[c2]++;
23         v[c1].push_back(c2);
24         v[c2].push_back(c1);
25     }
26     cin >> k;
27     while(k--) {
28         int cnt, num;
29         cin >> cnt;
30         for(int i = 1; i <= n; i++)
31             a[i] = aa[i];
32         for(int i = 0; i < cnt; i++) {
33             cin >> num;
34             a[num] = 0;
35             for(int j = 0; j < v[num].size(); j++)
36                 a[v[num][j]]--;
37         }
38         check();
39     }
40     return 0;
41 }

```

L2-026. 小字辈 [BFS广度优先搜索]

本题给定一个庞大家族的家谱，要请你给出最小一辈的名单。

输入格式：

输入在第一行给出家族人口总数 N （不超过 100 000 的正整数）—— 简单起见，我们把家族成员从 1 到 N 编号。随后第二行给出 N 个编号，其中第 i 个编号对应第 i 位成员的父/母。家谱中辈分最高的老祖宗对应的父/母编号为 -1。一行中的数字间以空格分隔。

输出格式：

首先输出最小的辈分（老祖宗的辈分为 1，以下逐级递增）。然后在第二行按递增顺序输出辈分最小的成员的编号。编号间以一个空格分隔，行首尾不得有多余空格。

输入样例：

9

2 6 5 5 -1 5 6 4 7

输出样例：

4

1 9

分析：广度优先搜索最长路径问题（深搜可能会超时），老祖宗的父母即root的id用0表示，队列中的结构体成员分别是，人的编号和所处层数。

```
1  #include <iostream>
2  #include <queue>
3  #include <vector>
4  using namespace std;
5  struct peo{
6      int id, level;
7  };
8  int main(){
9      int n, temp, maxlevel = -1;
10     queue<peo> q;
11     vector<peo> ans;
12     cin >> n;
13     vector<vector<int>> v(n+1);
14     for(int i = 1; i <= n; i++) {
15         cin >> temp;
16         if(temp == -1) temp = 0;
17         v[temp].push_back(i);
18     }
19     q.push({0,0});
20     while(!q.empty()) {
21         peo p = q.front();
22         int id = p.id, level = p.level;
23         if(p.level > maxlevel) maxlevel = p.level;
24         ans.push_back(p);
25         q.pop();
26         for(int i = 0; i < v[id].size(); i++)
27             q.push({v[id][i], level + 1});
28     }
29     cout << maxlevel << endl;
30     for(int i = 0; i < ans.size(); i++) {
31         if(ans[i].level == maxlevel) {
32             cout << ans[i].id;
33             if(i != ans.size() - 1) cout << " ";
34         }
35     }
36     return 0;
37 }
```

L2-027. 名人堂与代金券 [排序]

对于在中国大学MOOC学习“数据结构”课程的学生，想要获得一张合格证书，总评成绩必须达到 60 分及以上，并且有另加福利：总评分在 $[G, 100]$ 区间内者，可以得到 50 元 PAT 代金券；在 $[60, G)$ 区间内者，可以得到 20 元PAT代金券。全国考点通用，一年有效。同时任课老师还会把总评成绩前 K 名的学生列入课程“名人堂”。本题就请你编写程序，帮助老师列出名人堂的学生，并统计一共发出了面值多少元的 PAT 代金券。

输入格式：

输入在第一行给出 3 个整数，分别是 N（不超过 10 000 的正整数，为学生总数）、G（在 $(60, 100)$ 区间内的整数，为题面中描述的代金券等级分界线）、K（不超过 100 且不超过 N 的正整数，为进入名人堂的最低名次）。接下来 N 行，每行给出一位学生的账号（长度不超过15位、不带空格的字符串）和总评成绩（区间 $[0, 100]$ 内的整数），其间以空格分隔。题目保证没有重复的账号。

输出格式：

首先在一行中输出发出的 PAT 代金券的总面值。然后按总评成绩非升序输出进入名人堂的学生们的名次、账号和成绩，其间以 1 个空格分隔。需要注意的是：成绩相同的学生享有并列的排名，排名并列时，按账号的字母序升序输出。

输入样例：

```
10 80 5
cy@zju.edu.cn 78
cy@pat-edu.com 87
1001@qq.com 65
uh-oh@163.com 96
test@126.com 39
anyone@qq.com 87
zoe@mit.edu 80
jack@ucla.edu 88
bob@cmu.edu 80
ken@163.com 70
```

输出样例：

```
360
1 uh-oh@163.com 96
2 jack@ucla.edu 88
3 anyone@qq.com 87
3 cy@pat-edu.com 87
5 bob@cmu.edu 80
5 zoe@mit.edu 80
```

分析：给每个大于60分的学生发20块，如果还有人同时大于G，在发30块。把所有学生生排序后，重新排名。第一个学生为第一名,以后的学生如果分数和前一个学生并列，他的名次和前一个同学名次一样，否则他的名次为下标加一

```
1  #include <iostream>
2  #include <algorithm>
3  #include <vector>
4  using namespace std;
5  struct peo{
6      string name;
7      int sco;
8  };
9  bool cmp(peo a, peo b) {
10     if(a.sco != b.sco) return a.sco > b.sco;
11     return a.name < b.name;
12 }
13 int main(){
14     int n, g, k, sum = 0, rank[10001] = {0};
15     cin >> n >> g >> k;
16     vector<peo> v(n);
17     for(int i = 0; i < n; i++) {
18         cin >> v[i].name >> v[i].sco;
19         if(v[i].sco >= 60) sum += 20;
20         if(v[i].sco >= g) sum += 30;
21     }
22     sort(v.begin(), v.end(), cmp);
23     rank[0] = 1;
24     for(int i = 1; i < n; i++) {
25         if(v[i].sco == v[i-1].sco)
26             rank[i] = rank[i-1];
27         else
28             rank[i] = i + 1;
29     }
30     cout << sum << endl;
31     for(int i = 0; rank[i] <= k && i < n; i++)
32         printf("%d %s %d\n", rank[i], v[i].name.c_str(), v[i].sco);
33     return 0;
34 }
```

L3-001. 凑零钱 [01背包，动态规划]

韩梅梅喜欢满宇宙到处逛街。现在她逛到了一家火星店里，发现这家店有个特别的规矩：你可以用任何星球的硬币付钱，但是绝不找零，当然也不能欠债。韩梅梅手边有104枚来自各个星球的硬币，需要你帮她盘算一下，是否可能精确凑出要付的款额。

输入格式：

输入第一行给出两个正整数：N（≤104）是硬币的总个数，M（≤102）是韩梅梅要付的款额。第二行给出N枚硬币的正整数面值。数字间以空格分隔。

输出格式：

在一行中输出硬币的面值 $V_1 \leq V_2 \leq \dots \leq V_k$ ，满足条件 $V_1 + V_2 + \dots + V_k = M$ 。数字间以1个空格分隔，行首尾不得有多余空格。若解不唯一，则输出最小序列。若无解，则输出“No Solution”。

注：我们说序列 $\{A[1], A[2], \dots\}$ 比 $\{B[1], B[2], \dots\}$ “小”，是指存在 $k \geq 1$ 使得 $A[i]=B[i]$ 对所有 $i < k$ 成立，并且 $A[k] < B[k]$ 。

输入样例1：

8 9

5 9 8 7 2 3 4 1

输出样例1：

1 3 5

输入样例2：

4 8

7 2 4 3

输出样例2：

No Solution

分析：01背包问题，因为要输出从小到大的排列，可以先把硬币面额从大到小排列，然后用bool类型的choice[i][j]数组dp[i][j]是否选取，如果选取了就令choice为true；然后进行01背包问题求解，如果最后求解的结果不是恰好等于所需要的价值的，就输出No Soutlion，否则从choice[i][j]判断选取的情况，i从n到1表示从后往前看第i个物品的选取情况，j从m到0表示从容量m到0是否选取($j = j - w[i]$)，把选取情况压入arr数组中，最后输出arr数组

```
1  #include <cstdio>
2  #include <vector>
3  #include <algorithm>
4  using namespace std;
5  int dp[10010];
6  int w[10010];
7  bool choice[10010][10010];
8  int cmp1(int a, int b){return a > b;}
9  int main() {
10     int n, m;
11     scanf("%d%d", &n, &m);
12     for(int i = 1; i <= n; i++)
13         scanf("%d", &w[i]);
14     sort(w + 1, w + n + 1, cmp1);
15     for(int i = 1; i <= n; i++) {
16         for(int j = m; j >= w[i]; j--) {
17             if(dp[j] <= dp[j-w[i]] + w[i]) {
18                 choice[i][j] = true;
19                 dp[j] = dp[j-w[i]] + w[i];
20             }
```

```

21     }
22 }
23 if(dp[m] != m) printf("No Solution");
24 else {
25     vector<int> arr;
26     int v = m, index = n;
27     while(v > 0) {
28         if(choice[index][v] == true) {
29             arr.push_back(w[index]);
30             v -= w[index];
31         }
32         index--;
33     }
34     for(int i = 0; i < arr.size(); i++) {
35         if(i != 0) printf(" ");
36         printf("%d", arr[i]);
37     }
38 }
39 return 0;
40 }

```

L3-002. 堆栈 [树状数组]

大家都知道“堆栈”是一种“先进后出”的线性结构，基本操作有“入栈”（将新元素插入栈顶）和“出栈”（将栈顶元素的值返回并从堆栈中将其删除）。现请你实现一种特殊的堆栈，它多了一种操作叫“查中值”，即返回堆栈中所有元素的中值。对于N个元素，若N是偶数，则中值定义为第N/2个最小元；若N是奇数，则中值定义为第(N+1)/2个最小元。

输入格式：

输入第一行给出正整数N（≤ 105）。随后N行，每行给出一个操作指令，为下列3种指令之一：

Push key

Pop

PeekMedian

其中Push表示入栈，key是不超过105的正整数；Pop表示出栈；PeekMedian表示查中值。

输出格式：

对每个入栈指令，将key入栈，并不输出任何信息。对每个出栈或查中值的指令，在一行中打印相应的返回结果。若指令非法，就打印“Invalid”。

输入样例：

17

Pop

PeekMedian

Push 3

PeekMedian

Push 2

PeekMedian

Push 1

PeekMedian

Pop

Pop

Push 5

Push 4

PeekMedian

Pop

Pop

Pop

Pop

输出样例：

Invalid

Invalid

3

2

2

1

2

4

4

5

3

Invalid

分析：如果排序查找的话会超时，用树状数组，即求第 $k = (s.size() + 1) / 2$ 大的数。查询小于等于x的数的个数是否等于k的时候用二分法更快～

```
1 #include <cstdio>
2 #include <stack>
3 #define lowbit(i) ((i) & (-i))
```

```

4  const int maxn = 100010;
5  using namespace std;
6  int c[maxn];
7  stack<int> s;
8  void update(int x, int v) {
9      for(int i = x; i < maxn; i += lowbit(i))
10         c[i] += v;
11 }
12 int getsum(int x) {
13     int sum = 0;
14     for(int i = x; i >= 1; i -= lowbit(i))
15         sum += c[i];
16     return sum;
17 }
18 void PeekMedian() {
19     int left = 1, right = maxn, mid, k = (s.size() + 1) / 2;
20     while(left < right) {
21         mid = (left + right) / 2;
22         if(getsum(mid) >= k)
23             right = mid;
24         else
25             left = mid + 1;
26     }
27     printf("%d\n", left);
28 }
29 int main() {
30     int n, temp;
31     scanf("%d", &n);
32     char str[15];
33     for(int i = 0; i < n; i++) {
34         scanf("%s", str);
35         if(str[1] == 'u') {
36             scanf("%d", &temp);
37             s.push(temp);
38             update(temp, 1);
39         } else if(str[1] == 'o') {
40             if(!s.empty()) {
41                 update(s.top(), -1);
42                 printf("%d\n", s.top());
43                 s.pop();
44             } else {
45                 printf("Invalid\n");
46             }
47         } else {
48             if(!s.empty())
49                 PeekMedian();
50             else
51                 printf("Invalid\n");
52         }
53     }
54 }

```

```

53     }
54     return 0;
55 }

```

L3-003. 社交集群 [并查集]

在社交网络平台注册时，用户通常会输入自己的兴趣爱好，以便找到和自己兴趣相投的朋友。有部分兴趣相同的人们就形成了“社交集群”。现请你编写程序，找出所有的集群。

输入格式：

输入的第一行给出正整数N（ ≤ 1000 ），即社交网络中的用户总数（则用户从1到N编号）。随后N行，每行按下列格式列出每个人的兴趣爱好：

Ki: hi[1] hi[2] ... hi[Ki]

其中Ki（ > 0 ）是第i个人的兴趣的数量，hi[j]是第i个人的第j项兴趣的编号，编号范围为[1, 1000]内的整数。

输出格式：

首先在第一行输出整个网络中集群的数量，然后在第二行按非递增的顺序输出每个集群中用户的数量。数字间以1个空格分隔，行首尾不得有多余空格。

输入样例：

```

8
3: 2 7 10
1: 4
2: 5 3
1: 4
1: 3
1: 4
4: 6 8 1 5
1: 4

```

输出样例：

```

3
4 3 1

```

分析：并查集。先写好init、findFather、Union。

0. 每个社交圈的结点号是人的编号，而不是课程。课程是用来判断是否处在一个社交圈的。
1. course[t]表示任意一个喜欢t活动的人的编号。如果当前的课程t，之前并没有人喜欢过，那么就course[t] = i，i为它自己的编号，表示i为喜欢course[t]的一个人的编号
2. course[t]是喜欢t活动的人的编号，那么findFather(course[t])就是喜欢这个活动的人所处的社交圈子的根结点，合并根结点和当前人的编号的结点i。即Union(i, findFather(course[t])), 把它们

处在同一个社交圈子里面

3. isRoot[i]表示编号i的人是不是它自己社交圈子的根结点，如果等于0表示不是根结点，如果不等于0，每次标记isRoot[findFather(i)]++，那么isRoot保存的就是如果当前是根结点，那么这个社交圈里面的总人数
4. isRoot中不为0的编号的个数cnt就是社交圈圈子的个数
5. 把isRoot从大到小排列，输出前cnt个，就是社交圈人数的从大到小的输出顺序

```
1  #include <cstdio>
2  #include <vector>
3  #include <algorithm>
4  using namespace std;
5  vector<int> father, isRoot;
6  int cmp1(int a, int b){
7      return a > b;
8  }
9  int findFather(int x) {
10     int a = x;
11     while(x != father[x])
12         x = father[x];
13     while(a != father[a]) {
14         int z = a;
15         a = father[a];
16         father[z] = x;
17     }
18     return x;
19 }
20 void Union(int a, int b) {
21     int faA = findFather(a);
22     int faB = findFather(b);
23     if(faA != faB)
24         father[faA] = faB;
25 }
26 int main() {
27     int n, k, t, cnt = 0;
28     int course[1001] = {0};
29     scanf("%d", &n);
30     father.resize(n + 1);
31     isRoot.resize(n + 1);
32     for(int i = 1; i <= n; i++)
33         father[i] = i;
34     for(int i = 1; i <= n; i++) {
35         scanf("%d:", &k);
36         for(int j = 0; j < k; j++) {
37             scanf("%d", &t);
38             if(course[t] == 0)
39                 course[t] = i;
40             Union(i, findFather(course[t]));
41         }
42     }
```

```

43     for(int i = 1; i <= n; i++)
44         isRoot[findFather(i)]++;
45     for(int i = 1; i <= n; i++) {
46         if(isRoot[i] != 0)
47             cnt++;
48     }
49     printf("%d\n", cnt);
50     sort(isRoot.begin(), isRoot.end(), cmp1);
51     for(int i = 0; i < cnt; i++) {
52         printf("%d", isRoot[i]);
53         if(i != cnt - 1) printf(" ");
54     }
55     return 0;
56 }

```

L3-004. 肿瘤诊断 [广度优先搜索BFS]

在诊断肿瘤疾病时，计算肿瘤体积是很重要的一环。给定病灶扫描切片中标注出的疑似肿瘤区域，请你计算肿瘤的体积。

输入格式：

输入第一行给出4个正整数：M、N、L、T，其中M和N是每张切片的尺寸（即每张切片是一个M×N的像素矩阵。最大分辨率是1286×128）；L（≤60）是切片的张数；T是一个整数阈值（若疑似肿瘤的连通体体积小于T，则该小块忽略不计）。

最后给出L张切片。每张用一个由0和1组成的M×N的矩阵表示，其中1表示疑似肿瘤的像素，0表示正常像素。由于切片厚度可以认为是一个常数，于是我们只要数连通体中1的个数就可以得到体积了。麻烦的是，可能存在多个肿瘤，这时我们只统计那些体积不小于T的。两个像素被认为是“连通的”，如果它们有一个共同的切面，如下图所示，所有6个红色的像素都与蓝色的像素连通。

输出格式：

在一行中输出肿瘤的总体积。

输入样例：

```

3 4 5 2
1 1 1 1
1 1 1 1
1 1 1 1
0 0 1 1
0 0 1 1
0 0 1 1
1 0 1 1
0 1 0 0

```

0 0 0 0

1 0 1 1

0 0 0 0

0 0 0 0

0 0 0 1

0 0 0 1

1 0 0 0

输出样例：

26

分析：三维的广度优先搜索~XYZ三个数组判断方向，对每一个点广度优先累计肿瘤块的大小，如果大于等于t就把结果累加。用visit数组标记当前的点有没有被访问过，被访问过的结点是不会再访问的。。judge判断是否超过了边界，或者是否当前结点为0不是肿瘤~

```
1  #include <cstdio>
2  #include <queue>
3  using namespace std;
4  struct node {
5      int x, y, z;
6  };
7  int m, n, l, t;
8  int X[6] = {1, 0, 0, -1, 0, 0};
9  int Y[6] = {0, 1, 0, 0, -1, 0};
10 int Z[6] = {0, 0, 1, 0, 0, -1};
11 int arr[1300][130][80];
12 bool visit[1300][130][80];
13 bool judge(int x, int y, int z) {
14     if(x < 0 || x >= m || y < 0 || y >= n || z < 0 || z >= l) return
false;
15     if(arr[x][y][z] == 0 || visit[x][y][z] == true) return false;
16     return true;
17 }
18 int bfs(int x, int y, int z) {
19     int cnt = 0;
20     node temp;
21     temp.x = x, temp.y = y, temp.z = z;
22     queue<node> q;
23     q.push(temp);
24     visit[x][y][z] = true;
25     while(!q.empty()) {
26         node top = q.front();
27         q.pop();
28         cnt++;
29         for(int i = 0; i < 6; i++) {
```

```

30         int tx = top.x + X[i];
31         int ty = top.y + Y[i];
32         int tz = top.z + Z[i];
33         if(judge(tx, ty, tz)) {
34             visit[tx][ty][tz] = true;
35             temp.x = tx, temp.y = ty, temp.z = tz;
36             q.push(temp);
37         }
38     }
39 }
40 if(cnt >= t)
41     return cnt;
42 else
43     return 0;
44
45 }
46 int main() {
47     scanf("%d %d %d %d", &m, &n, &l, &t);
48     for(int i = 0; i < l; i++)
49         for(int j = 0; j < m; j++)
50             for(int k = 0; k < n; k++)
51                 scanf("%d", &arr[j][k][i]);
52     int ans = 0;
53     for(int i = 0; i < l; i++) {
54         for(int j = 0; j < m; j++) {
55             for(int k = 0; k < n; k++) {
56                 if(arr[j][k][i] == 1 && visit[j][k][i] == false)
57                     ans += bfs(j, k, i);
58             }
59         }
60     }
61     printf("%d", ans);
62     return 0;
63 }

```

L3-005. 垃圾箱分布 [Dijkstra算法，最短路径]

大家倒垃圾的时候，都希望垃圾箱距离自己比较近，但是谁都不愿意守着垃圾箱住。所以垃圾箱的位置必须选在到所有居民点的最短距离最长的地方，同时还要保证每个居民点都在距离它一个不太远的范围内。

现给定一个居民区的地图，以及若干垃圾箱的候选地点，请你推荐最合适的地点。如果解不唯一，则输出到所有居民点的平均距离最短的那个解。如果这样的解还是不唯一，则输出编号最小的地点。

输入格式：

输入第一行给出4个正整数：N（≤ 103）是居民点的个数；M（≤ 10）是垃圾箱候选地点的个数；K（≤ 104）是居民点和垃圾箱候选地点之间的道路的条数；DS是居民点与垃圾箱之间不能超过的最大距离。所有的居民点从1到N编号，所有的垃圾箱候选地点从G1到GM编号。

随后K行，每行按下列格式描述一条道路：

P1 P2 Dist

其中P1和P2是道路两端点的编号，端点可以是居民点，也可以是垃圾箱候选点。Dist是道路的长度，是一个正整数。

输出格式：

首先在第一行输出最佳候选地点的编号。然后在第二行输出该地点到所有居民点的最小距离和平均距离。数字间以空格分隔，保留小数点后1位。如果解不存在，则输出“No Solution”。

输入样例1：

4 3 11 5

1 2 2

1 4 2

1 G1 4

1 G2 3

2 3 2

2 G2 1

3 4 2

3 G3 2

4 G1 3

G2 G1 1

G3 G2 2

输出样例1：

G1

2.0 3.3

输入样例2：

2 1 2 10

1 G1 9

2 G1 20

输出样例2：

No Solution

题目大意：从m个垃圾站里面选取1个站点，让他离居民区的最近的人最远，并且没有超出服务范围ds之内。如果有很多个最远的垃圾站，输出距离所有居民区距离平均值最小的那个。如果平均值还是一样，就输出按照顺序排列垃圾站编号最小的那个、

分析：

因为垃圾站之间也是彼此有路连接的，所以最短路径计算的时候也要把垃圾站算上。所以我们就是堆 $n+m$ 个点进行Dijkstra计算最短路径。要求计算出1~m号垃圾站距离其他站点的最短路径。这时候可以遍历dis数组，如果dis存在一个距离大于服务范围ds的距离，那么我们就舍弃这个垃圾站。取最短的路径，这就是距离它最近的垃圾站mindis。如果mindis > ansdis，就是说找到了一个距离居民最小距离的垃圾站是更远的，那就选这个垃圾站，更新ansid为它的id。最后输出

对于垃圾站的字符串编号的处理：如果最近居民区最大的值没有变化但是找到了一个更小的平均距离，那就选这个。我们可以根据输入的是G还是数字，如果是数字就令编号为他自己，如果是G开头的，编号设为n+G后面的数字。

```
1  #include <cstdio>
2  #include <algorithm>
3  #include <iostream>
4  #include <string>
5  using namespace std;
6  const int inf = 999999999;
7  int n, m, k, ds, station;
8  int e[1020][1020], dis[1020];
9  bool visit[1020];
10 int main() {
11     fill(e[0], e[0] + 1020 * 1020, inf);
12     fill(dis, dis + 1020, inf);
13     scanf("%d%d%d%d", &n, &m, &k, &ds);
14     for(int i = 0; i < k; i++) {
15         int tempdis;
16         string s, t;
17         cin >> s >> t >> tempdis;
18         int a, b;
19         if(s[0] == 'G') {
20             s = s.substr(1);
21             a = n + stoi(s);
22         } else {
23             a = stoi(s);
24         }
25         if(t[0] == 'G') {
26             t = t.substr(1);
27             b = n + stoi(t);
28         } else {
29             b = stoi(t);
30         }
31         e[a][b] = tempdis;
32         e[b][a] = tempdis;
33     }
34     int ansid = -1;
35     double ansdis = -1, ansaver = inf;
36     for(int index = n + 1; index <= n + m; index++) {
37         double mindis = inf, aver = 0;
38         fill(dis, dis + 1020, inf);
39         fill(visit, visit + 1020, false);
```

```

40     dis[index] = 0;
41     for(int i = 0; i < n + m; i++) {
42         int u = -1, minn = inf;
43         for(int j = 1; j <= n + m; j++) {
44             if(visit[j] == false && dis[j] < minn) {
45                 u = j;
46                 minn = dis[j];
47             }
48         }
49         if(u == -1) break;
50         visit[u] = true;
51         for(int v = 1; v <= n + m; v++) {
52             if(visit[v] == false && dis[v] > dis[u] + e[u][v])
53                 dis[v] = dis[u] + e[u][v];
54         }
55     }
56     for(int i = 1; i <= n; i++) {
57         if(dis[i] > ds) {
58             mindis = -1;
59             break;
60         }
61         if(dis[i] < mindis) mindis = dis[i];
62         aver += 1.0 * dis[i];
63     }
64     if(mindis == -1) continue;
65     aver = aver / n;
66     if(mindis > ansdis) {
67         ansid = index;
68         ansdis = mindis;
69         ansaver = aver;
70     } else if(mindis == ansdis && aver < ansaver) {
71         ansid = index;
72         ansaver = aver;
73     }
74 }
75 if(ansid == -1)
76     printf("No Solution");
77 else {
78     printf("G%d\n", ansid - n);
79     printf("%.1f %.1f", ansdis, ansaver);
80 }
81 return 0;
82 }

```

L3-007. 天梯地图 [Dijkstra算法 + DFS, 最短路径]

本题要求你实现一个天梯赛专属在线地图，队员输入自己学校所在地和赛场地点后，该地图应该推荐两条路线：一条是最快到达路线；一条是最短距离的路线。题目保证对任意的查询请求，地图上至少存在一条可达路线。

输入格式：

输入在第一行给出两个正整数N ($2 \leq N \leq 500$) 和M，分别为地图中所有标记地点的个数和连接地点的道路条数。随后M行，每行按如下格式给出一条道路的信息：

V1 V2 one-way length time

其中V1和V2是道路的两个端点的编号（从0到N-1）；如果该道路是从V1到V2的单行线，则one-way为1，否则为0；length是道路的长度；time是通过该路所需要的时间。最后给出一对起点和终点的编号。

输出格式：

首先按下列格式输出最快到达的时间T和用节点编号表示的路线：

Time = T: 起点 => 节点1 => ... => 终点

然后在下一行按下列格式输出最短距离D和用节点编号表示的路线：

Distance = D: 起点 => 节点1 => ... => 终点

如果最快到达路线不唯一，则输出几条最快路线中最短的那条，题目保证这条路线是唯一的。而如果最短距离的路线不唯一，则输出途径节点数最少的那条，题目保证这条路线是唯一的。

如果这两条路线是完全一样的，则按下列格式输出：

Time = T; Distance = D: 起点 => 节点1 => ... => 终点

输入样例1：

10 15

0 1 0 1 1

8 0 0 1 1

4 8 1 1 1

5 4 0 2 3

5 9 1 1 4

0 6 0 1 1

7 3 1 1 2

8 3 1 1 2

2 5 0 2 2

2 1 1 1 1

1 5 0 1 3

1 4 0 1 1

9 7 1 1 3

3 1 0 2 5

6 3 1 2 1

5 3

输出样例1:

Time = 6: 5 => 4 => 8 => 3

Distance = 3: 5 => 1 => 3

输入样例2:

7 9

0 4 1 1 1

1 6 1 3 1

2 6 1 1 1

2 5 1 2 2

3 0 0 1 1

3 1 1 3 1

3 2 1 2 1

4 5 0 2 2

6 5 1 2 1

3 5

输出样例2:

Time = 3; Distance = 4: 3 => 2 => 5

分析：用两个Dijkstra + DFS。一个求最快路径（如果相同求路径的那条），一个求最短路径（如果相同求结点数最小的那条）~~求最快路径可以直接在Dijkstra里面求前驱结点Timepre数组~~~求最短路径因为要求结点数最小的那条，所以要用dispre的二维数组存储所有结点的最短路径，然后用DFS求出满足条件的结点数最小的那条~~

注意：

1.一开始最后一个测试用例“答案错误”，后来发现是自己在求最短路径（第二个答案distance）的时候忘记了temppath每一次深搜结束后的pop_back();

2.如果直接使用DFS的话，会导致最后一个测试用例“运行超时”~

```
1 #include <cstdio>
2 #include <algorithm>
3 #include <vector>
4 using namespace std;
5 const int inf = 999999999;
6 int dis[510], Time[510], e[510][510], w[510][510], Timepre[510],
  weight[510];
7 bool visit[510];
```

```

8  vector<int> Timepath, dispath, temppath, dispre[510];
9  int st, fin, minnode = inf;
10 void dfsTimepath(int v) {
11     Timepath.push_back(v);
12     if(v == st) {
13         return ;
14     }
15     dfsTimepath(Timepre[v]);
16 }
17
18 void dfsdispath(int v) {
19     temppath.push_back(v);
20     if(v == st) {
21         if(temppath.size() < minnode) {
22             minnode = temppath.size();
23             dispath = temppath;
24         }
25         temppath.pop_back();
26         return ;
27     }
28     for(int i = 0; i < dispre[v].size(); i++) {
29         dfsdispath(dispre[v][i]);
30     }
31     temppath.pop_back();
32 }
33 int main() {
34     fill(dis, dis + 510, inf);
35     fill(Time, Time + 510, inf);
36     fill(weight, weight + 510, inf);
37     fill(e[0], e[0] + 510 * 510, inf);
38     fill(w[0], w[0] + 510 * 510, inf);
39     int n, m;
40     scanf("%d %d", &n, &m);
41     int a, b, flag, len, t;
42     for(int i = 0; i < m; i++) {
43         scanf("%d %d %d %d %d", &a, &b, &flag, &len, &t);
44         e[a][b] = len;
45         w[a][b] = t;
46         if(flag != 1) {
47             e[b][a] = len;
48             w[b][a] = t;
49         }
50     }
51     scanf("%d %d", &st, &fin);
52
53     Time[st] = 0;
54     for(int i = 0; i < n; i++) {
55         Timepre[i] = i;
56     }

```

```

57     for(int i = 0; i < n; i++) {
58         int u = -1, minn = inf;
59         for(int j = 0; j < n; j++) {
60             if(visit[j] == false && Time[j] < minn) {
61                 u = j;
62                 minn = Time[j];
63             }
64         }
65         if(u == -1) break;
66         visit[u] = true;
67         for(int v = 0; v < n; v++) {
68             if(visit[v] == false && w[u][v] != inf) {
69                 if(w[u][v] + Time[u] < Time[v]) {
70                     Time[v] = w[u][v] + Time[u];
71                     Timepre[v] = u;
72                     weight[v] = weight[u] + e[u][v];
73                 } else if(w[u][v] + Time[u] == Time[v] && weight[v] >
weight[u] + e[u][v]) {
74                     weight[v] = weight[u] + e[u][v];
75                     Timepre[v] = u;
76                 }
77             }
78         }
79     }
80     dfsTimepath(fin);
81
82     fill(visit, visit + 510, false);
83     dis[st] = 0;
84     for(int i = 0; i < n; i++) {
85         int u = -1, minn = inf;
86         for(int j = 0; j < n; j++) {
87             if(visit[j] == false && minn > dis[j]) {
88                 u = j;
89                 minn = dis[j];
90             }
91         }
92         if(u == -1) break;
93         visit[u] = true;
94         for(int v = 0; v < n; v++) {
95             if(visit[v] == false && e[u][v] != inf) {
96                 if(e[u][v] + dis[u] < dis[v]) {
97                     dis[v] = e[u][v] + dis[u];
98                     dispre[v].clear();
99                     dispre[v].push_back(u);
100                 } else if(e[u][v] + dis[u] == dis[v]) {
101                     dispre[v].push_back(u);
102                 }
103             }
104         }

```

```

105     }
106     dfsdispath(fin);
107     printf("Time = %d", Time[fin]);
108     if(dispath == Timepath) {
109         printf("; Distance = %d: ", dis[fin]);
110     } else {
111         printf(": ");
112         for(int i = Timepath.size() - 1; i >= 0; i--) {
113             printf("%d", Timepath[i]);
114             if(i != 0) printf(" => ");
115         }
116         printf("\n");
117         printf("Distance = %d: ", dis[fin]);
118     }
119     for(int i = dispath.size() - 1; i >= 0; i--) {
120         printf("%d", dispath[i]);
121         if(i != 0) printf(" => ");
122     }
123     return 0;
124 }

```

L3-008. 喊山 [广度优先搜索BFS]

喊山，是人双手围在嘴边成喇叭状，对着远方高山发出“喂—喂喂—喂喂喂……”的呼唤。呼唤声通过空气的传递，回荡于深谷之间，传送到人们耳中，发出约定俗成的“讯号”，达到声讯传递交流的目的。原来它是彝族先民用来求援呼救的“讯号”，慢慢地人们在生活实践中发现了它的实用价值，便把它作为一种交流工具世代传袭使用。

一个山头呼喊的声音可以被临近的山头同时听到。题目假设每个山头最多有两个能听到它的临近山头。给定任意一个发出原始信号的山头，本题请你找出这个信号最远能传达到的地方。

输入格式：

输入第一行给出3个正整数n、m和k，其中n (≤ 10000) 是总的山头数（于是假设每个山头从1到n编号）。接下来的m行，每行给出2个不超过n的正整数，数字间用空格分开，分别代表可以听到彼此的两个山头的编号。这里保证每一对山头只被输入一次，不会有重复的关系输入。最后一行给出k (≤ 10) 个不超过n的正整数，数字间用空格分开，代表需要查询的山头的编号。

输出格式：

依次对于输入中的每个被查询的山头，在一行中输出其发出的呼喊能够连锁传达到的最远的那个山头。注意：被输出的首先必须是被查询的个山头能连锁传到的。若这样的山头不只一个，则输出编号最小的那个。若此山头的呼喊无法传到任何其他山头，则输出0。

输入样例：

7 5 4

1 2

2 3

3 1

4 5

5 6

1 4 5 7

输出样例：

2

6

4

0

分析：用二维数组，结点push_back的方式存储，因为每次都从1遍历到n会超时，而只要遍历到v[top].size()就不会超时。

用广度优先搜索，level记录当前结点的层数，level为自己的上一层结点的level+1，遇到更大层数的时候就令ans又为最大值，让ans取当层的最小值（在不等于被搜索结点本身的时候）。book记录当前结点有没有被访问过，如果没有被访问过就压入队列中，因为可能重复然后导致无限循环，无法输出
~

```
1  #include <cstdio>
2  #include <queue>
3  #include <vector>
4  #include <algorithm>
5  using namespace std;
6  vector<vector<int>>> v(10010);
7  int book[10010];
8  int main() {
9      int n, m, k, a, b;
10     scanf("%d%d%d", &n, &m, &k);
11     for(int i = 0; i < m; i++) {
12         scanf("%d%d", &a, &b);
13         v[a].push_back(b);
14         v[b].push_back(a);
15     }
16     for(int i = 0; i < k; i++) {
17         scanf("%d", &a);
18         queue<int> q;
19         fill(book, book + 10010, 0);
20         q.push(a);
21         int level[10010];
22         int ans = 10010, maxlevel = 0;
23         book[a] = 1;
24         while(!q.empty()) {
25             int top = q.front();
26             q.pop();
27             if(level[top] > maxlevel) {
28                 maxlevel = level[top];
```

```

29         ans = 10010;
30     }
31     if(top != a)
32         ans = min(ans, top);
33     for(int j = 0; j < v[top].size(); j++) {
34         if(book[v[top][j]] == 0) {
35             q.push(v[top][j]);
36             book[v[top][j]] = 1;
37             level[v[top][j]] = level[top] + 1;
38         }
39     }
40 }
41 if(ans != 10010) printf("%d\n", ans);
42 else printf("0\n");
43 }
44 return 0;
45 }

```

L3-010. 是否完全二叉搜索树 [完全二叉树]

将一系列给定数字顺序插入一个初始为空的二叉搜索树（定义为左子树键值大，右子树键值小），你需要判断最后的树是否一棵完全二叉树，并且给出其层序遍历的结果。

输入格式：

输入第一行给出一个不超过20的正整数N；第二行给出N个互不相同的正整数，其间以空格分隔。

输出格式：

将输入的N个正整数顺序插入一个初始为空的二叉搜索树。在第一行中输出结果树的层序遍历结果，数字间以1个空格分隔，行的首尾不得有多余空格。第二行输出“YES”，如果该树是完全二叉树；否则输出“NO”。

输入样例1：

```

9
38 45 42 24 58 30 67 12 51

```

输出样例1：

```

38 45 24 58 42 30 12 67 51

```

YES

输入样例2：

```

8
38 24 12 45 58 67 42 51

```

输出样例2：

```

38 45 24 58 42 12 67 51

```

NO

分析：根据二叉树的性质，i结点的左孩子为 $root \ll 1$ ，右孩子为 $root \ll 1 | 1$ 。建造并用数组存储这个二叉搜索树，然后判断是否是完全二叉树。

```
1  #include <stdio>
2  using namespace std;
3  int tree[1<<20];
4  int num;
5  void BST(int a) {
6      if(tree[a] == 0)
7          tree[a] = num;
8      else if(tree[a] < num)
9          BST(a<<1);
10     else
11         BST(a<<1|1);
12 }
13 int main() {
14     int n;
15     scanf("%d", &n);
16     for(int i = 0; i < n; i++) {
17         scanf("%d", &num);
18         BST(1);
19     }
20     bool flag = true;
21     for(int cnt = 1, k = 1; cnt <= n; k++) {
22         if(tree[k] == 0)
23             flag = false;
24         else {
25             printf("%d", tree[k]);
26             if(cnt++ != n) printf(" ");
27         }
28     }
29     if(flag) printf("\nYES");
30     else printf("\nNO");
31     return 0;
32 }
```

L3-013. 非常弹的球 [物理题]

刚上高一的森森为了学好物理，买了一个“非常弹”的球。虽然说是非常弹的球，其实也就是一般的弹力球而已。森森玩了一会儿弹力球后突然想到，假如他在地上用力弹球，球最远能弹到多远去呢？他不太会，你能帮他解决吗？当然为了刚学习物理的森森，我们对环境做一些简化：

假设森森是一个质点，以森森为原点设立坐标轴，则森森位于(0, 0)点。

小球质量为 $w/100$ 千克 (kg)，重力加速度为9.8米/秒平方 (m/s²)。

森森在地上用力弹球的过程可简化为球从(0, 0)点以某个森森选择的角度 ang ($0 < ang < \pi/2$) 向第一象限抛出，抛出时假设动能为1000 焦耳 (J)。

小球在空中仅受重力作用，球纵坐标为0时可视为落地，落地时损失p%动能并反弹。

地面可视为刚体，忽略小球形状、空气阻力及摩擦阻力等。

森森为你准备的公式：

动能公式： $E = m * v^2 / 2$

牛顿力学公式： $F = m * a$

重力： $G = m * g$

其中：

E - 动能，单位为“焦耳”

m - 质量，单位为“千克”

v - 速度，单位为“米/秒”

a - 加速度，单位为“米/秒平方”

g - 重力加速度

输入格式：

输入在一行中给出两个整数： $1 \leq w \leq 1000$ 和 $1 \leq p \leq 100$ ，分别表示放大100倍的小球质量、以及损失动力的百分比p。

输出格式：

在一行输出最远的投掷距离，保留3位小数。

输入样例：

100 90

输出样例：

226.757

分析：

因为 $E = 1/2mv^2$ ，所以 $v^2 = 2E/m$

当v分解为垂直方向的 $v\sin\theta$ 和水平方向的 $v\cos\theta$ ，所以水平方向的v为 $v\sin\theta$ ，t为 $v\cos\theta/g$ ，抛到最高点时通过的路程为 $s = vt = v\sin\theta v\cos\theta/g$ ，落下来同样相同的距离， $s = 2v\sin\theta v\cos\theta/g$

因为 $2\sin\theta v\cos\theta = \sin 2\theta$ ， $\sin 2\theta$ 的最大值为1，即 $2\theta = 90^\circ$ ， $\theta = 45^\circ$ 的时候s取得最大值～

这样 $s = v^2/g$

又因为 $v^2 = 2E/m$ ，在代码中 v^2 为变量v2，所以 $v2 = 2 * 1000 * 100 / w$ ，即 $E = 1000$ ， $m = w/100$

每次将 $v2/g$ 即 $v2 / 9.8$ 的结果累加到s中，可以得到s为最后求得的总距离，每一次while循环，都要将v2损失百分比p，直到v2足够小（这里取0.000001可以通过所有测试用例，如果是0.00001会有一个测试点答案错误）的时候退出循环～


```

1  #include <stdio>
2  int main() {
3      double w, p, v2, s = 0;
4      scanf("%lf%lf", &w, &p);
5      v2 = 2 * 1000 * 100 / w;
6      while (v2 > 0.000001) {
7          s += v2 / 9.8;
8          v2 *= (100 - p) * 0.01;
9      }
10     printf("%.3f", s);
11     return 0;
12 }

```

L3-014. 周游世界 [深度优先搜索DFS]

周游世界是件浪漫事，但规划旅行路线就不一定了..... 全世界有成千上万条航线、铁路线、大巴线，令人眼花缭乱。所以旅行社会选择部分运输公司组成联盟，每家公司提供一条线路，然后帮助客户规划由联盟内企业支持的旅行路线。本题就要求你帮旅行社实现一个自动规划路线的程序，使得对任何给定的起点和终点，可以找出最顺畅的路线。所谓“最顺畅”，首先是指中途经停站最少；如果经停站一样多，则取需要换乘线路次数最少的路线。

输入格式：

输入在第一行给出一个正整数N (≤ 100)，即联盟公司的数量。接下来有N行，第i行 ($i=1, \dots, N$) 描述了第i家公司所提供的线路。格式为：

M S[1] S[2] ... S[M]

其中M (≤ 100) 是经停站的数量，S[i] ($i=1, \dots, M$) 是经停站的编号（由4位0-9的数字组成）。这里假设每条线路都是简单的一条可以双向运行的链路，并且输入保证是按照正确的经停顺序给出的——也就是说，任意一对相邻的S[i]和S[i+1] ($i=1, \dots, M-1$) 之间都不存在其他经停站点。我们称相邻站点之间的线路为一个运营区间，每个运营区间只承包给一家公司。环线是有可能存在的，但不会不经停任何中间站点就从出发地回到出发地。当然，不同公司的线路是可能在某些站点有交叉的，这些站点就是客户的换乘点，我们假设任意换乘点涉及的不同公司的线路都不超过5条。

在描述了联盟线路之后，题目将给出一个正整数K (≤ 10)，随后K行，每行给出一位客户的需求，即始发地的编号和目的地的编号，中间以空格分隔。

输出格式：

处理每一位客户的需求。如果没有现成的线路可以使其到达目的地，就在一行中输出“Sorry, no line is available.”；如果目的地可达，则首先在一行中输出最顺畅路线的经停站数量（始发地和目的地不包括在内），然后按下列格式给出旅行路线：

Go by the line of company #X1 from S1 to S2.

Go by the line of company #X2 from S2 to S3.

.....

其中Xi是线路承包公司的编号，Si是经停站的编号。但必须只输出始发地、换乘点和目的地，不能输出中间的经停站。题目保证满足要求的路线是唯一的。

输入样例：

4

7 1001 3212 1003 1204 1005 1306 7797

9 9988 2333 1204 2006 2005 2004 2003 2302 2001

13 3011 3812 3013 3001 1306 3003 2333 3066 3212 3008 2302 3010 3011

4 6666 8432 4011 1306

4

3011 3013

6666 2001

2004 3001

2222 6666

输出样例：

2

Go by the line of company #3 from 3011 to 3013.

10

Go by the line of company #4 from 6666 to 1306.

Go by the line of company #3 from 1306 to 2302.

Go by the line of company #2 from 2302 to 2001.

6

Go by the line of company #2 from 2004 to 1204.

Go by the line of company #1 from 1204 to 1306.

Go by the line of company #3 from 1306 to 3001.

Sorry, no line is available.

分析：【很简单的，别跑^_^】

【很简单的，别跑^_^】

一遍DFS即可～DFS过程中要维护两个变量：minCnt-中途经停的最少的站；minTransfer-需要换乘的最小次数～

0.可以这样计算出一条线路的换乘次数：在line[10000][10000]的数组中保存每两个相邻站中间的线路是几号线～从头到尾遍历最终保存的路径，preLine为前一小段的线路编号，如果当前的结点和前一个结点组成的这条路的线路编号和preLine不同，说明有一个换乘，就将cnt+1，最后遍历完累加的cnt即是换乘的次数～

1.可以这样计算出一条线路中途停站的次数：在dfs的时候有个变量cnt，表示当前路线是所需乘的第几个站，每次dfs时候将cnt+1表示向下遍历一层～cnt就是当前中途停站的次数～

2.可以这样输出结果：和计算线路换乘次数的思路一样，每当preLine和当前Line值不同的时候就输出一句话～保存preTransfer表示上一个换乘站，最后不要忘记输出preTransfer和最后一个站之间的路即使最后一个站并不是换乘站～

喵喵喵～

```
1  #include <stdio>
2  #include <vector>
3  using namespace std;
4  vector<vector<int>>> v(10000);
5  int line[10000][10000], visit[10000];
6  vector<int> path, tempPath;
7  int transferCnt(vector<int> a) {
8      int cnt = -1, preLine = 0;
9      for (int i = 1; i < a.size(); i++) {
10         if (line[a[i-1]][a[i]] != preLine) cnt++;
11         preLine = line[a[i-1]][a[i]];
12     }
13     return cnt;
14 }
15 void dfs(int node, int end, int cnt, int &minCnt, int &minTransfer) {
16     if (node == end && (cnt < minCnt || (cnt == minCnt &&
17         transferCnt(tempPath) < minTransfer))) {
18         minCnt = cnt;
19         minTransfer = transferCnt(tempPath);
20         path = tempPath;
21     }
22     if (node == end) return;
23     for (int i = 0; i < v[node].size(); i++) {
24         if (visit[v[node][i]] == 0) {
25             visit[v[node][i]] = 1;
26             tempPath.push_back(v[node][i]);
27             dfs(v[node][i], end, cnt + 1, minCnt, minTransfer);
28             visit[v[node][i]] = 0;
29             tempPath.pop_back();
30         }
31     }
32 }
33 int main() {
34     int n, m, k, pre, temp, a, b;
35     scanf("%d", &n);
36     for (int i = 0; i < n; i++) {
37         scanf("%d%d", &m, &pre);
38         for (int j = 1; j < m; j++) {
39             scanf("%d", &temp);
40             v[pre].push_back(temp);
41             v[temp].push_back(pre);
42             line[pre][temp] = line[temp][pre] = i + 1;
43             pre = temp;
44         }
45     }
```

```

43     }
44 }
45 scanf("%d", &k);
46 for (int i = 0; i < k; i++) {
47     scanf("%d%d", &a, &b);
48     int minCnt = 99999, minTransfer = 99999;
49     tempPath.clear();
50     tempPath.push_back(a);
51     visit[a] = 1;
52     dfs(a, b, 0, minCnt, minTransfer);
53     visit[a] = 0;
54     if (minCnt == 99999) {
55         printf("Sorry, no line is available.\n");
56         continue;
57     }
58     printf("%d\n", minCnt);
59     int preLine = 0, preTransfer = a;
60     for (int j = 1; j < path.size(); j++) {
61         if (line[path[j-1]][path[j]] != preLine) {
62             if (preLine != 0) printf("Go by the line of company #%d
63 from %04d to %04d.\n", preLine, preTransfer, path[j-1]);
64             preLine = line[path[j-1]][path[j]];
65             preTransfer = path[j-1];
66         }
67     }
68     printf("Go by the line of company #%d from %04d to %04d.\n",
69 preLine, preTransfer, b);
70 }

```

L3-015. 球队“食物链” [深度优先搜索DFS]

某国的足球联赛中有N支参赛球队，编号从1至N。联赛采用主客场双循环赛制，参赛球队两两之间在双方主场各赛一场。

联赛战罢，结果已经尘埃落定。此时，联赛主席突发奇想，希望从中找出一条包含所有球队的“食物链”，来说明联赛的精彩程度。“食物链”为一个1至N的排列{ T1 T2 ... TN }，满足：球队T1战胜过球队T2，球队T2战胜过球队T3，.....，球队T(N-1)战胜过球队TN，球队TN战胜过球队T1。

现在主席请你从联赛结果中找出“食物链”。若存在多条“食物链”，请找出字典序最小的。

注：排列{ a1 a2 ...aN }在字典序上小于排列{ b1 b2 ... bN }，当且仅当存在整数K (1 ≤ K ≤ N)，满足：aK < bK且对于任意小于K的正整数i，ai=bi。

输入格式：

输入第一行给出一个整数N (2 ≤ N ≤ 20)，为参赛球队数。随后N行，每行N个字符，给出了N×N的联赛结果表，其中第i行第j列的字符为球队i在主场对阵球队j的比赛结果：“W”表示球队i战胜球队j，“L”表示球队i负于球队j，“D”表示两队打平，“-”表示无效（当i=j时）。输入中无多余空格。

输出格式：

按题目要求找到“食物链”T1 T2 ... TN，将这N个数依次输出在一行上，数字间以1个空格分隔，行的首尾不得有多余空格。若不存在“食物链”，输出“No Solution”。

输入样例1：

5

-LWDW

W-LDW

WW-LW

DWW-W

DDLW-

输出样例1：

1 3 5 4 2

输入样例2：

5

-WDDW

D-DWL

DD-DW

DDW-D

DDDD-

输出样例2：

No Solution

分析：

0.既然是字典序最小的，而且必须包含所有的球队编号，那如果存在的话1肯定是结果数组的第一位～

1.接收数据：用v[21][21]存储比赛结果，如果是W就将v[i][j]置为1，表示i战胜过j，如果是L就将v[j][i]置为1，表示j战胜过i，其余的不用管～

2.dfs(int index, int num): index从1到n用来表示当前即将写入的结果数组result的下标，num表示当前的球队的编号，从1开始。

3.flag用来标记是否已经找到一个这样的结果数组，如果已经找到了（flag == 1）就return; 否则将当前下标index的结果写入num值。

当index等于n的时候，并且v[num][1]等于true，表示已经写到了n个球员，而且这个球队num也曾经战胜过球队1～如果仅仅是index == n，就直接return不用继续向下找了～

4.cut标记剪枝，首先cut等于false，当所有没有访问过的结点中没有一个结点的v[i][1] == true，即没有一个球队战胜过1，就直接return，因为这条路径肯定找不到回路啦～

5.然后将num球队编号标记为已经访问过【visit[num] = true;】遍历num所有打败过的球队，并将index + 1，直到dfs结束的最后将visit标记回false

6.最后根据flag的值输出结果，如果flag为0说明没有满足题意的球队，就输出No Solution，如果flag = 1就输出result数组中保存的结果～

```
1  #include <stdio>
2  using namespace std;
3  int n, flag = 0;
4  int result[21];
5  bool v[21][21], visit[21];
6  void dfs(int index, int num) {
7      if (flag == 1) return;
8      result[index] = num;
9      if (index == n && v[num][1] == true) {
10         flag = 1;
11         return;
12     }
13     if (index == n) return;
14     bool cut = false;
15     for (int i = 1; i <= n; i++) {
16         if (visit[i] == false && v[i][1] == true)
17             cut = true;
18     }
19     if (cut == false) return;
20     visit[num] = true;
21     for (int i = 1; i <= n; i++) {
22         if (visit[i] == false && v[num][i] == true)
23             dfs(index + 1, i);
24     }
25     visit[num] = false;
26 }
27 int main() {
28     scanf("%d", &n);
29     char s[21];
30     for (int i = 1; i <= n; i++) {
31         scanf("%s", s + 1);
32         for (int j = 1; j <= n; j++) {
33             if (s[j] == 'W') v[i][j] = true;
34             if (s[j] == 'L') v[j][i] = true;
35         }
36     }
37     dfs(1, 1);
38     if (flag == 1) {
39         for (int i = 1; i <= n; i++)
40             printf("%s%d", i == 1 ? "" : " ", result[i]);
41     } else {
42         printf("No Solution");
43     }
```

```
44     return 0;  
45 }
```